Fundamentos de la Inteligencia Artificial

Manual de apoyo

Visión artificial aplicada al desarrollo de producto y la fabricación aditiva

Definiciones, intuiciones matemáticas, ejemplos y guía de interpretación de los seis temas del curso

Para leer y para experimentar

Este documento amplía las transparencias y acompaña los notebooks. Está pensado para lectores con Python de usuario y matemáticas elementales. Las fórmulas se explican desde su significado y sus supuestos, antes de utilizarlas.

Versión docente 1.1 · Octubre de 2026

Índice general

Cómo utilizar este manual **v**

1. Herramientas matemáticas para interpretar los modelos **1**

1.1. Números, índices y funciones 1
1.2. Vectores y matrices 2
1.3. Producto escalar, norma y distancia 4
1.4. Media, variación y desviación típica 5
1.5. Probabilidad: frecuencias y condiciones 7
1.6. Exponencial y logaritmo sin perder la intuición 9
1.7. Derivada, pendiente y gradiente 10
1.8. Taller de repaso con soluciones razonadas 12
1.9. Mapa de conexión con el resto del curso 13

2. Tema 1. Las imágenes como datos **14**

2.1. Qué representa una imagen digital 14
2.2. Dimensiones, coordenadas y memoria 15
2.3. Color, gris y pérdida de información 17
2.4. Recorte, resolución e interpolación 19
2.5. Tipos numéricos y normalización 21
2.6. Ruido, iluminación y visualización 22
2.7. Guía de trabajo con el notebook 23
2.8. Aplicación a producto y fabricación aditiva 24
2.9. Errores frecuentes y cómo reconocerlos 25
2.10. Ejercicios de repaso con soluciones razonadas 25

3. Tema 2. Aprender a clasificar imágenes **27**
3.1. Entrada, etiqueta y modelo . . . 27
3.2. Por qué necesitamos tres conjuntos . . . 28
3.3. Una referencia y un clasificador lineal . . . 30
3.4. Sigmoide, softmax y probabilidades del modelo . . . 31
3.5. Pérdida y regularización . . . 32
3.6. Redes densas, activación y aprendizaje . . . 33
3.7. Sobreajuste y límites de la selección . . . 35
3.8. CNN: compartir filtros y usar estructura local . . . 36
3.9. Aprendizaje por transferencia . . . 36
3.10. Matriz de confusión y cambio de condiciones . . . 37
3.11. Guía de trabajo con las seis actividades . . . 38
3.12. Aplicación a producto y límites de la evidencia . . . 39
3.13. Ejercicios con soluciones razonadas . . . 39

4. Tema 3. Detección y segmentación **41**
4.1. La salida depende de la pregunta . . . 41
4.2. Cajas y convenciones geométricas . . . 42
4.3. Un segmentador aprendido muy sencillo . . . 43
4.4. Limpieza y conectividad . . . 45
4.5. IoU de máscaras y cajas . . . 46
4.6. Por qué una máscara buena puede contar mal . . . 48
4.7. Evaluar detecciones sin contar dos veces . . . 48
4.8. Modelos profundos: qué añaden . . . 50
4.9. De píxeles cuadrados a medidas físicas . . . 50
4.10. Guía de trabajo con las seis actividades . . . 51
4.11. Errores frecuentes y aplicaciones a producto . . . 52
4.12. Ejercicios con soluciones razonadas . . . 52

5. Tema 4. Inspección y detección de anomalías **54**
5.1. Normalidad estadística y conformidad . . . 54
5.2. Preparación y región de interés . . . 55
5.3. Referencia por posición y mapa de desviación . . . 56
5.4. Del mapa a la puntuación de una pieza . . . 58
5.5. Umbral y matriz de decisiones . . . 59
5.6. Resultado: el umbral ordena un compromiso . . . 60
5.7. Resultado: frecuencia de defectos y precisión . . . 61
5.8. Mapas, reconstrucción y características profundas . . . 62
5.9. Diagnóstico de captura e incertidumbre . . . 63
5.10. Guía de trabajo con las seis actividades . . . 64
5.11. Errores frecuentes y aplicación a fabricación . . . 65
5.12. Ejercicios con soluciones razonadas . . . 65

6. Tema 5. Modelos multimodales y generativos **67**
6.1. Representar imagen y texto en un espacio común . . . 67
6.2. Similitud y recuperación . . . 69
6.3. Clasificar sin ajuste y el efecto de las candidatas . . . 70
6.4. Idioma, negación y cantidad . . . 71
6.5. Vectores guardados e inferencia en vivo . . . 72
6.6. Generación de imágenes: condicionamiento y muestreo . . . 73
6.7. El proceso directo de ruido y sus supuestos . . . 74
6.8. Difusión latente, guía y reproducibilidad . . . 76
6.9. De la instrucción al concepto de producto . . . 77
6.10. Guía de trabajo con las seis actividades . . . 78
6.11. Errores frecuentes y aplicaciones a desarrollo de producto . . . 79
6.12. Ejercicios con soluciones razonadas . . . 79

7. Tema 6. Del problema a la evidencia **81**
7.1. Formular una pregunta comprobable . . . 81
7.2. Diseño de una comparación . . . 82
7.3. Cuatro rutas y evidencia mínima . . . 84
7.4. Cierre de decisiones y reproducibilidad . . . 86
7.5. Informar sin exagerar . . . 87
7.6. Evaluación del proyecto y defensa . . . 88
7.7. Organización del taller de 150 minutos . . . 90
7.8. Guía de trabajo con las seis actividades . . . 90
7.9. Ejercicios con soluciones razonadas . . . 91
7.10. Cierre del curso: elegir, comprobar y comunicar . . . 92

A. Notación y glosario de consulta **93**

B. Ejercicios de comprensión **94**

C. Soluciones comentadas **95**

D. Lecturas y trazabilidad del material **97**
D.1. Lecturas para ampliar . . . 97
D.2. Correspondencia de archivos . . . 97

Cómo utilizar este manual

El objetivo del curso es aprender a formular problemas con imágenes, elegir una familia de técnicas y juzgar los resultados. La programación permite observar fenómenos y comprobar hipótesis. Una ejecución correcta es necesaria, pero no demuestra por sí sola que un sistema resuelva el problema de producto que nos interesa.

El capítulo inicial reúne las herramientas matemáticas que después se reutilizan. No es necesario memorizarlo entero: puede consultarse al encontrar una fórmula. Los seis capítulos siguientes corresponden a los seis temas. Los recuadros de ejemplo desarrollan cálculos sencillos y los de conexión señalan qué parte del notebook permite comprobar la explicación. Los apéndices ofrecen notación, ejercicios y soluciones.

Tres preguntas ante cualquier fórmula

¿Qué representa cada símbolo? ¿Qué operación estamos realizando? ¿Bajo qué supuestos tiene sentido interpretar su resultado? Si falta alguna respuesta, conviene volver al ejemplo antes de continuar.

Qué se ejecuta y qué se explica conceptualmente

Tema	Ejecución principal	Ampliación conceptual
1	Matrices de imagen y transformaciones	Adquisición y preparación de entradas
2	Regresión logística y red densa pequeña	CNN y aprendizaje por transferencia
3	Clasificación RGB por píxel y componentes	Detectores y segmentación profunda
4	Referencia estadística de piezas correctas	Autoencoders y características profundas
5	Vectores reales de CLIP guardados; ruido directo; revisión de renders	Difusión y modelos conversacionales visuales
6	Proyecto; ejemplo ejecutable de clasificación	Elección y evaluación de una solución

En el tema 5 hay además un notebook opcional que ejecuta CLIP en vivo. La ruta principal puede utilizar sus vectores previamente calculados. Añadir ruido a una imagen conocida no equivale a ejecutar un generador aprendido. Los conceptos visuales A y B sí proceden de generación previa con la herramienta integrada de imágenes; su modelo exacto y su semilla no fueron expuestos. No atribuimos a esa herramienta una arquitectura concreta.

Las imágenes de las primeras prácticas son sintéticas. Esto permite conocer etiquetas y controlar algunos factores; sus resultados no acreditan rendimiento industrial. Un render convincente tampoco es una pieza definida geométricamente, ni sustituye ensayos, tolerancias o preparación de fabricación.

Modelo es una regla parametrizada que transforma entradas en salidas; *entrenamiento*, el procedimiento para ajustar sus parámetros; *inferencia*, su aplicación una vez fijados. Una *métrica* resume un aspecto del resultado. Un *criterio de aceptación* expresa qué resultado sería suficiente para una decisión concreta. Son conceptos relacionados, pero diferentes.

Capítulo 1

Herramientas matemáticas para interpretar los modelos

Qué aprender y cómo recorrer el capítulo

Al terminar deberías poder leer la forma de un conjunto de imágenes, calcular una puntuación ponderada, interpretar una desviación y distinguir dos probabilidades condicionadas. También deberías poder explicar por qué un algoritmo modifica sus pesos al entrenar. No hace falta dominar cálculo diferencial ni demostrar todos los resultados.
Una primera lectura puede seguir las explicaciones y los ejemplos. Los apartados titulados «Para profundizar» ofrecen justificaciones que pueden reservarse para una segunda lectura. Las comprobaciones en Python son pequeñas celdas independientes: basta disponer de NumPy, incluido en el entorno del curso y en Colab.

El hilo conductor será una inspección visual de piezas. Podemos medir intensidad, color, área o una representación aprendida. Las matemáticas permiten organizar esos números, compararlos y expresar la incertidumbre de una decisión. Ninguna operación convierte por sí sola una imagen en una medición física válida: siguen siendo necesarios una adquisición adecuada y un problema bien definido.

1.1. Números, índices y funciones

Una variable numérica representa una cantidad. Por ejemplo, $x = 125$ puede ser la intensidad de un canal de un píxel. Una función aplica una regla a una entrada: $f(x) = x/255$ cambia la escala numérica. Evaluar $f(125)$ significa sustituir x por 125 y efectuar la operación: $125/255 \approx 0{,}490$.

Los subíndices identifican posiciones o ejemplos. En x_i, la letra i no multiplica a x: indica qué elemento seleccionamos. En x_{ij}, los índices pueden referirse a fila y columna. Es importante conocer el rango de índices. Las fórmulas matemáticas suelen empezar en 1; NumPy suele empezar en 0. Ninguna opción cambia el contenido si se mantiene la convención.

El símbolo $\sum$ abrevia una suma. Así, $\sum_{i=1}^{3} x_i = x_1 + x_2 + x_3$. Un promedio divide la suma entre el número de elementos. Si los valores son 2, 4 y 9, la media es $(2 + 4 + 9)/3 = 5$. Que la media sea 5 no implica que alguno de los valores originales sea 5.

1.1.1. Una guía para leer una expresión

En matemáticas, una igualdad afirma que dos expresiones tienen el mismo valor; una aproximación, indicada por $\approx$, admite redondeo u otra simplificación. En $x \in \mathbb{R}$, el símbolo $\in$ significa «pertenece a» y $\mathbb{R}$ designa los números reales. Un resultado negativo es perfectamente válido para una diferencia o una puntuación, aunque no lo sea para un número de píxeles.

Conviene distinguir tres papeles de las letras. Una **variable de entrada** contiene información de un ejemplo, como su intensidad media x. Un **parámetro** pertenece al modelo, como un peso w que se ajusta al entrenar. Un **hiperparámetro** configura el procedimiento, como el tamaño de paso del entrenamiento. La distinción depende del contexto: no de que se use una

letra griega o latina.

Ejemplo: Leer una media sin saltarse pasos

En $\overline{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$, primero identificamos las n observaciones. Después sumamos sus valores y dividimos entre n. Para cuatro intensidades $20, 40, 60, 80$, obtenemos $\overline{x} = 200/4 = 50$. La barra de $\overline{x}$ es parte de la notación de la media; no significa negación.

Una **función** necesita un dominio de entradas admisibles. La regla $f(x) = 1/x$ no está definida en cero. La regla $g(x) = \sqrt{x}$, con resultados reales, necesita $x \geq 0$. Estas restricciones reaparecen al dividir por una desviación, normalizar un vector o calcular un logaritmo.

Componer funciones significa aplicar una después de otra: $(g \circ f)(x) = g(f(x))$. Si $f(x) = x/255$ y $g(t) = t - 0{,}5$, entonces $g(f(128)) \approx 0{,}002$. Invertir el orden da $f(g(128)) = 127{,}5/255 = 0{,}5$. El preprocesamiento tiene un orden y cambiarlo puede cambiar la entrada del modelo.

1.1.2. Unidades, proporciones y porcentajes

Una proporción de 0,08 corresponde al 8 %. Pasar de un acierto del 80 % al 88 % supone ocho puntos porcentuales; el aumento relativo respecto al valor inicial es $8/80 = 10\,\%$. Conviene indicar cuál se está comunicando.

Las unidades también forman parte del cálculo. Sumar milímetros a intensidades carece de interpretación directa. Dividir un área en píxeles cuadrados por una escala en píxeles por milímetro no produce milímetros cuadrados: hay que dividir por el *cuadrado* de esa escala. Si hay 10 píxeles por mm y una región ocupa 400 píxeles cuadrados, su área estimada es $400/10^2 = 4\,\mathrm{mm}^2$, bajo una escala plana uniforme.

1.2. Vectores y matrices

Un **vector** es una lista ordenada de números. El color de un píxel RGB puede representarse como $x = (55, 125, 190)$. Una matriz organiza valores en filas y columnas; una imagen en gris de 240 filas y 320 columnas es una matriz $I \in \mathbb{R}^{240\times320}$. Una imagen RGB añade un eje de canales: $I \in \mathbb{R}^{240\times320\times3}$.

La **dimensión** indica cuántas componentes tiene un vector. Un vector de CLIP de 512 componentes no corresponde a 512 colores ni a 512 categorías con nombre. Es una representación aprendida cuyas coordenadas contribuyen conjuntamente a una comparación.

Ejemplo: Aplanar conserva números, pero cambia la organización

Una imagen en gris de 32×32 contiene 1024 valores. Podemos escribirlos en una lista de longitud 1024. La lista conserva los valores y su orden, de modo que podemos reconstruir la matriz si conocemos la forma. Sin embargo, una capa densa recibe posiciones como variables distintas; no incorpora por sí sola el supuesto de que ciertos elementos son vecinos de una imagen.

Sumar vectores o multiplicarlos por un número se hace componente a componente. Si $u = (1, 2)$ y $v = (3, 4)$, entonces $u + v = (4, 6)$ y $2u = (2, 4)$. Sumar un brillo constante a todos los píxeles puede representarse mediante una operación sobre todos los elementos de una matriz, siempre que el tipo numérico y el rango permitan efectuarla.

1.2.1. Ejes, lotes y tensores

En este curso llamamos **tensor** a un arreglo numérico con varios ejes. Un escalar no necesita un eje; un vector tiene uno; una matriz, dos. Una imagen RGB añade un tercero para los canales. Un lote de 20 imágenes RGB de 32×32 puede tener forma $(20, 32, 32, 3)$. Su primer eje identifica la imagen, y los restantes identifican fila, columna y canal. Algunas bibliotecas colocan los canales antes de las dimensiones espaciales: la forma debe interpretarse según su convención.

El término *dimensión* puede referirse al número de ejes o al tamaño de un espacio de vectores. Una imagen gris tiene dos ejes, pero al aplanarla puede representar un vector con 1024 componentes. Es mejor decir «dos ejes» y «1024 componentes» cuando pueda haber confusión.

Ejemplo: Aplanar y recuperar una imagen pequeña

Consideremos

$$I = \begin{pmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{pmatrix}.$$

Leyendo por filas, el vector es $(10, 20, 30, 40, 50, 60)$. Conocer que había seis elementos no basta para recuperar la forma original: podría haber sido 1×6, 2×3 o 3×2. Hay que guardar la forma y el orden utilizados.

1.2.2. Operar por componentes y combinar componentes

El producto componente a componente de $(1, 2)$ y $(3, 4)$ produce $(3, 8)$. El producto escalar, que veremos enseguida, suma esos productos y produce el número 11. Son operaciones distintas. En NumPy, para arreglos de estas formas, se escriben con `*` y `@`, respectivamente.

El **broadcasting** permite ciertas operaciones entre formas diferentes. Por ejemplo, restar un vector RGB de forma `(3,)` a una imagen de forma `(alto,ancho,3)` resta una referencia por canal a cada píxel. Una forma compatible para el programa puede seguir siendo equivocada para nuestra intención: hay que comprobar qué eje se está comparando.

1.2.3. Comprobación en Python: forma y ejes

```
import numpy as np
I = np.array([[10., 20., 30.], [40., 50., 60.]])
print(I.shape, I.size)          # (2, 3), 6
print(I[0, 2])                  # 30.0
print(I.mean(axis=0))           # [25. 35. 45.]
print(I.mean(axis=1))           # [20. 50.]
v = I.reshape(-1)
print(v)
print(np.array_equal(v.reshape(I.shape), I))  # True
```

La media con `axis=0` elimina el eje de filas y conserva un resultado por columna. La media con `axis=1` elimina las columnas y conserva un resultado por fila. Antes de ejecutar, predice cuántos valores debe producir cada operación.

1.3. Producto escalar, norma y distancia

El **producto escalar** combina dos vectores de la misma dimensión:

$$u \cdot v = \sum_{i=1}^{d} u_i v_i. \tag{1.1}$$

Para $u = (1, 2)$ y $v = (3, 4)$ vale $1 \cdot 3 + 2 \cdot 4 = 11$. Aparece en los clasificadores: los pesos multiplican entradas y sus contribuciones se suman.

La **norma euclídea** mide la longitud de un vector: $\|u\| = \sqrt{\sum_i u_i^2}$. Para $(3, 4)$ la norma es 5. La **distancia euclídea** entre u y v es $\|u - v\|$. Una distancia pequeña significa proximidad respecto a esa representación; no demuestra similitud funcional de dos productos.

Normalizar un vector no nulo consiste aquí en dividirlo por su norma: $\widehat{u} = u/\|u\|$. Su longitud pasa a ser 1. No debe confundirse esta normalización con dividir intensidades por 255, ni con estandarizar una variable mediante media y desviación.

Por qué el coseno queda entre -1 y 1

La desigualdad de Cauchy–Schwarz establece $|u \cdot v| \leq \|u\|\|v\|$. Una intuición es que la proyección de un vector sobre la dirección de otro no puede superar su longitud. Algebraicamente, la suma de cuadrados $\|u - tv\|^2$ nunca es negativa para ningún t; su valor mínimo conduce a esa desigualdad. Para vectores no nulos, dividir por las normas permite definir un coseno entre -1 y 1.

Una matriz también puede representar una transformación. Si W tiene una fila por clase y una columna por entrada, Wx produce una puntuación por clase. Cada fila efectúa un producto escalar con x. Esto conecta la notación compacta de álgebra lineal con las sumas de una red neuronal.

1.3.1. Una puntuación es una suma de contribuciones

En $s = w \cdot x + b$, el vector x contiene características; w, sus pesos; y b, el sesgo o término independiente. Para $x = (0{,}2, 0{,}8)$, $w = (2, -1)$ y $b = 0{,}5$, las contribuciones son 0,4 y $-0{,}8$. Con el sesgo, la puntuación vale 0,1. Un peso negativo hace disminuir la puntuación cuando aumenta esa entrada, manteniendo las demás fijas. No demuestra una relación causal entre esa característica y el defecto.

Si calculamos dos puntuaciones a la vez, podemos escribir

$$W = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}, \quad x = \begin{pmatrix} 0{,}2 \\ 0{,}8 \end{pmatrix}, \quad b = \begin{pmatrix} 0{,}5 \\ 0 \end{pmatrix}, \quad Wx + b = \begin{pmatrix} 0{,}1 \\ 1{,}4 \end{pmatrix}.$$

Las dimensiones deben encajar: una matriz $K \times d$ multiplica un vector de d componentes y produce K puntuaciones. Aquí $K = 2$ y $d = 2$. El sesgo necesita también K componentes.

1.3.2. Distancia, escala y similitud

La norma de una diferencia mide separación en las coordenadas elegidas. Si comparamos área en píxeles y brillo entre 0 y 1, diferencias de cientos de píxeles pueden dominar la distancia aunque el brillo sea relevante. Cambiar las unidades cambia la distancia numérica. Escalar características es una decisión sobre cómo las comparamos; no asegura que todas sean igual de útiles.

El **coseno** compara direcciones:

$$\cos(u, v) = \frac{u \cdot v}{\|u\|\|v\|}.$$

Para $u = (3, 4)$ y $v = (6, 8)$, la distancia es 5, pero el coseno es 1: tienen la misma dirección y diferente longitud. Para $(1, 0)$ y $(0, 1)$ vale 0; para $(1, 0)$ y $(-1, 0)$ vale -1. En representaciones aprendidas, un coseno alto indica proximidad según esa representación. Su significado práctico debe comprobarse con ejemplos de la tarea.

Para profundizar: distancia entre vectores unitarios

Al desarrollar el cuadrado de la distancia,

$$\|u - v\|^2 = \|u\|^2 + \|v\|^2 - 2u \cdot v.$$

Si ambos vectores tienen norma 1, resulta $\|u - v\|^2 = 2 - 2\cos(u, v)$. Por tanto, ordenar candidatos normalizados por mayor coseno equivale a ordenarlos por menor distancia euclídea. La equivalencia necesita esa normalización y no convierte ninguno de los resultados en probabilidad.

El vector cero no tiene dirección y no puede dividirse por su norma. Añadir una pequeña constante al denominador puede evitar un error numérico, pero el resultado de un vector nulo no adquiere por ello una dirección significativa.

1.3.3. Comprobación en Python: dos operaciones y una semejanza

```
u = np.array([3., 4.])
v = np.array([6., 8.])
print(u * v)                          # [18. 32.]
print(u @ v)                          # 50.0
print(np.linalg.norm(u - v))          # 5.0
un = u / np.linalg.norm(u)
vn = v / np.linalg.norm(v)
print(un @ vn)                        # aproximadamente 1
W = np.array([[2., -1.], [-1., 2.]])
x = np.array([0.2, 0.8])
print(W @ x + np.array([0.5, 0.]))    # [0.1 1.4]
```

En el tema 5 se calcula una tabla de similitudes entre imágenes y textos. Si hay N imágenes, M textos y vectores de d componentes, la operación combina una matriz $N \times d$ con otra $d \times M$. El resultado tiene forma $N \times M$: una comparación para cada pareja.

1.4. Media, variación y desviación típica

Para observaciones $x_1, \ldots, x_n$, definimos su media muestral como

$$\overline{x} = \frac{1}{n} \sum_{i=1}^{n} x_i. \tag{1.2}$$

Las diferencias $x_i - \overline{x}$ pueden ser positivas o negativas y suman cero. Si queremos medir dispersión sin que se cancelen, podemos elevarlas al cuadrado. La varianza muestral habitual y su desviación son

$$s^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \overline{x})^2, \qquad s = \sqrt{s^2}, \quad n > 1. \tag{1.3}$$

La raíz devuelve la unidad original. Una desviación de 3 intensidades es más interpretable que una varianza de 9 intensidades al cuadrado.

Ejemplo: Tres mediciones

Para 10, 12 y 14, la media es 12. Las desviaciones son $-2, 0, 2$ y sus cuadrados suman 8. Con divisor $n-1=2$, la varianza muestral es 4 y la desviación es 2. Con divisor $n=3$, la media de cuadrados sería $8/3$. Ambas operaciones existen: hay que saber qué convención se está utilizando.

El divisor $n-1$ aparece porque la media se ha estimado a partir de los mismos datos. Bajo observaciones independientes con igual distribución y varianza finita, corrige el sesgo de la estimación de varianza. No convierte datos dependientes en independientes ni garantiza una buena estimación con pocas piezas.

Una **estandarización** transforma $z=(x-\mu)/\sigma$, con $\sigma>0$. Expresa alejamiento en unidades de la desviación de referencia. Si las mediciones fueran normales o gaussianas, podríamos relacionar ciertos valores con probabilidades de esa distribución. Sin ese supuesto, $z=3$ no equivale por sí mismo a una probabilidad universal de defecto.

1.4.1. Mediana, valores extremos y percentiles

La **mediana** es el valor central tras ordenar las observaciones; con un número par de datos usamos aquí la media de los dos centrales. Para $10, 11, 12, 13, 100$, la media es 29,2 y la mediana es 12. El valor 100 tira mucho de la media. La mediana resiste mejor algunos valores extremos, pero no revela si ese extremo es un error de medida o un defecto importante.

Un **percentil** sitúa un valor dentro de una lista ordenada. El percentil 95 señala aproximadamente el nivel por debajo del cual queda el 95 % de los datos. En muestras pequeñas las bibliotecas pueden interpolar de maneras distintas. La media del 1 % de valores mayores, empleada como puntuación en el tema 4, es una operación diferente del percentil 99: selecciona una cola y promedia sus valores.

1.4.2. Qué se promedia: píxeles, imágenes o piezas

El eje de cálculo determina qué pregunta respondemos. La media de todos los píxeles de una imagen resume su intensidad global. La media de una posición fija a través de muchas imágenes resume el comportamiento de esa posición. Esta última comparación necesita que las imágenes estén alineadas: de lo contrario se mezclan posiciones físicas distintas.

Ejemplo: Por qué repetir capturas no crea piezas independientes

Supongamos diez piezas fotografiadas veinte veces cada una. Hay 200 imágenes, pero solo diez piezas distintas. Las capturas de una misma pieza comparten geometría y estado superficial; pueden compartir además iluminación. Al evaluar nuevas piezas, tratarlas como 200 unidades independientes puede exagerar la información disponible. La separación de conjuntos y el análisis de incertidumbre deben respetar esa estructura.

1.4.3. Tres transformaciones que conviene distinguir

Operación	Cálculo	Interpretación
Reescalar intensidades	$x/255$	Cambiar la escala de 0–255 a 0–1.
Estandarizar	$(x-\mu)/\sigma$	Medir alejamiento de una referencia.
Normalizar un vector	$x/\|x\|$	Conservar dirección con longitud 1.

No son sustitutos universales. La primera presupone el rango de intensidades indicado; la segunda necesita una referencia y escala válidas; la tercera necesita un vector no nulo.

Para valores de entrenamiento $10, 12, 14$, usar la media 12 y desviación muestral 2 transforma un nuevo valor 16 en $z = 2$. No se recalculan la media y la desviación incorporando ese nuevo valor si queremos aplicar la referencia aprendida. Tampoco se estiman con prueba al entrenar un modelo.

1.4.4. La convención de la biblioteca importa

```
a = np.array([10., 12., 14.])
print(a.mean(), np.median(a))       # 12.0, 12.0
print(a.std(ddof=0))                # aproximadamente 1.633
print(a.std(ddof=1))                # 2.0
print((16 - a.mean()) / a.std(ddof=1))  # 2.0
```

El argumento `ddof=1` da el divisor $n-1$. El tema 4 lo utiliza explícitamente. El escalador `StandardScaler` del tema 2 utiliza la convención con divisor n al estimar la varianza. Ambas convenciones se interpretan a partir del procedimiento aplicado; no se debe atribuir a todos los escaladores la misma fórmula sin comprobarlo.

Si una variable es constante, su desviación es cero. En el tema 4 se impone un mínimo de 0,025 a la desviación para evitar divisiones por valores muy pequeños. Es una elección del método con efecto sobre las puntuaciones, no una desviación observada ni una constante matemática universal.

1.5. Probabilidad: frecuencias y condiciones

Una probabilidad $P(A)$ cuantifica la posibilidad de un suceso bajo un modelo. Una frecuencia observada es una fracción en un conjunto concreto. Si aparecen 5 defectos entre 100 piezas, la frecuencia es $5/100 = 0{,}05$. Esa observación no prueba que todas las futuras tandas tengan exactamente el 5 % de defectos.

La probabilidad condicionada $P(A \mid B)$ pregunta por A restringiéndonos a los casos donde ocurre B. En inspección son diferentes $P(\text{alarma} \mid \text{defecto})$ y $P(\text{defecto} \mid \text{alarma})$. La primera mira todos los defectos; la segunda mira todas las alarmas.

La regla del producto y la inversión de condiciones

Por definición, $P(A \cap B) = P(A \mid B)P(B)$. También es $P(B \mid A)P(A)$. Igualar ambas expresiones permite invertir una condición si el denominador es positivo:

$$P(B \mid A) = \frac{P(A \mid B)P(B)}{P(A)}.$$

Este es el teorema de Bayes. Su utilidad aquí es recordar que la frecuencia de defectos influye en cuántas alarmas corresponden a defectos reales.

1.5.1. Sucesos, complementos e independencia

Un **suceso** es un conjunto de resultados posibles: por ejemplo, que una pieza sea defectuosa. Su complemento A^c significa que no ocurre A, de modo que $P(A^c) = 1 - P(A)$. La intersección $A \cap B$ indica que ocurren ambos; la unión $A \cup B$, que ocurre al menos uno. La regla

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

resta los casos contados dos veces. Si los sucesos son incompatibles, su intersección está vacía. Eso es diferente de ser independientes.

Dos sucesos son **independientes** si $P(A \cap B) = P(A)P(B)$. Cuando $P(B) > 0$, equivale a que conocer B no cambie la probabilidad de A. Independencia es un supuesto sobre la relación entre resultados; no se obtiene por cambiar el orden de una tabla. Dos medidas pueden estar relacionadas porque comparten una misma pieza o una condición de captura.

Ejemplo: La misma tabla, dos denominadores

De 1000 piezas, 20 tienen defecto. El sistema detecta 16 de ellas y produce además 98 falsas alarmas entre las 980 piezas correctas. Entonces

$$P(\text{alarma} \mid \text{defecto}) \approx 16/20 = 0{,}80,$$

pero

$$P(\text{defecto} \mid \text{alarma}) \approx 16/(16 + 98) = 0{,}140.$$

Son frecuencias condicionadas en esta muestra. El sistema recupera el 80 % de defectos, mientras que aproximadamente el 14 % de alarmas corresponde a un defecto. Ninguna cifra contradice a la otra: responden a preguntas diferentes.

La probabilidad total permite calcular el denominador de Bayes. Si D indica defecto y A alarma,

$$P(A) = P(A \mid D)P(D) + P(A \mid D^c)P(D^c).$$

Separamos las alarmas en dos grupos que no se solapan: las procedentes de defectos y las procedentes de piezas correctas. En el ejemplo, $0{,}8 \cdot 0{,}02 + 0{,}1 \cdot 0{,}98 = 0{,}114$. Bayes da $0{,}016/0{,}114 \approx 0{,}140$.

1.5.2. Variable aleatoria y distribución

Una **variable aleatoria** asigna un número al resultado de un proceso incierto. Puede ser el número de defectos de una pieza o la intensidad medida bajo una captura variable. Su **distribución** describe qué valores son posibles y con qué probabilidades. Un histograma resume observaciones; una distribución es un modelo, que podemos contrastar con ellas.

La distribución normal o gaussiana tiene forma de campana y se describe mediante media μ y desviación $\sigma > 0$. Es útil para modelar algunas fuentes de ruido. No todo histograma es normal, y aplicar una estandarización no lo convierte en normal. Bajo una normal, aproximadamente el 68 % cae en $\mu \pm \sigma$ y el 95 % en $\mu \pm 2\sigma$; esas proporciones no son reglas para cualquier conjunto de intensidades.

1.5.3. Esperanza, varianza y una regla útil para el ruido

Para una variable con valores discretos x_j y probabilidades p_j, su **esperanza** es $\mathbb{E}[X] = \sum_j p_j x_j$: una media ponderada por probabilidades. No tiene por qué ser un valor posible. Si X vale 0 o 1 con igual probabilidad, su esperanza es 0,5. La media muestral estima una media poblacional bajo supuestos adecuados; no son la misma entidad.

La varianza poblacional es $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$. Si a, b son constantes,

$$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b, \qquad \mathrm{Var}(aX + b) = a^2 \mathrm{Var}(X).$$

Sumar una constante desplaza todos los valores sin cambiar su dispersión. Multiplicar por 2 duplica las distancias a la media y multiplica la varianza por 4. La desviación se multiplica por $|a|$.

Para profundizar: sumar ruido

Si X y Z tienen varianza finita y son independientes, $\mathrm{Var}(X+Z) = \mathrm{Var}(X) + \mathrm{Var}(Z)$. Sin independencia aparece un término adicional que depende de su relación. Si una intensidad fija x recibe ruido ϵ de media cero y desviación 0,04, entonces $x + \epsilon$ tiene media x y desviación 0,04. Al recortar después el resultado al intervalo $[0, 1]$, estas propiedades ya no se conservan en general, especialmente cerca de los límites.

Estas reglas explican los experimentos de ruido del tema 1 y la combinación de señal y ruido del tema 5. La forma gaussiana del ruido es un supuesto de esas simulaciones; no una consecuencia de que el dato sea una imagen.

1.6. Exponencial y logaritmo sin perder la intuición

La exponencial e^x es siempre positiva. Si aumentamos x, aumenta su valor. Además, $e^{a+b} = e^a e^b$. El logaritmo natural $\log x$, definido para $x > 0$, es su operación inversa: $\log(e^x) = x$. En aprendizaje, $-\log p$ penaliza mucho asignar una probabilidad muy pequeña a una etiqueta correcta.

No necesitamos calcular exponentes a mano para interpretar softmax. Nos basta saber que convierte diferencias de puntuación en pesos positivos y que el denominador los hace sumar uno. El capítulo del tema 2 desarrolla el cálculo completo.

1.6.1. El logaritmo convierte productos en sumas

Para $a, b > 0$, $\log(ab) = \log a + \log b$. Esto es útil cuando un modelo multiplica muchas probabilidades: el producto puede hacerse muy pequeño y el logaritmo permite trabajar con una suma. Interpretar el producto como probabilidad conjunta requiere los supuestos de independencia o de factorización condicional correspondientes.

Si $p = 1$, $-\log p = 0$. Si $p = 0{,}5$, vale aproximadamente 0,693; si $p = 0{,}1$, 2,303; y si $p = 0{,}01$, 4,605. Acercarse a cero aumenta la penalización sin límite. Un cero exacto no admite logaritmo:

las implementaciones emplean formulaciones numéricamente estables o protecciones que deben interpretarse según la biblioteca.

1.6.2. Una media ponderada necesita pesos adecuados

Si queremos combinar valores sin salir del intervalo comprendido entre su mínimo y su máximo, podemos usar pesos no negativos que sumen uno. Por ejemplo, $0{,}25 \cdot 20 + 0{,}75 \cdot 60 = 50$. Eso explica tanto una interpolación sencilla como la normalización de pesos positivos. Los pesos de una capa neuronal no tienen por qué ser no negativos ni sumar uno; por eso su puntuación no es automáticamente una media ni una probabilidad.

1.7. Derivada, pendiente y gradiente

La derivada mide cómo cambia una función al modificar ligeramente su entrada. Una pendiente positiva significa que aumentar la entrada aumenta localmente la salida. Para $f(w) = (w-3)^2$, la derivada es $f'(w) = 2(w-3)$. A la izquierda de 3 es negativa; a la derecha, positiva; en 3 es cero y la función alcanza su mínimo.

Para una función de muchos parámetros, el **gradiente** reúne las derivadas respecto a cada uno. Si el gradiente no es nulo, moverse una pequeña distancia en su dirección opuesta reduce localmente una función diferenciable cuando el paso es suficientemente pequeño. Esto no garantiza encontrar el mejor mínimo global de una red.

Ejemplo: Una actualización de aprendizaje

Con pérdida $L(w) = (w-3)^2$, partimos de $w = 0$. El gradiente es -6. Si el tamaño de paso es $\eta = 0{,}1$, la actualización $w_{\text{nuevo}} = w - \eta L'(w)$ da 0,6. La pérdida baja de 9 a 5,76. Un paso excesivo puede sobrepasar el mínimo y aumentar la pérdida.

La **regla de la cadena** permite derivar funciones compuestas. Si una salida depende de una capa, que depende de otra, su variación se obtiene combinando las sensibilidades intermedias. La retropropagación organiza ese cálculo eficientemente; no es un segundo modelo que conoce las respuestas.

1.7.1. De una diferencia finita a una pendiente local

Podemos aproximar una derivada comparando dos evaluaciones próximas:

$$f'(w) \approx \frac{f(w+h) - f(w)}{h}, \quad h \neq 0.$$

Para $f(w) = (w-3)^2$, en $w = 0$ y $h = 0{,}01$ obtenemos $(8{,}9401 - 9)/0{,}01 = -5{,}99$, próximo a -6. La derivada describe el límite de esa pendiente cuando el desplazamiento tiende a cero. No es una predicción exacta para cambios grandes. En un ordenador, hacer h extremadamente pequeño tampoco garantiza mayor precisión, debido al redondeo.

1.7.2. Dos parámetros y una actualización

Si $L(w_1, w_2) = (w_1 - 3)^2 + 2(w_2 + 1)^2$, las derivadas parciales son $2(w_1 - 3)$ y $4(w_2 + 1)$. Cada una estudia el cambio de un parámetro manteniendo fijo el otro. El gradiente es

$$\nabla L = \big(2(w_1 - 3), 4(w_2 + 1)\big).$$

Desde $(0, 0)$ vale $(-6, 4)$. Con $\eta = 0{,}1$, restarlo multiplicado por el paso conduce a $(0{,}6, -0{,}4)$. La pérdida pasa de 11 a 6,48. Los parámetros se actualizan usando el gradiente evaluado en el punto de partida.

Para profundizar: por qué se resta el gradiente

Una función diferenciable satisface localmente

$$L(\theta + \Delta) \approx L(\theta) + \nabla L(\theta) \cdot \Delta.$$

Si elegimos $\Delta = -\eta \nabla L(\theta)$ con $\eta > 0$, el término de cambio aproximado es $-\eta \|\nabla L(\theta)\|^2$, negativo cuando el gradiente no es cero. La aproximación explica un paso suficientemente pequeño; no garantiza descenso para cualquier paso. Además, un gradiente cero puede corresponder a un mínimo, un máximo o un punto de silla.

1.7.3. Regla de la cadena con un modelo mínimo

Tomemos una entrada fija $x = 2$, una predicción $\widehat{y} = wx$ y una etiqueta $y = 6$. La pérdida $L = (\widehat{y} - y)^2$ depende de w a través de $\widehat{y}$. La cadena da

$$\frac{dL}{dw} = \frac{dL}{d\widehat{y}} \frac{d\widehat{y}}{dw} = 2(\widehat{y} - y)x.$$

Si $w = 1$, la predicción vale 2 y la derivada vale $2(2 - 6)2 = -16$. Con paso 0,1, el nuevo peso es 2,6, la predicción pasa a 5,2 y la pérdida baja de 16 a 0,64. En una red grande se repite esta idea a través de muchas operaciones. La diferenciación automática calcula derivadas del programa; no decide por sí sola qué tarea o pérdida conviene utilizar.

1.7.4. Entrenar con ejemplos y comprobar fuera de ellos

La función que se minimiza suele promediar pérdidas de entrenamiento. El gradiente de una media es la media de los gradientes, cuando las funciones son diferenciables. Un lote usa una parte de los ejemplos para estimar una dirección de actualización. Por eso la pérdida puede fluctuar entre pasos; una fluctuación aislada no demuestra que el algoritmo esté fallando.

Reducir la pérdida de entrenamiento significa ajustar mejor ese objetivo en esos datos. No demuestra que mejore el rendimiento en piezas nuevas. La validación, la separación por pieza y la prueba final responden a esa segunda pregunta. Tampoco es obligatorio optimizar directamente la métrica que se presenta al usuario: una tasa de acierto cambia a saltos y suele entrenarse con otra función más manejable.

1.7.5. Comprobación en Python: aprender un peso

```
x, y = 2.0, 6.0
w, paso = 1.0, 0.1
for t in range(5):
    prediccion = w * x
    perdida = (prediccion - y)**2
    print(t, round(w, 4), round(perdida, 4))
    gradiente = 2 * (prediccion - y) * x
    w = w - paso * gradiente
```

Prueba después con paso 0,3, reiniciando $w = 1$. En este ejemplo cuadrático, el error respecto al óptimo $w = 3$ se multiplica en cada actualización por $1 - 8\eta$. Con $\eta = 0{,}1$ ese factor es 0,2; con 0,3 es $-1{,}4$: el error cambia de signo y crece. Este límite concreto corresponde a este ejemplo; no es una regla general para todas las redes.

1.8. Taller de repaso con soluciones razonadas

Resuelve primero sin ejecutar Python. Después comprueba el cálculo y escribe una frase que explique qué permite concluir. Estos ejercicios complementan los de los apéndices.

1. Forma y selección

Un lote tiene forma $(12, 24, 32, 3)$. ¿Cuántos valores contiene una imagen? ¿Qué forma tiene seleccionar una sola imagen? ¿Y seleccionar su canal rojo?

Solución. Cada imagen contiene $24 \cdot 32 \cdot 3 = 2304$ valores. Seleccionar una imagen produce forma $(24, 32, 3)$ y seleccionar después un canal produce $(24, 32)$. No se ha redimensionado la escena: se han seleccionado elementos de ejes existentes.

2. Dirección y longitud

Para $u = (1, 2)$ y $v = (2, 4)$, calcula producto escalar, normas y coseno. ¿Son el mismo vector?

Solución. El producto es 10; las normas son $\sqrt{5}$ y $2\sqrt{5}$. El coseno es $10/(\sqrt{5}\,2\sqrt{5}) = 1$. Tienen la misma dirección, pero sus componentes y longitudes son distintas. Un coseno igual a uno no implica igualdad de vectores.

3. Referencia estadística

Las intensidades de referencia son $8, 10, 12$. Un nuevo valor es 14. Calcula su desviación estandarizada usando divisor $n - 1$. ¿Puedes asignarle una probabilidad de defecto?

Solución. La media es 10; la suma de cuadrados de desviaciones es 8 y la desviación muestral es $\sqrt{8/2} = 2$. Así, $z = (14-10)/2 = 2$. El resultado expresa alejamiento respecto a la referencia. Sin un modelo de distribución, una relación validada con defectos y más información, no proporciona una probabilidad de defecto.

4. Invertir una condición

De 100 piezas, diez tienen defecto. Hay ocho alarmas verdaderas y nueve falsas alarmas. Calcula la fracción de defectos detectados y la fracción de alarmas que corresponden a defectos.

Solución. Detectamos $8/10 = 80\,\%$ de los defectos. Entre las 17 alarmas, $8/17 \approx 47{,}1\,\%$ corresponden a defectos. El primer denominador cuenta defectos; el segundo, alarmas. Hay además dos defectos omitidos y 81 piezas correctas sin alarma, de modo que los cuatro grupos suman 100.

5. Ruido y escala

Una variable X tiene media 10 y desviación 2. Para $Y = 3X + 5$, ¿cuáles son media, desviación y varianza? ¿Necesitas asumir normalidad?

Solución. La media es $3 \cdot 10 + 5 = 35$, la desviación es $|3| \cdot 2 = 6$ y la varianza es 36. Estas identidades solo necesitan que las cantidades correspondientes existan; no requieren normalidad.

6. Seguir un gradiente

Para $L(w) = (w - 4)^2$, parte de $w = 1$ y usa $\eta = 0{,}25$. Calcula una actualización y ambas pérdidas.

Solución. La derivada inicial es $2(1 - 4) = -6$. El nuevo peso vale $1 - 0{,}25(-6) = 2{,}5$. La pérdida baja de 9 a 2,25. El signo negativo de la derivada indica que aumentar w reduce localmente la pérdida; restar esa derivada hace precisamente que w aumente.

1.9. Mapa de conexión con el resto del curso

Tema 1: imágenes como datos.
Formas y ejes, unidades, promedios, reescalado y ruido. Comprueba siempre tipo numérico y rango antes de operar.

Tema 2: clasificación.
Productos escalares, matrices, estandarización, exponenciales, logaritmos y gradientes. Los parámetros del preprocesamiento se aprenden con entrenamiento.

Tema 3: detección y segmentación.
Índices, máscaras, proporciones y áreas. Cuenta qué conjunto de píxeles constituye el numerador y cuál el denominador de una métrica.

Tema 4: anomalías.
Mediana, media, desviación, colas de valores y probabilidades condicionadas. Un alejamiento estadístico no equivale automáticamente a defecto.

Tema 5: multimodalidad y generación.
Vectores, coseno, productos de matrices y combinación de señal y ruido. Similitud y probabilidad tienen significados diferentes.

Tema 6: proyecto.
Unidad de observación, independencia, separación de conjuntos y evaluación. Una cifra necesita denominador, procedimiento y contexto para sostener una conclusión.

Conexión con las prácticas

Las celdas de este capítulo son comprobaciones breves que se pueden copiar a un notebook vacío. Tras completarlas, vuelve al notebook de cada tema e identifica una operación conocida: un eje que se promedia, una referencia que se resta, un producto que combina características o un cociente que define una métrica. Explica con palabras qué representa antes de interpretar el número obtenido.

Capítulo 2

Tema 1. Las imágenes como datos

Objetivos y recorrido de lectura

Al terminar este capítulo deberías poder explicar qué números contiene una imagen, localizar una región, elegir una representación de color y anticipar qué puede perderse al transformar los datos. También deberías poder distinguir una imagen visualmente agradable de una captura útil para una tarea de inspección.
El recorrido sigue las seis actividades del notebook: lectura, coordenadas, color, resolución, escala numérica y perturbaciones. Los ejemplos adicionales se pueden ejecutar en una copia de trabajo. Las explicaciones de adquisición y muestreo dan contexto; no requieren disponer de una cámara ni entrenar un modelo.

2.1. Qué representa una imagen digital

Una imagen es una representación de una captura. La escena, la iluminación, la óptica, el sensor y el procesamiento contribuyen a sus valores. La imagen no contiene automáticamente una medición de la dureza, el material o la tolerancia de una pieza. Esas propiedades pueden requerir información adicional.

El **píxel** es una posición de la imagen digital con uno o varios valores asociados. En gris guarda una intensidad; en RGB, tres componentes de color. En nuestras prácticas se usan imágenes RGB de 8 bits por canal, con valores enteros entre 0 y 255. Un sistema también puede producir imágenes de 16 bits, valores decimales o canales que no representan color visible.

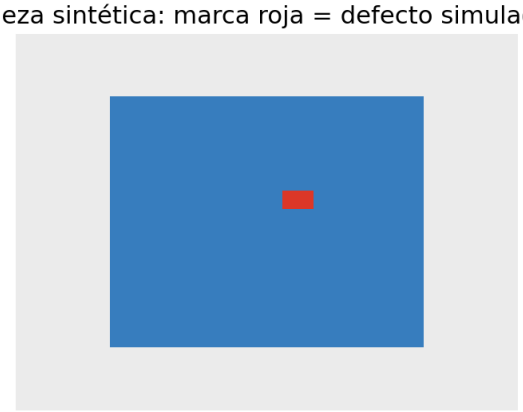


Figura 2.1: Pieza sintética del tema 1. La marca roja es un defecto simulado por construcción, no un diagnóstico obtenido de una fotografía.

2.1.1. De la pieza al archivo: una cadena de transformaciones

Una cámara registra luz que ha interactuado con la escena. La iluminación determina qué luz llega a la pieza; la superficie la refleja de manera diferente según material, textura y orientación.

La óptica forma una imagen sobre el sensor. Durante un tiempo de exposición, el sensor convierte la luz recibida en señales que el sistema transforma en números. El procesamiento puede modificar color, contraste, ruido o nitidez antes de guardar el archivo.

Por eso una zona clara puede corresponder a un material claro, a un reflejo, a una superficie orientada hacia la luz o a una modificación del procesado. De igual manera, una zona oscura no demuestra un poro. Para interpretar una marca necesitamos relacionarla con la tarea y con las condiciones de adquisición.

Ejemplo: Un reflejo que parece un defecto

Queremos localizar arañazos en una pieza metálica. Al mover una lámpara aparece una franja brillante que el sistema podría confundir con una marca. Si el patrón cambia al variar la iluminación y no al cambiar la pieza, hay evidencia de que la captura influye en la observación. No basta con etiquetar todas las franjas como defectos: conviene diseñar y documentar la iluminación antes de entrenar.

2.1.2. Muestreo y cuantificación: dos discretizaciones diferentes

El **muestreo espacial** elige posiciones en una rejilla. La **cuantificación** representa cada medida mediante un conjunto finito de niveles. El primer proceso limita dónde tenemos datos; el segundo, qué diferencias de valor podemos representar. Aumentar la profundidad de bits no añade posiciones a la rejilla. Aumentar el número de píxeles tampoco garantiza que la óptica resuelva detalles más pequeños.

Con b bits por canal hay 2^b códigos posibles. Con 8 bits son 256, numerados del 0 al 255; con 16 bits son 65 536. El máximo es $2^b - 1$ porque el cero también ocupa un código. Que un archivo pueda guardar muchos niveles no significa que el sensor distinga todos esos niveles con precisión útil.

Modelo intuitivo de un píxel

Podemos pensar el valor de un píxel como una respuesta a la luz recibida en una pequeña región y durante un intervalo temporal. No es necesariamente la medida de un punto físico infinitamente pequeño. La óptica, el movimiento y la respuesta del sensor mezclan información antes de que aparezca la matriz. El modelo es orientativo: el procesamiento de una cámara de color puede añadir otras mezclas y transformaciones.

2.1.3. Qué conocemos de la imagen sintética del curso

La imagen se construye con un fondo uniforme, un rectángulo azul y una marca roja. Sabemos dónde está cada región porque la hemos definido por código. No intervienen lente, exposición ni sensor. Esto permite aislar efectos y comprobar cálculos, pero hace la escena mucho más sencilla que una fotografía de una pieza real.

La palabra «defecto» se usa en la práctica para una marca simulada. Que un píxel sea rojo no es una definición general de defecto: una pieza roja correcta podría tener el mismo color. La etiqueta depende de una especificación de producto y de evidencia apropiada.

2.2. Dimensiones, coordenadas y memoria

En NumPy, `imagen[fila, columna, canal]` selecciona un valor. La forma `(240,320,3)` indica alto, ancho y canales. La primera fila está arriba y la primera columna a la izquierda. Pillow

recibe el tamaño de redimensionado como `(ancho,alto)`: conviene comprobar la convención en cada biblioteca.

Un intervalo como `60:80` incluye 60 y excluye 80. Contiene 20 índices. Así, `imagen[80:140, 140:220]` produce un recorte de 60 filas y 80 columnas. Si la imagen es RGB, conserva sus tres canales.

Ejemplo: Tamaño de almacenamiento sin compresión

Una imagen RGB de 240×320 con un byte por canal requiere $240 \cdot 320 \cdot 3 = 230\,400$ bytes para sus valores. Un `float32` usa cuatro bytes por valor y requeriría 921 600 bytes. No son los tamaños exactos de un archivo PNG o JPEG: estos incluyen codificación, compresión y metadatos.

2.2.1. Coordenadas: distinguir posición y tamaño

Para una imagen de alto H y ancho W, los índices de fila van de 0 a $H-1$ y los de columna de 0 a $W-1$. En las coordenadas gráficas habituales, x recorre las columnas y y las filas. NumPy escribe primero fila y después columna: $I[y, x]$. En la visualización usual de imágenes, y aumenta hacia abajo, a diferencia de algunos gráficos cartesianos.

Un recorte $I[y_0 : y_1, x_0 : x_1]$ tiene alto $y_1 - y_0$ y ancho $x_1 - x_0$, suponiendo índices dentro del rango y paso 1. Se excluye el extremo final. Especificar estos intervalos de forma explícita evita discrepancias de un píxel al comparar anotaciones.

Ejemplo: Calcular la región de la marca

En el notebook la marca ocupa filas `100:112` y columnas `170:190`. Su alto es 12 píxeles; su ancho, 20; y su área, $12 \cdot 20 = 240$ píxeles cuadrados. La posición $(105, 180)$ pertenece a ella. La posición $(112, 180)$ queda fuera porque la fila 112 está excluida.
El recorte de filas `80:140` y columnas `140:220` contiene la marca completa. Dentro de ese recorte, la marca empieza en fila $100 - 80 = 20$ y columna $170 - 140 = 30$. Las coordenadas cambian de referencia; la marca no se desplaza en la escena original.

2.2.2. Una imagen, un lote y su coste de memoria

El tamaño de los datos sin compresión se calcula como

$$B = H\,W\,C\,q,$$

donde C es el número de canales y q los bytes por valor. Para un lote de N imágenes, se multiplica por N. Es una estimación de los valores almacenados: el programa puede necesitar además copias, resultados intermedios y memoria de sus modelos.

Por ejemplo, cien imágenes del notebook necesitan 23 040 000 bytes en `uint8`, aproximadamente 23,04 MB decimales o 21,97 MiB. En `float32` necesitan cuatro veces más. Un MB equivale a 10^6 bytes y un MiB a 2^{20} bytes. Conviene indicar la unidad al comparar memoria o almacenamiento.

2.2.3. Archivo y matriz no son lo mismo

Un archivo codifica datos y puede incluir metadatos. Al abrirlo obtenemos una matriz según el modo y la biblioteca utilizados. Una imagen puede llegar en gris, RGB, RGBA con transparencia o con una paleta de colores. La extensión no basta para deducir la forma final: hay que inspeccionar modo, dimensiones y tipo.

PNG admite guardar imágenes habituales sin pérdida de sus valores decodificados. JPEG suele utilizar compresión con pérdida y puede introducir alteraciones alrededor de bordes o texturas. La calidad visual puede ser suficiente para mostrar una pieza y resultar insuficiente para estudiar un pequeño defecto. Convertir después un JPEG a PNG conserva la imagen ya decodificada; no reconstruye lo que se descartó al comprimir.

La orientación indicada por metadatos también puede afectar a cómo se muestra una fotografía. En un sistema de inspección, las coordenadas deben referirse a la misma orientación que la matriz y las anotaciones utilizadas. En la imagen sintética esta dificultad no aparece.

2.2.4. Comprobación en Python: conservar una referencia

Ejecuta las siguientes celdas después de crear `imagen` como en el notebook. Se usa `copy()` para trabajar sobre una copia independiente.

```
print(imagen.shape, imagen.dtype, imagen.nbytes)
marca = imagen[100:112, 170:190].copy()
print(marca.shape)                          # (12, 20, 3)
print(marca.shape[0] * marca.shape[1])    # 240
copia = imagen.copy()
copia[100:112, 170:190] = [0, 0, 0]
print(imagen[105, 180].tolist())          # [220, 55, 40]
```

Una selección básica por intervalos de NumPy suele ser una vista: puede compartir datos con el original. Si queremos modificar un recorte sin afectar a la referencia, creamos una copia explícita. No es un concepto de visión artificial, pero evita conclusiones equivocadas al comparar experimentos.

2.3. Color, gris y pérdida de información

RGB combina rojo, verde y azul. En el ejemplo, la pieza tiene $(55, 125, 190)$ y la marca $(220, 55, 40)$. En el canal rojo, la marca presenta un valor mayor que la pieza. Mostrar cada canal en gris permite comparar esa contribución, usando el mismo rango de visualización.

Convertir a gris suele combinar canales con diferentes pesos. Una aproximación didáctica habitual es $Y \approx 0{,}299R + 0{,}587G + 0{,}114B$, aunque el significado físico depende del espacio de color y de las transformaciones aplicadas. No debemos interpretarla sin más como luminancia física medida. Pillow realiza su propia conversión de modo; en el notebook la observamos sin usar esa fórmula como ley universal.

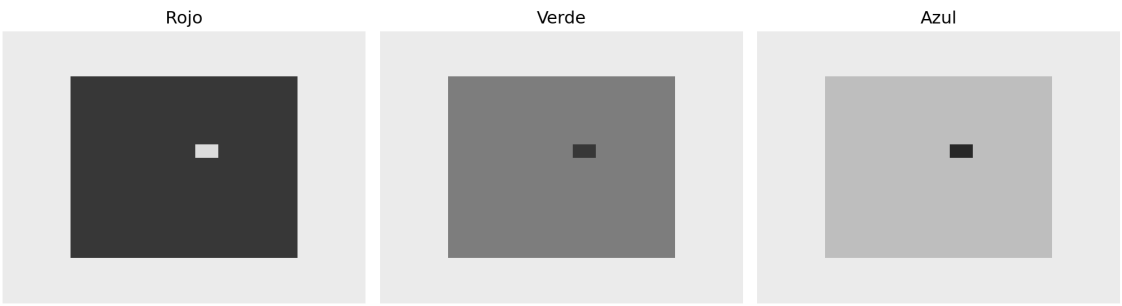


Figura 2.2: Canales de la misma imagen. Todos se visualizan con rango 0–255. Una escala automática diferente en cada panel podría inducir comparaciones engañosas.

Dos colores distintos pueden producir grises parecidos. La transformación reduce tres números a uno y generalmente no permite recuperar el color original. Si distinguir piezas depende del color, esa pérdida puede ser importante.

2.3.1. Un canal mostrado en gris no es una imagen en gris convertida

Mostrar el canal rojo con una paleta de grises representa la magnitud de R mediante brillo. Una zona clara significa «valor rojo alto», no «color blanco en la imagen original». En cambio, convertir RGB a gris combina información de los tres canales. Ambas visualizaciones pueden tener dos ejes, pero contienen cantidades diferentes.

Para el color de pieza $(55, 125, 190)$, la combinación aproximada anterior produce 111,48. Para la marca $(220, 55, 40)$ produce 102,63. En el canal rojo hay una diferencia de 165 niveles, mientras que en esa combinación gris la diferencia es de unos 8,85. La marca puede destacar mucho menos. Pillow puede redondear el resultado a códigos enteros: aquí los cálculos ilustran la transformación y no sustituyen la comprobación de la biblioteca.

Ejemplo: Colores distintos con grises casi iguales

El rojo $(255, 0, 0)$ da aproximadamente 76,245 con los pesos indicados. El verde $(0, 130, 0)$ da 76,31. Son colores distintos que producen casi el mismo gris. Un método que reciba solo esa intensidad no podrá recuperar cuál de los dos colores originó el valor sin información adicional.

2.3.2. RGB, orden de canales y transparencia

Los nombres de los canales deben acompañar a los números. Si una biblioteca entrega BGR y otra espera RGB, interpretar directamente sus tres valores intercambia rojo y azul. La forma sigue siendo $(H, W, 3)$: el número de ejes no detecta ese error. Podemos verificarlo con un parche de color conocido.

Un cuarto canal suele representar **alfa**, relacionado con transparencia, y no un cuarto color. Si se combina una imagen con transparencia sobre un fondo, el color visible depende también de ese fondo. Antes de usarla en un modelo entrenado con RGB hay que definir una conversión coherente. El notebook parte de RGB sin transparencia.

2.3.3. Valores de color y luz física

Los valores de una imagen habitual ya pueden incorporar curvas de codificación y ajustes de cámara. Por ello, duplicar un valor RGB almacenado no equivale en general a duplicar la luz de la escena. Para el curso operamos sobre los códigos de la imagen; una medición fotométrica necesitaría especificar espacio de color, respuesta y calibración.

En una inspección de color también importan la iluminación y los ajustes automáticos de balance de blancos. Dos capturas de la misma pieza pueden presentar valores distintos. Comparar colores exige controlar o modelar esas condiciones, además de elegir una fórmula.

2.3.4. Comprobación en Python: medir la pérdida de contraste

```
pieza = np.array([55., 125., 190.])
marca_rgb = np.array([220., 55., 40.])
pesos = np.array([0.299, 0.587, 0.114])
print(pieza @ pesos, marca_rgb @ pesos)  # 111.48, 102.625
```

```
print(abs(marca_rgb[0] - pieza[0]))      # 165.0
print(abs((marca_rgb - pieza) @ pesos)) # aproximadamente 8.855
```

Compara este resultado con la actividad 3. No concluyas que el canal rojo será siempre la mejor entrada: este resultado depende de los colores que hemos elegido para la simulación.

2.4. Recorte, resolución e interpolación

Recortar selecciona una región. **Reducir resolución** disminuye el número de posiciones que representan la escena. **Interpolar** estima valores en nuevas posiciones a partir de los disponibles. Una ampliación puede hacer una imagen más cómoda de ver, pero no recupera automáticamente mediciones originales que fueron descartadas.

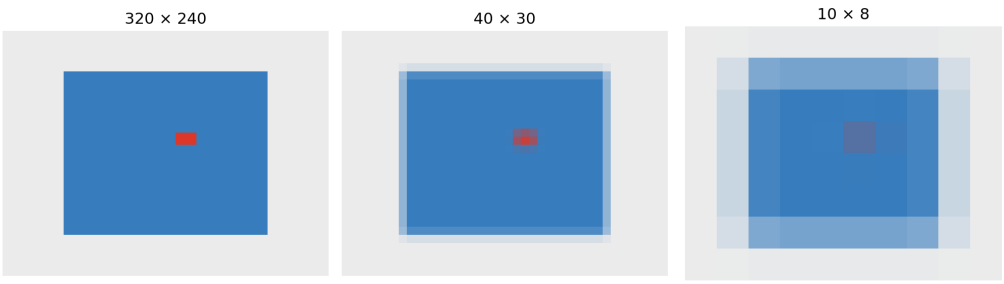


Figura 2.3: El defecto se diluye al reducir la resolución. El comportamiento depende también del filtro utilizado y de su posición respecto a la rejilla.

Con vecino más cercano se elige un valor próximo; con interpolación bilineal se combinan valores de vecinos con pesos. En reducción también importa evitar que patrones finos se conviertan en patrones falsos, fenómeno relacionado con el *aliasing*. No es suficiente observar el número final de píxeles: el procedimiento de remuestreo cambia el resultado.

Ejemplo: De tamaño físico a píxeles

Si una anchura de 100 mm ocupa 1000 píxeles en un plano de escala uniforme, hay 10 píxeles por mm. Una marca de 0,2 mm ocuparía aproximadamente 2 píxeles. Ese cálculo solo convierte escala: enfoque, contraste, ruido y orientación pueden impedir distinguirla. Dos píxeles no constituyen un umbral universal de detección fiable.

2.4.1. Campo de visión y detalle relevante

El **campo de visión** es la región de la escena que aparece en la captura. Con un mismo número de píxeles, abarcar más superficie suele reducir los píxeles disponibles por unidad física en el plano observado. Si un ancho de 160 mm ocupa 320 píxeles, la escala es 2 píxeles por mm y un detalle de 0,5 mm ocupa aproximadamente un píxel. Con el mismo ancho de imagen, una captura de solo 40 mm daría 8 píxeles por mm.

La conversión supone una escala uniforme en ese plano. Con perspectiva, curvatura o distinta profundidad, la escala puede variar dentro de una imagen. Una regla situada fuera del plano de interés no proporciona necesariamente la escala correcta de la pieza.

2.4.2. Interpolar: un cálculo sencillo

En una dimensión, la interpolación lineal entre valores a y b usa

$$v(t) = (1 - t)a + tb, \qquad 0 \le t \le 1.$$

Si $a = 20$, $b = 100$ y $t = 0{,}25$, resulta $v = 40$. El valor se estima combinando los disponibles; no procede de una nueva observación. En dos dimensiones, la interpolación bilineal combina vecinos en ambas direcciones. Las convenciones de coordenadas, los bordes y los filtros aplicados durante una reducción también influyen en el resultado exacto de una biblioteca.

Para profundizar: por qué una reducción puede ser irreversible

Consideremos un resumen que sustituye un bloque de cuatro valores por su media. Los bloques $(0, 0, 100, 100)$ y $(50, 50, 50, 50)$ producen ambos 50. Conocer ese 50 no permite decidir qué bloque había antes. Hay múltiples entradas con la misma salida: la operación no tiene inversa única. Este ejemplo explica pérdida de información por promediado; no afirma que Pillow utilice exactamente ese resumen en todos sus redimensionados.

2.4.3. Aliasing y detalle aparente

Si un patrón varía muy deprisa respecto a la rejilla, distintos patrones pueden producir las mismas muestras o patrones falsos. Este fenómeno se denomina *aliasing*. Una textura fina de capas podría aparecer con bandas más gruesas, desaparecer o cambiar al desplazar la pieza ligeramente. El patrón observado puede depender tanto del muestreo como de la superficie.

Para una señal ideal limitada en frecuencia, el resultado de muestreo de Nyquist–Shannon exige una frecuencia de muestreo superior al doble de su máxima frecuencia para una reconstrucción ideal sin ambigüedad. Aquí basta retener la intuición: hay que muestrear suficientemente rápido las variaciones y filtrar las que no se podrán representar. Una marca aislada y una cámara real no cumplen automáticamente esas condiciones ideales; este resultado no fija un número universal de píxeles por defecto.

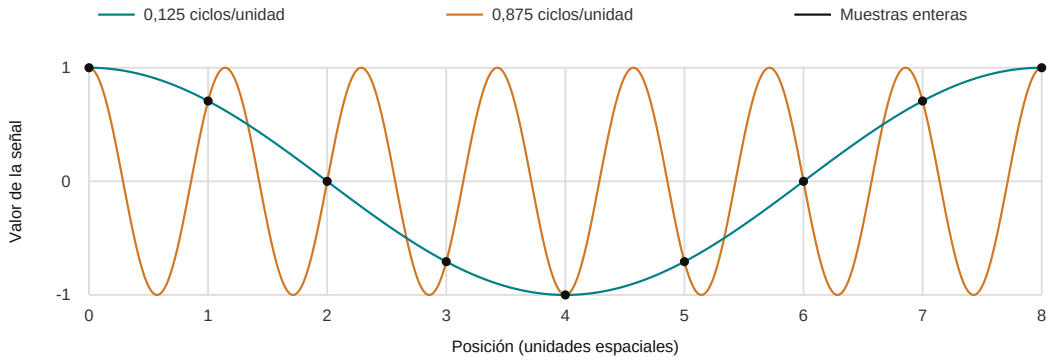


Figura 2.4: Ejemplo unidimensional de aliasing. Dos señales de distinta frecuencia coinciden en todas las posiciones enteras muestreadas. Las curvas continuas son ejemplos matemáticos, no mediciones recuperadas a partir de los puntos.

2.4.4. Elegir un remuestreo según el dato

En una fotografía pueden interesar valores suaves y un filtrado adecuado al reducir. En una máscara de clases, los códigos son categorías: interpolar entre clase 0 y clase 2 podría inventar

clase 1. El vecino más cercano suele preservar los códigos existentes, aunque puede eliminar regiones pequeñas y no garantiza conservar áreas. La transformación debe aplicarse de forma compatible a la imagen y a sus anotaciones.

Cambiar de 320×240 a 40×30 mantiene la relación de aspecto $4 : 3$. Cambiar a 10×8 la modifica ligeramente: pasa a $5 : 4$. En la actividad 4, esa última transformación combina una reducción fuerte con una pequeña deformación. Para aislar solo el efecto de resolución, puede compararse adicionalmente con 12×9, que mantiene $4 : 3$.

2.4.5. Propuesta de experimento: detalle y posición

En una copia, añade una marca de 2×2 píxeles y reduce la imagen. Repite desplazándola un píxel antes de reducir, sin cambiar su color ni tamaño. Mantén el mismo filtro y rango de visualización. Compara si el valor y el contraste de la marca reducida cambian. El experimento permite observar sensibilidad a la rejilla; no proporciona una estimación de la tasa de detección de un modelo que todavía no hemos entrenado.

Anota siempre tamaño de entrada, tamaño de salida, filtro y si se conserva la relación de aspecto. Si luego amplías para mostrar, distingue ese segundo remuestreo del primero. En la práctica se usa `interpolation="nearest"` al mostrar los resultados para ver las celdas de la imagen ya reducida.

2.5. Tipos numéricos y normalización

El tipo `uint8` representa enteros no negativos de 8 bits. Su aritmética puede volver al inicio del rango al desbordarse. En el ejemplo de NumPy, $250 + 20$ almacenado de nuevo en `uint8` produce 14, porque $270 - 256 = 14$. No es un ajuste de brillo correcto.

Para operar de manera controlada, convertimos primero a decimales. Podemos aplicar $x' = \text{clip}(x + 20, 0, 255)$ y convertir después al tipo adecuado. El recorte evita el retorno modular, pero los valores por encima de 255 se saturan y pierden diferencias.

Dividir por 255 tras convertir a `float32` produce intensidades entre 0 y 1. Si se visualizan conforme a ese rango, la apariencia puede ser la misma. El número de píxeles no cambia. Algunos modelos exigen además medias y escalas específicas por canal; se debe respetar el preprocesamiento asociado a sus pesos.

2.5.1. Convertir el tipo no es cambiar la escala

`astype(np.float32)` transforma la representación numérica: un código 125 pasa a ser 125,0. Solo después de dividir por 255 pasa a 0,490 . . .. Si una función espera RGB decimal entre 0 y 1 y recibe valores entre 0 y 255, la visualización puede saturarse. El tipo por sí solo no documenta el rango.

Para volver de la escala $[0, 1]$ a enteros de 8 bits se puede multiplicar por 255, redondear, limitar al intervalo válido y convertir. Convertir directamente a entero descarta la parte decimal; el orden de estas operaciones importa. Tras muchas transformaciones, redondear en cada paso puede acumular diferencias: suele ser preferible calcular con decimales y cuantificar al final cuando proceda.

2.5.2. Saturación, desbordamiento y pérdida de orden

Son fenómenos diferentes. El desbordamiento modular del ejemplo convierte 270 en 14; un valor grande puede terminar pareciendo oscuro. La saturación lo limita a 255. En este segundo caso, entradas distintas como 250 y 255, tras sumar 20 y recortar, terminan ambas en 255. Se evita el retorno modular, pero se pierden diferencias entre zonas brillantes.

Ejemplo: Una resta que necesita signo

La diferencia real entre intensidades 20 y 250 es -230. Si la resta se efectúa en `uint8`, puede producir 26 por aritmética modular. Para estudiar diferencias entre imágenes, convierte ambos operandos a un tipo apropiado antes de restar. Mostrar el valor absoluto después no corrige una resta que ya se calculó con el tipo equivocado.

2.5.3. Comprobación en Python: transformar y recuperar

```
f = imagen.astype(np.float32) / 255.0
recuperada = np.rint(np.clip(f, 0, 1) * 255).astype(np.uint8)
print(np.array_equal(imagen, recuperada))  # True en este caso
segura = np.clip(imagen.astype(np.float32) + 20, 0, 255)
print(segura.min(), segura.max())
diferencia = segura - imagen.astype(np.float32)
print(diferencia.min(), diferencia.max()) # 20.0, 20.0
```

En esta imagen, el máximo inicial es 235 y sumar 20 no supera 255. Si modificas un píxel para que valga 250, ese píxel aumentará solo cinco unidades tras recortar. El mismo procedimiento puede ser reversible en un caso y perder información en otro.

Una preparación coherente entre entrenamiento y uso

Si un modelo aprende con imágenes divididas por 255, normalmente debemos aplicar la misma preparación al usarlo. Si sus pesos exigen otra transformación, se sigue esa especificación. Reescalar cada imagen por su propio máximo también cambia las relaciones entre capturas y puede eliminar información de brillo absoluto. No equivale a dividir siempre por una constante conocida.

2.6. Ruido, iluminación y visualización

En una simulación aditiva podemos escribir $I' = I + \epsilon$, donde ϵ varía por píxel. El notebook utiliza ruido gaussiano de media cero y una desviación elegida. Esto permite estudiar variación aleatoria; no reproduce todos los mecanismos físicos de una cámara.

La iluminación se ilustra multiplicando valores por un factor. En una captura real puede cambiar también el contraste, las sombras, los reflejos y la relación entre señal y ruido. Una simulación sencilla no autoriza a concluir que hemos probado todos esos efectos.

Las imágenes se deben comparar con escalas coherentes. En una imagen RGB decimal, muchas funciones esperan valores entre 0 y 1. En un mapa de una sola variable, la visualización puede reescalar automáticamente mínimos y máximos. Por eso una imagen oscura podría parecer clara si no se fijan los límites.

2.6.1. Separar ruido, desenfoque y variación de luz

El **ruido** introduce variación en los valores; el **desenfoque** mezcla contribuciones de posiciones vecinas; una variación de **iluminación** puede alterar brillo, sombras y reflejos. Una imagen borrosa puede tener poco ruido y aun así haber perdido el borde de un defecto. Una imagen nítida puede tener ruido que dificulte distinguir una marca débil. La solución no tiene por qué ser el mismo filtro.

En el notebook el ruido se genera por componente RGB, con desviación σ expresada en niveles de 0 a 255. Con $\sigma = 25$, la escala equivalente antes de recortar, al dividir por 255, sería $25/255 \approx 0{,}098$. Si trabajamos ya en $[0, 1]$, añadir ruido con desviación 25 sería una perturbación completamente diferente.

La marca y la pieza se diferencian mucho en rojo, pero poco en la combinación gris indicada. Un mismo nivel de ruido puede dificultar una representación más que otra. Comparar únicamente el aspecto general de la imagen puede ocultar lo que ocurre en la región decisiva.

2.6.2. Qué cambia al recortar el ruido

Antes del recorte, el ruido simulado tiene media cero por construcción de su distribución. Una realización finita no tiene por qué presentar media exactamente cero. Después de limitar al rango $[0, 255]$, los valores que superarían un borde quedan acumulados allí; cerca de los extremos puede cambiar la media y la distribución de la perturbación final. La conversión posterior a entero añade cuantificación.

Una semilla permite repetir el experimento si reiniciamos el generador y mantenemos la misma secuencia de operaciones y entorno compatible. Si ejecutamos solo la llamada que extrae ruido, sin reiniciar el generador, consumimos nuevos números. Para comparar filtros con exactamente el mismo ruido, es mejor generar una matriz una vez y reutilizarla.

2.6.3. Visualizar es una transformación más

En un mapa de grises, `vmin` y `vmax` fijan qué valores se asignan a los extremos de la paleta. Si cada imagen usa sus propios mínimo y máximo, dos imágenes con rangos muy distintos pueden parecer igualmente contrastadas. Para comparar intensidades, se usa una escala común y se informa de ella.

Un histograma cuenta cuántos valores caen en cada intervalo. Ayuda a localizar acumulaciones en 0 o 255 y a comparar distribuciones, pero elimina la posición espacial. Dos imágenes con los mismos píxeles reorganizados tienen el mismo histograma y pueden representar formas diferentes.

Ejemplo: Por qué el histograma no basta para localizar

En una matriz 2×2 podemos poner dos valores negros y dos blancos por filas o en diagonal. Ambas imágenes tienen dos ceros y dos valores 255. Sus histogramas coinciden, mientras que su organización espacial es distinta. Una marca pequeña también puede afectar poco al histograma global aunque sea decisiva para aceptar una pieza.

2.7. Guía de trabajo con el notebook

No es necesario añadir todas las ampliaciones a la sesión presencial. Estas preguntas permiten profundizar al preparar la clase o al estudiar después.

Actividad 1: describir antes de interpretar.
Anota forma, tipo, rango observado y procedencia sintética. Distingue rango permitido por el tipo del rango realmente presente: `uint8` admite 0–255, aunque esta imagen no utilice todos los niveles.

Actividad 2: predecir coordenadas.
Calcula alto y ancho del cuadrado amarillo antes de moverlo. Dentro de la pieza, una posición posible es `180:200,240:260`. Comprueba que el extremo excluido mantiene el cuadrado dentro del rectángulo.

Actividad 3: comparar representaciones.
Explica qué significa una zona clara en cada canal. Después prueba los parches rojo y verde del ejemplo de grises casi iguales. No confundas seleccionar un canal con convertir a gris.

Actividad 4: controlar el experimento.
Registra filtro, tamaño y relación de aspecto. Compara primero reducciones que conserven esa relación. Usa una escala de visualización coherente y distingue lo que se ve en pantalla de los valores almacenados.

Actividad 5: comprobar el rango.
Calcula 125/255 y verifica que cambiar a decimales sin dividir conserva 125. Compara desbordamiento y saturación con un píxel de valor 250.

Actividad 6: cambiar un factor cada vez.
Mantén la referencia y la matriz de ruido cuando proceda. Compara varias desviaciones con la misma luz y varios factores de luz sin ruido añadido. Solo después combina perturbaciones.

Para cada experimento basta una tabla con cuatro campos: predicción, cambio aplicado, observación y explicación. Por ejemplo: «Al reducir de 320×240 a 40×30, la marca ocupa menos celdas y se mezcla con el entorno. Se observa menor contraste. Esto sugiere pérdida de información útil en esta simulación; no permite afirmar una tasa de fallos para un detector real».

2.8. Aplicación a producto y fabricación aditiva

Antes de elegir un modelo, formula qué propiedad visual necesita conservar la imagen. Para distinguir familias de piezas puede ser decisiva la silueta. Para estudiar textura superficial pueden importar escala, enfoque e iluminación. Para medir una anchura se necesitan además geometría de captura, calibración y un análisis de incertidumbre adecuado.

Ejemplo: Preparar una inspección de una marca superficial

Supongamos una marca de interés de 0,3 mm sobre una cara plana de 60 mm de ancho. Con 1200 píxeles sobre ese ancho, la escala nominal es de 20 píxeles por mm y la marca ocuparía unos seis píxeles en la dirección considerada. Es un cálculo de diseño inicial. Hay que comprobar capturas reales de marcas y piezas correctas, distintas posiciones, reflejos, enfoque y variación entre piezas antes de concluir si la información resulta suficiente.

Documenta también qué no es observable. Una fotografía superficial no revela por sí sola todos los defectos internos, la resistencia mecánica o la conformidad dimensional completa. El criterio de aceptación debe corresponder a la propiedad evaluada y al medio de medida disponible.

2.9. Errores frecuentes y cómo reconocerlos

Confundir ancho y alto.
El recorte aparece alargado en otra dirección o el tamaño no coincide. Escribe qué representa cada eje y recuerda el orden de Pillow al redimensionar.

Interpretar brillo como defecto.
Una marca aparece solo al cambiar iluminación o ajustes. Revisa la adquisición y la definición de defecto antes de etiquetar.

Creer que ampliar recupera detalle.
La imagen ocupa más pantalla, pero los valores proceden de interpolación. Compara la información disponible antes y después de la reducción.

Operar con un tipo inadecuado.
Aparecen valores pequeños al sumar o positivos al restar. Convierte antes de operar y comprueba los extremos numéricos.

Comparar escalas automáticas distintas.
Una imagen de poca amplitud parece tan contrastada como otra. Fija límites comunes al comparar mapas de intensidad.

Confundir repetibilidad con realismo.
Una semilla repite una simulación; no acredita que la simulación reproduzca una cámara o una población de piezas.

2.10. Ejercicios de repaso con soluciones razonadas

1. Una región y su memoria

Una imagen RGB `uint8` tiene forma $(300, 400, 3)$. Recortamos `[50:110,120:200]`. ¿Qué forma y cuántos bytes de valores tendría una copia independiente del recorte?

Solución. El alto es 60 y el ancho 80; la forma es $(60, 80, 3)$. Con un byte por valor, son $60 \cdot 80 \cdot 3 = 14\,400$ bytes. Una vista puede compartir la memoria de la imagen completa; la pregunta especifica una copia de los valores del recorte.

2. Trasladar coordenadas al recorte

Un punto de la imagen original está en fila 75 y columna 160. ¿En qué posición aparece dentro del recorte del ejercicio anterior? ¿Está incluido el punto de fila 110 y columna 160?

Solución. El primer punto aparece en $(75 - 50, 160 - 120) = (25, 40)$. El segundo queda fuera: la fila final 110 está excluida. Las coordenadas locales describen el mismo punto con un origen diferente.

3. Interpolación y observación

Entre dos muestras de valores 40 y 120, calcula el valor interpolado con $t = 0{,}75$. ¿Es una nueva medida de la escena?

Solución. El valor es $0{,}25 \cdot 40 + 0{,}75 \cdot 120 = 100$. Es una estimación definida por una regla de interpolación, no una medida nueva. Una estructura estrecha entre las muestras podría no estar representada en ellas.

4. Resolución física nominal

Una cara plana de 80 mm ocupa 1600 píxeles de ancho. Una marca de 0,2 mm se observa en ese plano. ¿Cuántos píxeles ocuparía aproximadamente? ¿Y tras reducir a 400 píxeles de ancho conservando la relación de aspecto?

Solución. La escala inicial es $1600/80 = 20$ píxeles por mm y la marca ocupa aproximadamente cuatro píxeles. Tras reducir por un factor cuatro, ocuparía aproximadamente uno. Son tamaños nominales: el valor final también depende de filtro, posición, contraste y óptica. No garantizan detección.

5. Saturación frente a retorno modular

Sumamos 20 a valores `uint8` 240 y 250. Compara el resultado modular con convertir primero a decimal y limitar después a 255.

Solución. Los valores matemáticos son 260 y 270. Con retorno módulo 256, resultan 4 y 14. Con saturación, ambos dan 255. La saturación conserva mejor la idea de «muy brillante», pero elimina la diferencia original de diez unidades.

6. Histograma y posición

¿Tienen el mismo histograma las matrices $\left(\begin{smallmatrix} 0 & 0 \\ 255 & 255 \end{smallmatrix}\right)$ y $\left(\begin{smallmatrix} 0 & 255 \\ 255 & 0 \end{smallmatrix}\right)$? ¿Permite eso afirmar que servirán igual para identificar una forma?

Solución. Sí: ambas contienen dos ceros y dos valores 255. No permite afirmar que sirvan igual para identificar una forma, porque el histograma descarta las posiciones y las dos distribuciones espaciales son diferentes.

Conexión con las prácticas

Las actividades 1–3 comprueban forma, coordenadas y color; la 4 estudia recorte y resolución; la 5, escala y tipo numérico; la 6, ruido e iluminación. Antes de ejecutar, predice qué valores o propiedades deberían cambiar. Después distingue el resultado observado de una conclusión sobre piezas reales. Material de la sesión en el repositorio.

Capítulo 3

Tema 2. Aprender a clasificar imágenes

Objetivos y recorrido

El objetivo es comprender cómo se aprende una regla de clasificación y qué evidencia permite juzgarla. Al terminar deberías poder distinguir parámetros de hiperparámetros, explicar una predicción mediante puntuaciones, leer una matriz de confusión y detectar una comparación experimental engañosa.
La práctica ejecuta una referencia de mayoría, una regresión logística y una MLP pequeña. Las CNN y el aprendizaje por transferencia amplían el mapa conceptual, pero no se ejecutan en este notebook. Las comprobaciones adicionales de este capítulo utilizan NumPy y no necesitan GPU ni descargar modelos.

3.1. Entrada, etiqueta y modelo

Clasificar significa asignar una clase a una entrada según una tarea definida. En el notebook, una imagen recibe arandela, placa o escuadra. Las etiquetas se conocen porque las formas se generaron por código. El modelo recibe números, no el significado físico de esos nombres.

Escribimos el conjunto de entrenamiento como $\{(x_i, y_i)\}_{i=1}^{n}$. El vector x_i contiene las entradas y y_i la clase. El modelo f_θ depende de parámetros θ. Entrenar consiste en elegir esos parámetros a partir de ejemplos; inferir consiste en aplicar el modelo fijado a una entrada.

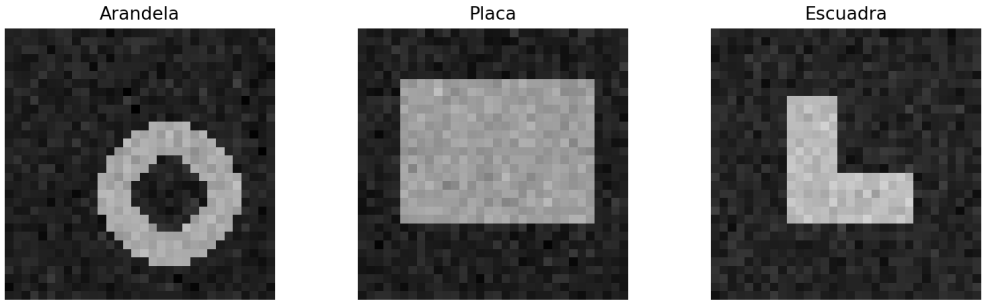


Figura 3.1: Tres familias sintéticas utilizadas para estudiar clasificación. Las variaciones de posición, tamaño y ruido no cubren toda la variación de fotografías reales.

3.1.1. Definir la pregunta antes de reunir imágenes

Una clasificación asigna una etiqueta de un conjunto definido. En nuestra tarea, cada imagen contiene una pieza y recibe una de tres familias. No estamos localizando cada objeto de una escena ni midiendo su tolerancia. Si la pregunta fuera «¿dónde están las piezas?», necesitaríamos otra definición de salida, que se estudia en el tema siguiente.

En una clasificación de una sola etiqueta, las clases deben tener una interpretación coherente. Si una imagen puede contener varios tipos a la vez, quizá necesitemos múltiples etiquetas o separar objetos. Si contiene una pieza de una familia desconocida, un clasificador que solo dispone de

tres salidas normalmente seguirá eligiendo una de ellas. La opción «desconocida» o la abstención requieren un diseño y una evaluación específicos.

Ejemplo: Familia de pieza y conformidad son tareas distintas

Una arandela con una marca superficial sigue siendo una arandela como familia geométrica, aunque pudiera incumplir un criterio de acabado. Si usamos etiquetas de familia, el modelo no aprende automáticamente conformidad. Para estudiar defectos habría que definir esa segunda tarea, su criterio de referencia y sus datos.

3.1.2. Qué significa supervisado

En aprendizaje **supervisado**, los ejemplos de entrenamiento incluyen la salida deseada. El programa busca parámetros que relacionen entradas y etiquetas. Las etiquetas no se entregan al modelo durante la inferencia; se conservan aparte cuando queremos evaluar sus respuestas.

El nombre «Arandela» es una interpretación humana del código 0. Los códigos 0, 1 y 2 identifican categorías: no dicen que una escuadra sea el doble de una placa ni que las clases tengan un orden físico. La pérdida y la salida deben tratar esos códigos como clases, no como una magnitud continua que debamos aproximar por distancia numérica.

3.1.3. Del lote de imágenes a una tabla de características

El notebook crea un arreglo de forma $(540, 32, 32)$ y lo reorganiza como $X \in \mathbb{R}^{540\times1024}$. Cada fila corresponde a una captura y cada columna a una posición de píxel. El vector y contiene 540 etiquetas. El vector de grupos identifica la pieza que originó cada captura.

Una **característica** es un valor que se utiliza como entrada. Aquí son píxeles. En otro método podrían ser área, circularidad o vectores aprendidos. Aplanar no calcula características nuevas: cambia la organización de los mismos valores y conserva el orden acordado.

La geometría de cada pieza sintética se mantiene entre sus tres capturas, mientras cambian posición, luz y ruido. Los identificadores de grupo sirven para separar piezas; no se añaden como una entrada del clasificador. Darlos al modelo podría permitir asociaciones que no existirían para una pieza nueva.

3.2. Por qué necesitamos tres conjuntos

Entrenamiento sirve para aprender parámetros y ajustes como el escalado. Validación permite elegir opciones: regularización, número de neuronas o transformaciones. Prueba se utiliza después de cerrar esas decisiones, para observar el resultado en datos que no participaron en esa elección.

No basta con asignar nombres diferentes a tres listas. Si fotografías casi idénticas de una misma pieza aparecen en entrenamiento y prueba, el modelo puede beneficiarse de esa dependencia. La unidad de separación debe corresponder a lo que se espera que sea nuevo al usar el sistema: pieza, lote o sesión de captura, según el objetivo.

Ejemplo: 540 imágenes no son 540 piezas

El tema 2 contiene 180 piezas sintéticas y tres capturas por pieza. Se reservan 108 piezas para entrenamiento, 36 para validación y 36 para prueba: 324, 108 y 108 imágenes. Tres capturas de una pieza van siempre al mismo conjunto. El reparto por clase se comprueba; la separación por grupos no garantiza por sí misma idénticas proporciones de clases.

Ajustar medias y desviaciones usando prueba también introduce información. Un *pipeline* ayuda a aprender el preprocesamiento solo en entrenamiento y aplicarlo después de la misma manera. Tampoco evita todas las fugas: si elegimos la preparación tras mirar prueba, ya estamos usando esa información.

3.2.1. Qué aprende cada fase

Fase	Decisiones o ajustes permitidos	Pregunta
Entrenamiento	Pesos, sesgos y referencias del preprocesado	¿Cómo ajustar la regla?
Validación	Modelo, regularización, arquitectura o epoch	¿Qué opción elegimos?
Prueba	Evaluación del procedimiento ya fijado	¿Cómo funciona fuera de la selección?

La validación influye indirectamente en el resultado porque decide qué modelo conservamos. «No usar validación para actualizar pesos» no significa que sus datos no intervengan en ninguna decisión. Precisamente por esa intervención reservamos prueba.

3.2.2. La unidad de separación depende del uso esperado

Si queremos funcionar con piezas nuevas, todas las capturas de una pieza deben quedar juntas. Si queremos funcionar además en otra máquina o en otro día de captura, separar por pieza quizá no baste: las condiciones compartidas pueden facilitar el resultado. No existe una partición universalmente correcta para cualquier objetivo.

La práctica usa dos separaciones por grupos: primero reserva el 40 % de las piezas y después divide esa reserva por la mitad. Las proporciones finales son 60/20/20. Es una separación aleatoria reproducible de grupos, no una garantía de idéntico número de piezas de cada clase en todos los conjuntos. Hay que revisar los recuentos y la presencia de las tres clases.

Fuga de información: un ejemplo sencillo

Si calculamos la media de cada píxel utilizando las 540 imágenes antes de separar, las imágenes de prueba influyen en la representación. El procedimiento ya ha observado información del conjunto reservado, aunque no haya utilizado sus etiquetas. La preparación que necesita ajustarse se aprende dentro de entrenamiento; después se aplica sin reajustar a validación y prueba.

3.2.3. Comprobación en Python: qué estamos separando

Esta celda se ejecuta después de la actividad 2 del notebook y no consulta predicciones ni resultados de prueba.

```
for nombre, ids in [("train", train), ("val", val), ("test", test)]:
    print(nombre, X[ids].shape,
          len(np.unique(grupos[ids])),
          np.bincount(y[ids], minlength=3))
assert not set(grupos[train]).intersection(grupos[val])
assert not set(grupos[train]).intersection(grupos[test])
assert not set(grupos[val]).intersection(grupos[test])
```

Estos controles detectan grupos compartidos y permiten examinar tamaños. No detectan todas las dependencias: dos identificadores diferentes podrían corresponder accidentalmente a la misma pieza física en un conjunto real.

3.3. Una referencia y un clasificador lineal

La referencia de mayoría predice siempre la clase más frecuente en entrenamiento. Puede funcionar aparentemente bien si una clase domina. Sirve para preguntar si la propuesta aporta algo respecto a una regla muy sencilla.

El clasificador lineal asigna a cada clase k una puntuación

$$s_k = w_k \cdot x + b_k. \tag{3.1}$$

Los pesos w_k determinan la contribución de cada entrada y b_k desplaza la puntuación. La diferencia entre dos puntuaciones define una frontera lineal en el espacio de entradas. Aunque no podamos visualizar un espacio de 1024 dimensiones, la operación sigue siendo sumar contribuciones ponderadas.

Ejemplo: Dos entradas y una puntuación

Con $x = (0{,}2, 0{,}8)$, $w = (2, -1)$ y $b = 0{,}5$, la puntuación es $2 \cdot 0{,}2 - 0{,}8 + 0{,}5 = 0{,}1$. No es una probabilidad: podría ser negativa o mayor que uno si cambian los valores.

3.3.1. Por qué una referencia sencilla puede revelar problemas

Supongamos 90 imágenes de placas y diez de arandelas. Predecir siempre «placa» logra 90 % de acierto y no reconoce ninguna arandela. Una cifra global elevada puede resultar inútil para una clase importante. La referencia permite exigir una mejora concreta y contextualizar el resultado, no solo celebrar un porcentaje.

La referencia del notebook aprende qué clase es mayoritaria en entrenamiento y la predice para todas las entradas. Su acierto en validación depende de la proporción de esa misma clase allí. Aunque el conjunto completo tenga las clases equilibradas, cada partición puede presentar pequeñas diferencias.

3.3.2. Frontera lineal no significa imagen sencilla

Para dos clases, elegir la primera cuando $s_1 > s_2$ equivale a comprobar

$$(w_1 - w_2) \cdot x + (b_1 - b_2) > 0.$$

La frontera de empate es una recta en dos características, un plano en tres o un hiperplano en más dimensiones. «Lineal» describe esa relación en el espacio de entradas elegido, no la apariencia visual de una pieza. Cambiar la representación puede hacer una tarea más o menos separable para el mismo tipo de clasificador.

Ejemplo: Una frontera que podemos leer

Con características x_1, x_2 y puntuaciones $s_1 = 2x_1 - x_2 + 0{,}5$ y $s_2 = -x_1 + 2x_2$, la primera clase gana cuando $3x_1 - 3x_2 + 0{,}5 > 0$. La frontera es $x_2 = x_1 + 1/6$. Para $(0{,}2, 0{,}8)$ gana la segunda; para $(0{,}8, 0{,}2)$ gana la primera. En el notebook hay tres clases y 1024 entradas, pero cada comparación utiliza la misma idea de sumar contribuciones.

3.3.3. Escalar cambia la geometría del ajuste

El escalador estima una media y una escala por columna de X. Cada columna corresponde aquí al mismo píxel en distintas imágenes, no a todos los píxeles de una imagen concreta. Después calcula $(x_j - \mu_j)/\sigma_j$ con esas referencias. Los valores transformados pueden ser negativos y no tienen por qué quedar entre 0 y 1.

El escalado influye en el ajuste y en el significado de la penalización sobre los pesos. No vuelve iguales las distribuciones de todas las clases ni alinea las piezas. Si aplicamos una preparación distinta al predecir, cambiamos la representación que el modelo espera.

3.4. Sigmoide, softmax y probabilidades del modelo

Para dos clases, la sigmoide $\sigma(s) = 1/(1 + e^{-s})$ transforma una puntuación real en un valor entre 0 y 1. En varias clases, softmax produce

$$p_k = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}}. \tag{3.2}$$

Todos los términos son positivos y suman uno porque el denominador es la suma de los numeradores. Eso permite utilizarlos como distribución predicha sobre las clases definidas. No garantiza que esa distribución esté calibrada respecto a las frecuencias reales de acierto.

Ejemplo: Tres puntuaciones

Si $s = (2, 1, 0)$, las exponenciales son aproximadamente $(7{,}389, 2{,}718, 1)$. Al dividir por su suma obtenemos $(0{,}665, 0{,}245, 0{,}090)$. La clase elegida es la primera. El peso 0,665 depende de las otras opciones y de cómo se obtuvieron esas puntuaciones.

Resultado: sumar la misma constante no cambia softmax

Si sustituimos s_k por $s_k + c$, cada numerador y el denominador se multiplican por e^c, que se cancela. Por eso se suele restar la puntuación máxima antes de exponenciar: mantiene el resultado matemático y evita números excesivamente grandes. En cambio, multiplicar las puntuaciones por una constante positiva modifica los pesos relativos, aunque conserva su orden.

3.4.1. Elegir una clase: máximo y empates

La decisión habitual de una sola etiqueta elige $\widehat{y} = \operatorname{argmax}_k p_k$: el índice del valor mayor. El máximo es el valor; el argumento del máximo es la clase que lo alcanza. Softmax conserva el orden de las puntuaciones porque la exponencial es creciente y el denominador es común. Por tanto, elegir el máximo de s_k o de p_k produce la misma clase, salvo detalles de empates y precisión numérica.

Para dos puntuaciones, la primera probabilidad se puede escribir como

$$\frac{e^{s_1}}{e^{s_1} + e^{s_2}} = \frac{1}{1 + e^{-(s_1 - s_2)}}.$$

La sigmoide depende de la diferencia de puntuaciones. Una diferencia cero da 0,5; diferencias grandes positivas favorecen la primera clase. Un umbral distinto de 0,5 puede cambiar la decisión binaria, pero debe elegirse según la tarea y con datos apropiados.

3.4.2. Probabilidad del modelo y calibración

Un valor predicho de 0,9 expresa la distribución que produce el modelo sobre sus clases. Decimos que está bien calibrado en un contexto si, entre muchos casos comparables con valores cercanos a 0,9, la fracción correspondiente de etiquetas resulta aproximadamente del 90 %. La calibración se evalúa con datos; no la garantiza la fórmula softmax.

Un sistema puede acertar la clase más probable y estar demasiado seguro. También puede generar un máximo alto para una imagen ajena a sus tres familias. Añadir un umbral de abstención sin comprobarlo con ejemplos relevantes no garantiza detectar clases desconocidas.

3.4.3. Comprobación en Python: estabilidad y confianza

La siguiente celda es independiente del ajuste del notebook.

```
import numpy as np

def softmax_estable(s):
    e = np.exp(s - np.max(s))
    return e / e.sum()

s = np.array([2., 1., 0.])
p = softmax_estable(s)
print(np.round(p, 3))                  # [0.665 0.245 0.090]
print(np.allclose(p, softmax_estable(s + 1000)))  # True
print(np.argmax(p), np.argmax(softmax_estable(3*s)))
print(np.round(softmax_estable(3*s), 3))
```

Multiplicar las puntuaciones por 3 conserva la clase elegida pero concentra más la distribución en el máximo. La magnitud de la probabilidad puede cambiar sin que cambie una sola clasificación. Restar el máximo antes de exponenciar evita desbordamientos en este cálculo; no mejora por sí mismo la calibración.

3.5. Pérdida y regularización

Para una etiqueta verdadera y, la entropía cruzada de una observación es $L = -\log p_y$. Asignar 0,9 a la clase correcta produce una pérdida de aproximadamente 0,105; asignar 0,1 produce 2,303. Se penaliza más una predicción muy segura y equivocada.

La pérdida media resume ejemplos y puede complementarse con una penalización de complejidad:

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} L(f_\theta(x_i), y_i) + \lambda\|w\|^2. \tag{3.3}$$

Esta es una formulación ilustrativa: las convenciones de escala y penalización exactas dependen de la biblioteca y el método. La regularización busca limitar ajustes excesivamente específicos. En la regresión logística de scikit-learn, un C menor implica regularización más fuerte; no conviene identificar sin comprobar todas las convenciones C con una fórmula universal de λ.

3.5.1. La pérdida distingue predicciones con la misma clase

Con etiqueta correcta en la primera posición, las distribuciones $(0{,}6, 0{,}3, 0{,}1)$ y $(0{,}9, 0{,}05, 0{,}05)$ aciertan ambas. Sus pérdidas son $-\log 0{,}6 \approx 0{,}511$ y $-\log 0{,}9 \approx 0{,}105$. El acierto no cambia,

pero la pérdida sí. Si la clase correcta fuese la tercera, una confianza mayor en la primera podría empeorar el resultado.

La media de pérdidas considera todos los ejemplos. Un caso muy seguro y equivocado puede contribuir mucho. La curva `loss_` de la MLP refleja el objetivo informado por el entrenamiento, que incluye su configuración de regularización; no debe compararse numéricamente con una tasa de acierto como si fueran la misma unidad.

Para profundizar: una derivada que explica el aprendizaje

Para softmax y entropía cruzada de un ejemplo,

$$L = -s_y + \log\left(\sum_j e^{s_j}\right), \qquad \frac{\partial L}{\partial s_k} = p_k - \mathbf{1}_{k=y}.$$

El símbolo $\mathbf{1}_{k=y}$ vale 1 si k es la clase correcta y 0 en otro caso. La derivada se obtiene al derivar cada término: el primero aporta -1 solo en la clase correcta; el segundo aporta p_k. Si pudiéramos modificar directamente las puntuaciones con un paso de descenso, aumentaríamos la correcta y reduciríamos las otras. En un modelo real, esas variaciones se propagan hacia pesos compartidos por muchos ejemplos.

3.5.2. Regularizar es introducir una preferencia

Dos conjuntos de pesos pueden ajustar de manera parecida los ejemplos disponibles. Una penalización cuadrática favorece pesos menores en la representación utilizada. Su gradiente es $2\lambda w$ con la convención de $\lambda\|w\|^2$ de este capítulo. Esto modifica el equilibrio entre ajustar entrenamiento y limitar ciertas soluciones; no demuestra que los pesos obtenidos sean físicamente interpretables.

Una regularización demasiado fuerte puede impedir aprender relaciones útiles y producir **subajuste**. Una regularización débil puede permitir un ajuste demasiado específico. En la actividad 3 se exploran valores de C y se comparan en validación. Para la MLP, `alpha` configura su propia penalización con la convención de esa implementación: no se equipara directamente al C del clasificador lineal.

3.6. Redes densas, activación y aprendizaje

Una capa densa calcula $z = Wx + b$. Una activación no lineal se aplica después. ReLU usa $\max(0, z)$ componente a componente. Si encadenáramos solo transformaciones lineales con sesgos, su composición seguiría siendo una transformación afín; la no linealidad permite representar relaciones diferentes.

La MLP del notebook recibe 1024 valores, usa 32 neuronas ocultas y tres salidas. Incluyendo sesgos, tiene $1024 \cdot 32 + 32 + 32 \cdot 3 + 3 = 32\,899$ parámetros. Contar parámetros ayuda a entender capacidad y memoria, pero no permite predecir directamente generalización.

Un **lote** contiene varios ejemplos usados en una actualización. Una **epoch** recorre entrenamiento. El optimizador usa gradientes de la pérdida para modificar pesos. En el notebook se observa la evolución durante 50 epochs; las etiquetas de validación se usan para medir, no para actualizar esos pesos.

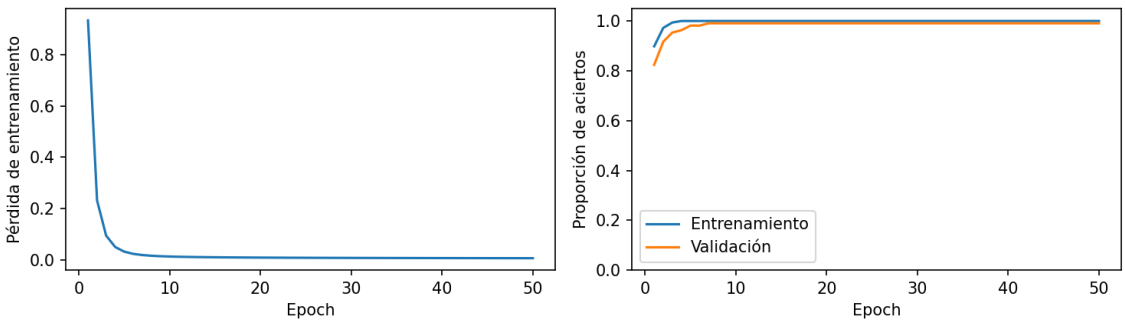


Figura 3.2: Curvas obtenidas en el ejemplo. Pérdida y acierto miden aspectos distintos; no tienen por qué variar al mismo ritmo.

3.6.1. Una pasada hacia delante de una red pequeña

Para ver la operación completa, tomemos dos entradas, dos neuronas ocultas y dos salidas:

$$h = \mathrm{ReLU}(W_1 x + b_1), \qquad s = W_2 h + b_2, \qquad p = \mathrm{softmax}(s).$$

Con $x = (0{,}2, 0{,}8)$, $W_1 = \left(\begin{smallmatrix} 1 & -1 \\ 1 & 1 \end{smallmatrix}\right)$ y $b_1 = (0, 0)$, la primera combinación da $(-0{,}6, 1)$. ReLU la transforma en $(0, 1)$. Si $W_2 = \left(\begin{smallmatrix} 1 & 2 \\ -1 & 1 \end{smallmatrix}\right)$ y $b_2 = (0, 0)$, obtenemos puntuaciones $(2, 1)$ y probabilidades aproximadas $(0{,}731, 0{,}269)$.

Estos pesos son un ejemplo fijado a mano, no pesos extraídos de la MLP del notebook. Una neurona oculta combina entradas, pero no tiene por qué corresponder a una propiedad con nombre como «agujero» o «borde».

3.6.2. Por qué necesitamos activaciones no lineales

Sin activación, dos capas darían $W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2)$. Podemos reunirlas en una sola transformación afín. Añadir capas de ese tipo no crea por sí solo nuevas fronteras no lineales. ReLU cambia la regla según qué combinaciones sean positivas y permite relaciones más variadas. En cero no tiene derivada clásica única; los programas utilizan una convención para el cálculo del gradiente allí.

3.6.3. Contar parámetros y actualizaciones

Una capa densa de d entradas y m salidas tiene dm pesos y m sesgos. Para 1024 entradas, ocho neuronas y tres clases, el total es $1024 \cdot 8 + 8 + 8 \cdot 3 + 3 = 8227$. Para 32 neuronas es 32 899. Reducir capacidad cambia el modelo y el tiempo de cálculo, pero no garantiza una mejora ni un empeoramiento en datos nuevos.

Con 324 ejemplos y lotes de hasta 64, un recorrido utiliza cinco lotes completos y uno de cuatro ejemplos: seis actualizaciones por epoch en esta configuración. Cincuenta recorridos representan unas 300 actualizaciones. El optimizador Adam adapta sus pasos usando información de gradientes anteriores; no es exactamente la regla de paso constante que usamos para introducir descenso por gradiente.

3.6.4. Qué observar en las curvas

La pérdida y el acierto de entrenamiento describen el ajuste a datos vistos. El acierto de validación observa un conjunto separado sin actualizar los pesos con sus etiquetas. Como el

acierto cambia cuando se cruza una frontera de decisión, puede permanecer constante mientras las probabilidades siguen cambiando. Las fluctuaciones pueden proceder de lotes, optimización o tamaño de muestra.

El notebook recorre 50 epochs mediante llamadas sucesivas a `partial_fit`. No selecciona automáticamente el mejor epoch ni restaura sus pesos. Si decidimos elegir un epoch mediante validación, esa elección forma parte del procedimiento que debe cerrarse antes de consultar prueba. Conservar pesos requiere hacerlo explícitamente: recordar la mejor cifra de una curva no restaura el estado de la red.

3.7. Sobreajuste y límites de la selección

El **sobreajuste** aparece cuando un modelo se adapta a detalles del entrenamiento que no se trasladan al uso esperado. Un acierto de entrenamiento alto y un resultado de validación peor pueden ser una señal; no existe una forma única obligatoria de las curvas. Un conjunto demasiado fácil puede ocultar fragilidad.

Probar muchas opciones y conservar la mejor también adapta decisiones a validación. Una gran búsqueda sobre una validación pequeña puede dar una impresión demasiado optimista. La prueba independiente y un registro de intentos limitan esa confusión, aunque no eliminan la necesidad de datos representativos.

3.7.1. Tres diagnósticos que conviene separar

Un resultado bajo en entrenamiento y validación puede indicar representación poco útil, subajuste o un problema de datos; no basta para elegir una causa. Un resultado alto en entrenamiento y peor en validación invita a revisar sobreajuste, dependencias y diferencias entre conjuntos. Un resultado alto en ambos con fallo en otra cámara sugiere que la evaluación anterior no cubría ese cambio de condiciones.

La primera acción útil puede ser mirar ejemplos y etiquetas antes de ampliar la red. Si el conjunto de validación no contiene las condiciones difíciles, escoger una arquitectura mayor no corrige esa falta de evidencia.

3.7.2. Aumentar datos conservando el significado

El **aumento de datos** genera variaciones de ejemplos de entrenamiento: pequeños desplazamientos o cambios de brillo pueden ser razonables si conservan la etiqueta y representan el uso esperado. Un giro podría cambiar la orientación que queremos distinguir; un recorte podría eliminar la parte decisiva. No toda transformación mejora un conjunto.

Las versiones de una misma pieza permanecen dentro de entrenamiento. Crear variaciones antes de separar y repartirlas entre conjuntos introduce dependencia. Al evaluar, hay que distinguir una prueba sobre piezas nuevas de una comparación entre transformaciones de los mismos ejemplos.

Ejemplo: No elegir otra vez después de ver prueba

Comparamos el lineal y la MLP en validación y elegimos el lineal. Después prueba revela fallos con una clase. Podemos formular una mejora y estudiar esos errores, pero no presentar el rendimiento de una nueva elección sobre ese mismo conjunto como una prueba independiente del proceso de selección. Para una nueva evaluación final se necesita otra reserva adecuada.

3.8. CNN: compartir filtros y usar estructura local

Una convolución combina vecinos con un filtro reutilizado espacialmente. En una imagen en gris, una expresión simplificada es

$$h(i,j) = b + \sum_{u,v} K(u,v) I(i+u, j+v). \tag{3.4}$$

En imágenes con canales se suma también sobre ellos. Muchas bibliotecas aplican técnicamente una correlación cruzada, sin invertir el filtro; en redes se mantiene el nombre habitual de convolución. Lo relevante aquí es la combinación local con pesos compartidos.

Un filtro 3×3 sobre RGB tiene 27 pesos. Con un sesgo y 16 filtros hay $(27+1)16 = 448$ parámetros, sin contar otras capas. Una capa densa desde una entrada $32 \times 32 \times 3$ a 16 neuronas tendría $3072 \cdot 16 + 16 = 49\,168$. No producen exactamente el mismo tipo de salida, pero la comparación muestra cómo compartir pesos cambia la parametrización.

Las capas sucesivas amplían el contexto que puede influir en una respuesta, llamado **campo receptivo**. El pooling y los cambios de escala resumen información. Estas operaciones pueden ayudar frente a ciertas variaciones, pero no garantizan invariancia ante cualquier desplazamiento, giro o cambio de cámara.

3.8.1. Un filtro que podemos calcular a mano

Sobre el bloque $\left(\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right)$ aplicamos el filtro $\left(\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right)$, sin invertirlo y sin sesgo. La suma de productos es $1 - 4 = -3$. Al desplazar ese mismo filtro a otro bloque, reutilizamos los mismos pesos. El filtro puede responder a un contraste local; durante entrenamiento sus pesos se aprenden, no tienen que elegirse a mano.

La **zancada** o *stride* indica cuántas posiciones avanzamos. El **relleno** o *padding* añade valores en bordes para controlar el tamaño de salida. Sin relleno y con paso 1, una entrada 5×5 y un filtro 3×3 producen una salida 3×3. Con varios filtros obtenemos varios mapas de características.

Para profundizar: contexto y desplazamiento

Con dos capas de filtros 3×3, paso 1 y sin otras operaciones, un valor interior de la segunda capa depende de una región 5×5 de la entrada. La segunda capa reúne tres posiciones que ya resumían vecindarios de tres: el campo receptivo crece. En cambio, bordes, relleno, submuestreo y recortes pueden cambiar la respuesta ante un desplazamiento. Compartir filtros favorece reutilizar patrones locales, pero no garantiza que la clasificación final sea idéntica en cualquier posición.

El *pooling* resume regiones, por ejemplo mediante su máximo. Para $(1, 4, 2, 3)$, el máximo es 4 y se pierde el resto de valores y parte de su localización. Resumir puede ayudar a reducir sensibilidad a cambios pequeños y también eliminar detalles importantes. La elección requiere comprobar la escala de los rasgos decisivos.

3.9. Aprendizaje por transferencia

Un extractor preentrenado transforma imágenes en características. Podemos congelarlo y aprender una nueva cabeza de clasificación. Después, si está justificado, el *ajuste fino* actualiza parte del extractor. Ambas decisiones requieren datos y evaluación: que los pesos procedan de un entrenamiento grande no prueba utilidad para una tarea específica.

El tamaño de entrada, orden de canales y normalización forman parte de la definición de uso de esos pesos. Transferir la arquitectura sin sus pesos no es el mismo procedimiento. En el tema 2, CNN y transferencia se explican conceptualmente; la práctica principal entrena un modelo lineal y una MLP desde cero.

3.9.1. Tres estrategias y qué se aprende en cada una

Entrenar desde cero.
Inicializamos los parámetros y los ajustamos con los datos de la tarea. Es lo que hacemos con los clasificadores del notebook.

Extractor congelado y cabeza nueva.
Aplicamos unos pesos preentrenados sin modificarlos y ajustamos un clasificador sobre sus características. Se aprende la cabeza; el extractor conserva su estado.

Ajuste fino.
Actualizamos además algunos o todos los parámetros del modelo preentrenado con datos de la tarea. Esto introduce más decisiones y puede perder parte de la representación útil si el ajuste es inadecuado.

La transferencia puede aprovechar regularidades visuales aprendidas previamente, pero una superficie industrial puede diferir mucho de los datos originales. Hay que comprobar escalas, textura y tipos de error relevantes. El nombre de la arquitectura no sustituye esa evaluación.

El preprocesamiento de los pesos forma parte del método. También importa evitar que imágenes relacionadas con evaluación hayan intervenido en entrenamiento o selección. En modelos externos la trazabilidad de los datos puede ser incompleta: conviene documentar lo conocido y el alcance de las conclusiones.

3.10. Matriz de confusión y cambio de condiciones

La matriz M_{ij} cuenta ejemplos de clase real i predichos como j. La diagonal contiene aciertos. El acierto global divide la suma de la diagonal entre todos los ejemplos. Es útil mirar qué clases se confunden, sus recuentos y sus consecuencias.

La actividad final desplaza imágenes del mismo conjunto de prueba. Es un diagnóstico de sensibilidad, no una evaluación nueva e independiente. Si después diseñamos una mejora a partir de ese diagnóstico, debemos evaluarla en otros datos reservados. El fallo de un modelo denso ante desplazamientos no demuestra que cualquier CNN resolvería automáticamente el problema.

3.10.1. Precisión, exhaustividad y soporte por clase

Para una clase k, tratamos esa clase como positiva y las demás como negativas. Los verdaderos positivos TP_k son aciertos de esa clase; los falsos positivos FP_k, ejemplos de otras clases predichos como k; y los falsos negativos FN_k, ejemplos de k predichos como otra clase. Entonces

$$\text{precision}_k = \frac{TP_k}{TP_k + FP_k}, \qquad \text{recall}_k = \frac{TP_k}{TP_k + FN_k}.$$

El **soporte** es el número de ejemplos reales de esa clase. Si un denominador es cero, la razón no está definida; el notebook pide informar cero mediante `zero_division=0`. Es una convención de presentación, no evidencia de una medición bien determinada.

Ejemplo: Leer una matriz completa

Supongamos filas reales y columnas predichas en orden arandela, placa y escuadra:

$$M = \begin{pmatrix} 8 & 2 & 0 \\ 1 & 9 & 0 \\ 0 & 3 & 7 \end{pmatrix}.$$

Hay 30 ejemplos y 24 aciertos: acierto global 0,8. Para placa, $TP = 9$, $FP = 2 + 3 = 5$ y $FN = 1$. Su precisión es $9/14 \approx 0{,}643$ y su recall, $9/10 = 0{,}9$. Detectar casi todas las placas no impide llamar «placa» a piezas de otras familias.

El indicador $F_1 = 2PR/(P + R)$ combina precisión P y recall R cuando la expresión está definida. El promedio *macro* da el mismo peso a cada clase; el promedio ponderado usa sus soportes. El acierto global coincide con el promedio de recalls ponderado por soportes en una clasificación de una sola etiqueta. Ninguno sustituye examinar consecuencias concretas de los errores.

3.10.2. Comprobación en Python: métricas con denominadores visibles

```
M = np.array([[8, 2, 0], [1, 9, 0], [0, 3, 7]])
acierto = np.trace(M) / M.sum()
recall = np.diag(M) / M.sum(axis=1)
precision = np.diag(M) / M.sum(axis=0)
print(acierto)                    # 0.8
print(np.round(recall, 3))        # [0.8 0.9 0.7]
print(np.round(precision, 3))     # [0.889 0.643 1.   ]
```

Los ejes se interpretan según la convención declarada. Si una herramienta intercambia filas y columnas, los denominadores también cambian. Aquí todos son positivos; otros casos necesitan tratar explícitamente la división por cero.

3.10.3. Cambio de condiciones: qué está comparando la práctica

La actividad 6 mueve las imágenes seis píxeles a la derecha y cuatro hacia abajo. El fondo de las nuevas posiciones vale 0,15. No se usa un desplazamiento circular que haga reaparecer la pieza por el lado contrario. Parte de la geometría puede quedar recortada; por ello el experimento combina cambio de posición y posible pérdida de contenido.

El modelo permanece fijo y las etiquetas se conservan como familias de las piezas originales. Si el recorte eliminase toda la información distintiva, la dificultad no se resolvería simplemente con más capacidad. Podemos usar el diagnóstico para proponer capturas o aumentos más adecuados, distinguiendo esa investigación de una nueva prueba independiente.

3.11. Guía de trabajo con las seis actividades

1. Datos y etiquetas.
Identifica qué cambia entre capturas y qué determina la clase. Cuenta piezas y capturas por separado. Explica por qué el código de clase no es una magnitud continua.

2. Separación.
Comprueba que no se comparten grupos y registra el número de ejemplos por clase. Decide

qué tipo de novedad pretende representar la prueba.

3. Referencias.
Compara mayoría y lineal antes de entrenar la red. Si pruebas varios C, registra sus resultados en validación y conserva la misma partición.

4. MLP.
Relaciona las 1024 entradas con el aplanado y las tres salidas con las etiquetas. Compara curvas sin exigir que pérdida y acierto varíen al mismo ritmo. Si cambias neuronas, reinicia y reentrena el modelo para esa configuración.

5. Selección y evaluación.
La regla del notebook elige MLP solo si supera al lineal en validación; en empate elige el lineal. Fija esa decisión antes de evaluar prueba y lee el informe por clases.

6. Diagnóstico.
Conserva modelo y preparación. Describe desplazamiento, relleno y posibles recortes. Distingue los errores observados de una conclusión general sobre CNN o cámaras reales.

Un registro útil contiene: partición, preparación, modelo, hiperparámetros, resultado de validación y motivo de elección. El resultado de prueba se añade al cerrar la selección. Si exploras varias configuraciones de la MLP, deja claro si el informe corresponde al último estado ejecutado o al estado que decidiste conservar.

3.12. Aplicación a producto y límites de la evidencia

Para clasificar familias de piezas en una línea, debemos definir si las capturas contienen una pieza completa, qué orientaciones se permiten y qué familias pueden aparecer. Conviene recoger piezas físicas diferentes y variaciones representativas de fondo, máquina, material y sesión, según el uso esperado.

Un buen resultado en los dibujos del curso demuestra que el procedimiento encuentra regularidades útiles en ese generador y en la partición utilizada. No estima directamente el rendimiento sobre fotografías. Cambiar a imágenes reales también puede exigir revisar la definición de etiqueta y las condiciones de captura.

Si confundir una escuadra con una placa tiene más coste que otra confusión, podemos informar errores por pareja de clases y formular una decisión acorde. Un sistema de derivación a revisión humana necesita evaluar tanto los errores como cuántos casos envía a revisión. No basta con eliminar del informe las predicciones difíciles.

3.13. Ejercicios con soluciones razonadas

1. Piezas y capturas

Hay 150 piezas con cuatro capturas cada una. Reservamos por pieza el 60 %, 20 % y 20 %. ¿Cuántas piezas y capturas corresponden a cada conjunto? ¿Podemos asegurar reparto idéntico de clases?

Solución. Son 90, 30 y 30 piezas; respectivamente 360, 120 y 120 capturas. La separación por grupos preserva piezas completas, pero no garantiza equilibrio de clases sin un procedimiento adicional que lo controle.

2. Una red más pequeña

Cuenta parámetros de una MLP con 100 entradas, diez neuronas ocultas y tres salidas, incluyendo sesgos.

Solución. La primera capa tiene $100 \cdot 10 + 10 = 1010$ parámetros y la segunda $10 \cdot 3 + 3 = 33$. En total son 1043. Las activaciones ReLU no añaden parámetros en esta arquitectura.

3. Confianza y pérdida

Dos predicciones correctas asignan a la clase verdadera probabilidades 0,6 y 0,9. Calcula las pérdidas e indica si el acierto permite distinguirlas.

Solución. Las pérdidas son aproximadamente 0,511 y 0,105. Ambas cuentan como un acierto, de modo que el indicador discreto no distingue la confianza asignada. Una menor pérdida en esos dos casos no demuestra por sí sola mejor calibración global.

4. Selección cerrada

El lineal obtiene 95 aciertos de 108 en validación y la MLP, 94. Después, en prueba, la MLP resulta mejor. ¿Cuál corresponde a la regla de selección previamente fijada?

Solución. El lineal, porque ganó en validación. Elegir después la MLP por prueba utiliza ese conjunto para seleccionar; no podemos presentar su resultado como evaluación independiente de esa nueva elección. Los recuentos son un ejemplo, no resultados del notebook.

5. Métricas por clase

Usa la matriz del ejemplo para calcular precisión y recall de escuadra. ¿Puede tener precisión perfecta sin recuperar todas las escuadras?

Solución. Hay siete aciertos, cero falsos positivos y tres falsos negativos. La precisión es $7/7 = 1$ y el recall $7/10 = 0{,}7$. Sí: todas las predicciones «escuadra» son correctas, pero faltan tres escuadras que se predijeron como placa.

6. Un filtro compartido

Una capa tiene ocho filtros 3×3 sobre imágenes RGB y un sesgo por filtro. ¿Cuántos parámetros contiene? ¿Cambia ese número al aumentar el ancho de entrada manteniendo la misma capa?

Solución. Son $8(3 \cdot 3 \cdot 3 + 1) = 224$. El número de parámetros de esta capa no depende del ancho de entrada: los filtros se reutilizan. Sí pueden cambiar el número de posiciones de salida, la memoria y el trabajo de cálculo, y otras capas posteriores podrían depender del tamaño espacial.

Conexión con las prácticas

Las actividades 1–2 definen datos y particiones; 3 compara referencias; 4 entrena la MLP; 5 selecciona en validación y evalúa; 6 diagnostica el desplazamiento. Distingue aprender parámetros de elegir hiperparámetros. Material de la sesión en el repositorio.

Capítulo 4

Tema 3. Detección y segmentación

Objetivos y recorrido

Este capítulo explica cómo pasar de una etiqueta global a salidas espaciales. Al terminar deberías poder distinguir una máscara de una caja, calcular solapamientos, explicar por qué una región conectada no siempre es una pieza y evaluar detecciones sin premiar duplicados.
La práctica aprende pieza/fondo a partir de RGB y obtiene cajas mediante componentes conectados. Las formas arandela, placa y escuadra aparecen en la escena, pero el modelo no predice esas tres clases. Los detectores profundos y segmentadores de instancias se estudian como ampliación conceptual.

4.1. La salida depende de la pregunta

Una etiqueta global puede indicar que hay piezas en una bandeja, pero no permite contarlas o medir su superficie. Una **detección** suele incluir clase, caja y puntuación por objeto. Una **segmentación semántica** asigna una clase a cada píxel. Una **segmentación de instancias** separa cada objeto con una identidad propia, incluso cuando varios pertenecen a la misma clase.

La segmentación panóptica combina etiquetas semánticas con identidades para objetos contables. No es una salida necesaria para todas las aplicaciones. La elección empieza por la decisión: contar, localizar, medir o inspeccionar.

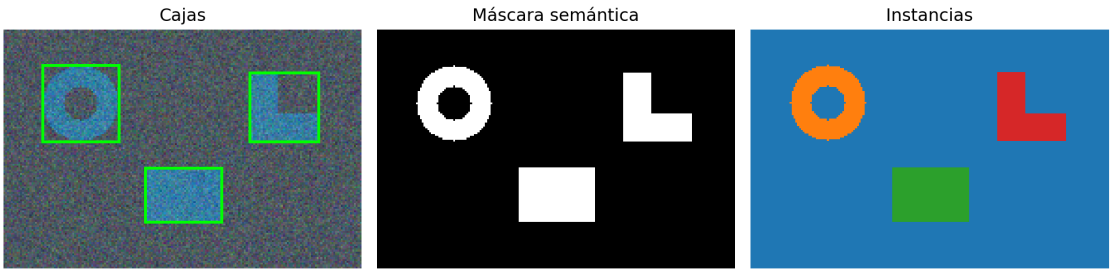


Figura 4.1: Cajas, máscara semántica e instancias de una escena sintética. Las referencias se conocen porque construimos los dibujos.

4.1.1. Cuatro preguntas sobre una misma escena

«¿Hay alguna pieza?» pide una decisión global. «¿Dónde está cada pieza?» pide localizaciones. «¿Qué píxeles pertenecen a material?» pide una máscara semántica. «¿Qué píxeles pertenecen a cada pieza individual?» pide instancias. Una salida puede permitir calcular otra bajo supuestos, pero no contiene necesariamente toda su información.

Ejemplo: Un agujero cambia qué queremos medir

Para estimar el área visible de material de una arandela, el agujero debe permanecer como fondo. Para localizarla en una bandeja, una caja que incluya el agujero puede ser suficiente. Para medir el diámetro interior, necesitamos un contorno y una calibración adecuados. «Segmentar bien» no es una especificación completa hasta definir qué propiedad se utilizará después.

4.1.2. Máscaras binarias y mapas de identificadores

Una **máscara binaria** M tiene el mismo alto y ancho que la imagen. En esta práctica, $M(i, j) = 1$ significa pieza y $M(i, j) = 0$ significa fondo. No es una imagen de intensidad: el 1 codifica pertenencia a una categoría. Podemos guardarla como booleanos o con otros códigos, siempre que la interpretación sea explícita.

Un mapa de instancias L asigna 0 al fondo y un entero positivo a cada objeto visible. Los valores 1, 2 y 3 identifican piezas; no expresan tamaño ni grado de defecto. Cambiar todos los identificadores mediante una permutación conserva la misma separación si se mantienen las regiones. Para obtener la máscara semántica del curso basta $M = (L > 0)$; ese paso elimina qué pieza originó cada píxel.

La operación inversa no es única. Una máscara unida puede corresponder a una pieza, a varias en contacto o a una predicción con un puente erróneo. El mapa semántico por sí solo no resuelve esas posibilidades.

4.1.3. La referencia también es una decisión

Las anotaciones de evaluación son una referencia según reglas declaradas. En imágenes reales puede haber bordes ambiguos, desenfoque o zonas ocultas. Antes de comparar modelos conviene decidir cómo tratar agujeros, objetos truncados, reflejos y regiones que no pueden anotarse con fiabilidad.

En el generador del notebook, una pieza dibujada después sobrescribe a las anteriores donde se solapan. Las instancias conservan solo las regiones visibles y la máscara semántica es su unión. Esta construcción da una referencia exacta para el dibujo, pero no representa toda la incertidumbre de una anotación humana.

4.2. Cajas y convenciones geométricas

El notebook usa $(x_{\text{mín}}, y_{\text{mín}}, x_{\text{máx}}, y_{\text{máx}})$, con extremos superiores excluidos. Anchura y altura son $w = x_{\text{máx}} - x_{\text{mín}}$ y $h = y_{\text{máx}} - y_{\text{mín}}$. Otros sistemas utilizan centro y tamaño, o coordenadas relativas a la imagen. Mezclar convenciones produce cajas desplazadas o de área incorrecta.

Una caja rectangular incluye fondo alrededor de objetos no rectangulares. En una arandela incluye su agujero. Por ello el área de la caja no equivale a superficie visible de material ni es una medida precisa del contorno.

4.2.1. Traducir entre formatos sin cambiar la caja

Para una caja con extremos excluidos,

$$x_c = \frac{x_{\text{mín}} + x_{\text{máx}}}{2}, \quad y_c = \frac{y_{\text{mín}} + y_{\text{máx}}}{2}, \quad w = x_{\text{máx}} - x_{\text{mín}}, \quad h = y_{\text{máx}} - y_{\text{mín}}.$$

Se recuperan los extremos como $x_{\text{mín}} = x_c - w/2$ y $x_{\text{máx}} = x_c + w/2$, y análogamente en vertical. Son coordenadas geométricas continuas de bordes bajo esta convención; no todos los centros corresponden al centro de un píxel entero.

La caja $(10, 20, 30, 35)$ tiene ancho 20, alto 15, centro $(20, 27{,}5)$ y área 300. En una imagen de ancho 100 y alto 50, sus extremos normalizados son $(0{,}1, 0{,}4, 0{,}3, 0{,}7)$, dividiendo las coordenadas horizontales por el ancho y las verticales por el alto. El orden de los cuatro números y la unidad deben acompañar al dato.

4.2.2. De índices de píxel a rectángulos dibujados

Si una región ocupa filas `20:35` y columnas `10:30`, los últimos índices presentes son 34 y 29. Su caja de extremos excluidos termina en 35 y 30. No se añade uno de nuevo al calcular el área con esta convención.

En la visualización del notebook, los centros de píxel están en coordenadas enteras. El borde izquierdo de la columna 10 se dibuja en 9,5. Por eso se desplaza el rectángulo de visualización media unidad, sin modificar la caja utilizada por la evaluación. Mezclar un ajuste gráfico con el cálculo geométrico introduciría un error innecesario.

4.2.3. Caja ajustada no significa contorno preciso

La caja que engloba una región usa sus mínimos y máximos, pero ignora la distribución interior. Dos máscaras muy distintas pueden tener la misma caja. Un único píxel aislado puede ampliar mucho una caja si no se ha filtrado; una pequeña erosión del extremo puede reducirla. La calidad de las cajas obtenidas hereda errores de la máscara y de la agrupación.

4.2.4. Comprobación en Python: una región rectangular

```
import numpy as np
M = np.zeros((50, 100), dtype=bool)
M[20:35, 10:30] = True
filas, columnas = np.nonzero(M)
caja = np.array([columnas.min(), filas.min(),
                 columnas.max()+1, filas.max()+1])
print(caja.tolist())              # [10, 20, 30, 35]
print(M.sum())                    # 300
print((caja[2]-caja[0]) * (caja[3]-caja[1]))  # 300
```

Aquí ambas áreas coinciden porque hemos creado un rectángulo lleno. En una arandela, el área de la caja incluye fondo exterior a la pieza y el agujero. Para una máscara vacía, los mínimos y máximos no están definidos: hay que tratar ese caso antes de construir la caja.

4.3. Un segmentador aprendido muy sencillo

El modelo de la práctica recibe tres números RGB por píxel. Aprende una regresión logística binaria con etiqueta pieza o fondo. La entrada de un píxel no incluye vecinos, coordenadas ni forma. Se entrena con píxeles de escenas de entrenamiento y se valida con otras escenas completas.

El mapa $p(i,j)$ contiene estimaciones del modelo. Un umbral t produce la máscara

$$\widehat{M}(i,j) = \begin{cases} 1, & p(i,j) \geq t, \\ 0, & p(i,j) < t. \end{cases} \tag{4.1}$$

Subir el umbral antes de la limpieza no puede añadir píxeles positivos al mapa binario. El número de regiones conectadas, sin embargo, puede aumentar si una región se divide o disminuir si desaparecen regiones. No debe inferirse que hay más objetos porque aumenten las componentes.

4.3.1. Qué aprende la regresión logística por píxel

El vector de entrada es $x = (R, G, B)$. Tras el escalado aprendido con entrenamiento, el modelo obtiene una puntuación lineal y la transforma mediante una sigmoide. Se puede escribir de forma conceptual como $p = \sigma(w \cdot \widetilde{x} + b)$, donde $\widetilde{x}$ representa el RGB escalado.

Si dos píxeles tienen exactamente el mismo RGB, reciben la misma salida, aunque uno esté en el borde de una placa y otro dentro de una arandela. El modelo no recibe esos contextos. Su utilidad depende de que las distribuciones de color de pieza y fondo sean suficientemente diferentes en las condiciones consideradas.

Ejemplo: Un límite de la información disponible

Imaginemos una pieza y un fondo con el mismo color y el mismo ruido, pero con una separación visible por textura. Un clasificador que solo vea un RGB aislado no puede aprovechar la disposición de esa textura. Una representación con vecindarios podría aportar información; ampliar únicamente el número de píxeles de entrenamiento no añade al modelo una entrada que no recibe.

4.3.2. Muestrear píxeles sin confundir unidades independientes

Se toman 1500 píxeles sin reemplazo de cada una de 12 escenas de entrenamiento: 18 000 filas RGB. El muestreo no fuerza igual número de piezas y fondo. La proporción depende del contenido de las escenas y de los píxeles seleccionados.

Aunque haya miles de filas, los píxeles de una escena comparten iluminación y construcción. No son miles de escenas independientes. Por eso validación utiliza otras escenas completas y no una reserva de píxeles de las mismas imágenes. El generador compartido sigue limitando la variedad representada por esa separación.

4.3.3. Tres decisiones que actúan en etapas diferentes

Decisión	Dónde actúa	Efecto principal
Umbral t	Mapa por píxel	Decide qué posiciones son positivas.
Área mínima	Componentes de la máscara	Descarta regiones pequeñas.
IoU mínima	Evaluación de cajas	Decide qué solapamientos admiten pareja.

Modificar la IoU mínima de evaluación no cambia la máscara predicha. Modificar el área mínima no reentrena la regresión. Distinguir estas etapas permite explicar de dónde procede una mejora o un fallo.

4.3.4. Elegir el umbral con validación

El notebook compara 0,25, 0,5 y 0,75 mediante la media de IoU de pieza en cuatro escenas de validación, después de filtrar regiones menores de 30 píxeles. Conserva el primer máximo de la lista si hay empate. La elección optimiza ese criterio entre esas opciones, no todas las tareas espaciales posibles.

Un umbral que mejora IoU semántica no tiene por qué mejorar el número de piezas o la localización de cada caja. Si cambiamos el objetivo del sistema, conviene definir un criterio de selección coherente y evaluarlo sin reutilizar prueba para elegir de nuevo.

4.4. Limpieza y conectividad

Una componente conexa reúne píxeles unidos según una vecindad. La conectividad de primer orden empleada en el notebook permite avanzar arriba, abajo, izquierda o derecha. Dos píxeles que solo se tocan por una esquina no se conectan directamente bajo esa regla.

Eliminar componentes menores de cierto área reduce pequeñas motas. También puede borrar piezas o defectos relevantes. Rellenar agujeros puede ser útil en algunos problemas, pero sería incorrecto si queremos conservar el hueco de una arandela. Las operaciones se justifican por la tarea, no por hacer que una máscara parezca más limpia.

4.4.1. Conectividad de cuatro y de ocho vecinos

Bajo cuatro vecinos, cada paso se hace horizontal o verticalmente. Bajo ocho vecinos se admiten además diagonales. La máscara

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

contiene dos componentes con cuatro vecinos y una con ocho. Los píxeles positivos no cambian: cambia la regla que los agrupa. Por eso el número de componentes requiere declarar conectividad.

Para etiquetar regiones, el programa recorre conexiones hasta asignar un identificador a cada grupo. Esos identificadores representan componentes geométricas y no clases aprendidas. Una pieza fragmentada puede generar varios identificadores; varias piezas unidas pueden generar uno.

4.4.2. Qué elimina el filtro de área

El área de una componente es su número de píxeles positivos. Con mínimo 30, una región de 29 se elimina y una de 30 se conserva. No se modifica automáticamente el contorno de las regiones mayores ni se rellenan sus agujeros. Una mota conectada por un puente a una región grande forma parte de esa región y no desaparece por ser pequeña por separado.

Ejemplo: El área mínima tiene escala

Con una escala de 0,1 mm por píxel en un plano, 30 píxeles cuadrados equivalen nominalmente a 0,3 mm^2. Si reducimos cada dimensión espacial a la mitad, el mismo objeto ocupa aproximadamente la cuarta parte de píxeles. Mantener el mínimo numérico 30 cambia qué tamaño físico se descarta. Es una aproximación geométrica; los bordes y el remuestreo pueden alterar el recuento exacto.

4.4.3. Morfología: una ampliación conceptual

Una **erosión** conserva posiciones cuyo vecindario satisface la condición de pertenencia y tiende a contraer regiones positivas. Una **dilatación** añade posiciones próximas y tiende a expandirlas. La vecindad o elemento estructurante determina la operación. Una **apertura** combina erosión y dilatación con el mismo elemento; un **cierre** invierte ese orden.

Estas operaciones pueden eliminar pequeños salientes, cerrar huecos o unir regiones según la configuración. También pueden destruir información de producto. No son equivalentes al filtrado por área y no se ejecutan como parte del procedimiento principal del notebook, que elimina componentes pequeñas. Presentarlas por separado evita atribuir a la práctica pasos que no utiliza.

Rellenar agujeros trata como material una región de fondo rodeada por positivos según una regla de conectividad. Si el agujero es una característica real de la pieza, esa corrección cambia la propiedad que medimos. La geometría del objeto y el objetivo deben justificar la operación.

4.5. IoU de máscaras y cajas

Para regiones A y B con unión no vacía, la intersección sobre unión es

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}. \tag{4.2}$$

Cuenta qué parte de todo lo cubierto por cualquiera de las dos regiones pertenece a ambas. Vale 1 cuando coinciden y 0 si no se solapan. Para dos máscaras vacías debe fijarse una convención; el notebook utiliza 1. Las escenas evaluadas contienen piezas y no dependen de ese caso.

Ejemplo: Dos cajas desplazadas

Dos cajas de 10×10, con el mismo intervalo vertical y un desplazamiento horizontal de 5, tienen intersección de 50. La unión es $100 + 100 - 50 = 150$; su IoU es $50/150 = 1/3$. Si el criterio acepta solo IoU de al menos 0,5, no se consideran una coincidencia.

En coordenadas de cajas, el ancho de intersección se calcula mediante

$$\text{máx}\Big(0, \text{mín}(x^A_{\text{máx}}, x^B_{\text{máx}}) - \text{máx}(x^A_{\text{mín}}, x^B_{\text{mín}})\Big)\,.$$

Se calcula análogamente la altura. El máximo con cero evita áreas negativas cuando las cajas están separadas.

Resultado: relación entre Dice e IoU

Si $I = |A \cap B|$ y $U = |A \cup B| > 0$, entonces $|A| + |B| = U + I$. El coeficiente Dice es $D = 2I/(|A| + |B|)$ y por tanto $D = 2J/(1 + J)$. Para $J = 0{,}5$, Dice vale $2/3$. No son porcentajes intercambiables. La relación se aplica a un mismo par de regiones; transformar una media de IoU no da necesariamente la media de Dice.

4.5.1. IoU como recuento de errores de la clase positiva

En una máscara binaria, la intersección cuenta verdaderos positivos TP, mientras que la unión contiene $TP + FP + FN$. Así,

$$\text{IoU} = \frac{TP}{TP + FP + FN}.$$

Los verdaderos negativos no aparecen: acertar una gran extensión de fondo no aumenta directamente este cociente. Para una imagen con mil píxeles de fondo por cada píxel de pieza, predecir todo como fondo puede dar un acierto global muy alto y una IoU de pieza igual a cero.

Ejemplo: Una máscara con diez píxeles omitidos

La referencia tiene 100 píxeles positivos. La predicción recupera 90 y añade 20 píxeles de fondo. Entonces $TP = 90$, $FN = 10$ y $FP = 20$. La unión tiene 120 píxeles y la IoU vale $90/120 = 0{,}75$. Dice vale $180/210 \approx 0{,}857$. No son dos estimaciones independientes: resumen los mismos recuentos mediante fórmulas diferentes.

4.5.2. Promediar escenas y reunir todos los píxeles

La media de IoU por escena da el mismo peso a cada imagen. Calcular una única IoU con todas las intersecciones y uniones reunidas pondera por el tamaño de las uniones. No son operaciones equivalentes.

Por ejemplo, una escena con intersección 9 y unión 10 tiene IoU 0,9. Otra con intersección 10 y unión 100 tiene IoU 0,1. La media por escena es 0,5, mientras que la IoU reunida es $19/110 \approx 0{,}173$. El notebook informa la media por escena; al comparar informes hay que comprobar la agregación.

4.5.3. Las regiones pequeñas son sensibles al borde

Una caja cuadrada de lado a desplazada una unidad horizontal mantiene intersección $a(a-1)$ y unión $a(a+1)$, si $a > 1$. Su IoU es $(a-1)/(a+1)$. Con $a = 2$ vale $1/3$; con $a = 20$, $19/21 \approx 0{,}905$. El mismo error absoluto de un píxel afecta de manera muy distinta según tamaño.

Una métrica de área tampoco destaca necesariamente un daño fino en un contorno. Si la decisión depende de una anchura o un borde, puede ser necesario complementar IoU con errores de medida o de contorno definidos para la aplicación. El criterio debe fijarse antes de elegir el modelo.

4.5.4. Comprobación en Python: máscara, IoU y Dice

```
a = np.zeros((20, 20), dtype=bool)
b = np.zeros_like(a)
a[0:10, 0:10] = True
b[0:10, 5:15] = True
inter = np.logical_and(a, b).sum()
union = np.logical_or(a, b).sum()
J = inter / union
D = 2 * inter / (a.sum() + b.sum())
print(inter, union)               # 50, 150
print(J, D)                      # 1/3 y 0.5
assert np.isclose(D, 2 * J / (1 + J))
```

Estas dos máscaras no están vacías. Si ambas lo estuvieran, las fórmulas exigirían una convención. La función de IoU de máscaras del notebook devuelve 1 en ese caso; la de cajas degeneradas con unión cero devuelve 0. Esas convenciones son distintas y no intervienen en el ejemplo anterior.

4.6. Por qué una máscara buena puede contar mal

Si dos piezas se tocan, pueden formar una sola componente aunque casi todos sus píxeles se separen correctamente del fondo. La IoU semántica puede ser alta y el recuento de objetos erróneo. El objetivo de una métrica debe coincidir con la decisión de la aplicación.

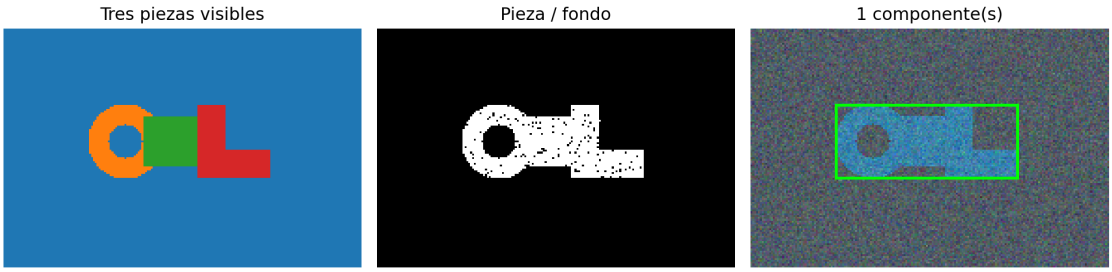


Figura 4.2: Tres piezas visibles forman una región conectada. La clasificación pieza/fondo no ha aprendido dónde separar sus identidades.

Las referencias del notebook identifican solo las partes visibles. Inferir la forma completa detrás de una oclusión requeriría otro criterio de anotación y otra evaluación. No debe llamarse error a una predicción por no incluir una parte oculta si la referencia solo describe lo visible.

4.6.1. Un solo píxel puede cambiar el recuento

Imaginemos dos cuadrados de 10×10 separados por una columna de fondo. La referencia semántica contiene 200 píxeles de pieza. Si la predicción añade un único píxel que conecta los cuadrados por cuatro vecinos y mantiene el resto exacto, la IoU es $200/201 \approx 0{,}995$. Aun así, aparece una sola componente en lugar de dos.

El problema contrario también ocurre: retirar un pequeño puente de material puede fragmentar una pieza. No hay una relación universal entre subir IoU y acercar el número de componentes al número de objetos. Las instancias necesitan una definición de identidad y un método que pueda separarlas.

4.6.2. Separar objetos unidos: qué información podría ayudar

Podemos introducir indicaciones de centro, contornos aprendidos o una representación de instancias. Otro procedimiento geométrico, la transformación de distancia seguida de una separación tipo *watershed*, puede resultar útil en algunas formas y condiciones. Necesita supuestos sobre las piezas y marcadores adecuados; no garantiza separar cualquier contacto u oclusión. No se aplica en este notebook.

Si la imagen oculta la separación de dos piezas idénticas, quizá se necesiten otra vista, profundidad o una modificación de la presentación de las piezas. Una arquitectura más compleja no crea automáticamente evidencia visual ausente.

4.7. Evaluar detecciones sin contar dos veces

Para declarar una detección correcta hay que emparejarla con una referencia. Cada predicción y cada referencia se usan como máximo una vez. Una caja emparejada es un verdadero positivo; una caja sin pareja, un falso positivo; una referencia no recuperada, un falso negativo.

Ejemplo: Duplicados

Hay una pieza real y dos cajas idénticas que la cubren perfectamente. Solo una puede ser verdadero positivo; la otra es un falso positivo. Informar dos aciertos por tener dos cajas con IoU alta premiaría un comportamiento indeseable.

La práctica utiliza una clase, pares ordenados por IoU decreciente y emparejamiento voraz con mínimo 0,5. Es un protocolo docente explícito. No es AP o mAP: esos protocolos incorporan puntuaciones, criterios de ordenación y agregaciones que deben especificarse por separado.

4.7.1. Un protocolo de emparejamiento explícito

El procedimiento docente calcula todas las IoU entre referencias y predicciones, ordena las parejas candidatas por IoU decreciente y acepta una pareja si supera o iguala el mínimo y ninguna de sus dos cajas se ha usado. Al final cuenta referencias sin pareja como FN y predicciones sin pareja como FP.

Con tres referencias y cuatro predicciones, si acepta dos parejas, resulta $TP = 2$, $FP = 2$ y $FN = 1$. La precisión es $2/4 = 0{,}5$ y el recall $2/3$. Los nombres de las métricas son los mismos que en clasificación, pero ahora un acierto requiere también una localización aceptable y una pareja única.

4.7.2. El emparejamiento voraz no maximiza siempre el número de parejas

Consideremos esta tabla hipotética de IoU, con referencias por filas y predicciones por columnas:

	P_1	P_2
R_1	0,8	0,6
R_2	0,6	0,3

Con mínimo 0,5, la regla voraz toma primero R_1 con P_1, y la pareja restante no alcanza el mínimo: obtiene un acierto. Elegir R_1 con P_2 y R_2 con P_1 daría dos parejas. Esta tabla ilustra la lógica del algoritmo; no describe cajas concretas de la práctica. Explica por qué hay que declarar el protocolo al comparar resultados.

4.7.3. Umbral de evaluación y métricas agregadas

Aumentar la IoU mínima exige localizaciones más coincidentes. Para el emparejamiento voraz con orden de candidatos fijo del notebook, no se aceptan más parejas al subir ese mínimo: solo se corta antes la lista de candidatos. Precisión y recall del mismo conjunto de cajas no aumentan por ese cambio si sus denominadores están definidos.

En otros protocolos, las predicciones se ordenan por puntuación del detector para construir una curva de precisión y recall. La precisión media, AP, resume esa curva bajo una definición concreta; mAP agrega entre clases y, según el protocolo, umbrales de IoU. La práctica no calcula esas curvas ni genera una puntuación de confianza propia para cada caja: no hay que llamar mAP a su precisión de un punto de operación.

Las seis escenas de prueba aportan 18 referencias visibles en esta construcción. Tener solo unas pocas escenas limita la evidencia sobre variaciones de captura, aunque se puedan contar muchos píxeles. Los diagnósticos de contacto y color nuevo se informan por separado y no se usan para elegir de nuevo el umbral.

4.8. Modelos profundos: qué añaden

Un detector aprende a producir localizaciones y clases con características de imagen. Familias como YOLO y modelos basados en propuestas o predicción de conjuntos usan mecanismos distintos. Algunos generan cajas solapadas del mismo objeto y emplean supresión no máxima, NMS, para filtrar duplicados según puntuaciones y solapamiento.

NMS no es el umbral de puntuación del modelo ni el umbral de IoU de evaluación. Suprimir agresivamente cajas cercanas puede eliminar objetos distintos. Tampoco todos los detectores requieren la misma posoperación.

En segmentación, una arquitectura como U-Net combina información de distintas escalas y recupera resolución mediante un decodificador. Mask R-CNN produce máscaras por objeto. Un modelo guiado, como SAM, usa indicaciones para segmentar; obtener una máscara no le asigna automáticamente una clase industrial correcta. Estas familias se presentan conceptualmente en el curso y no se ejecutan en la práctica principal del tema 3.

4.8.1. Localización y clasificación se aprenden conjuntamente

Un detector profundo utiliza características visuales para proponer o producir cajas y sus clases. Sus pérdidas suelen combinar términos de clasificación y de localización. Necesita anotaciones y un protocolo de entrenamiento coherentes. Una caja con buena clase y mala posición no es equivalente a una caja bien localizada de clase equivocada.

Un segmentador semántico profundo produce salidas por píxel utilizando contexto: puede reconocer relaciones espaciales que el clasificador RGB aislado no recibe. Reducir la resolución interna permite reunir contexto, mientras que recuperar una salida detallada requiere combinarlo con información espacial. El compromiso importa para piezas pequeñas y bordes finos.

Un segmentador de instancias añade separación entre objetos. La máscara de una instancia y su clase tienen funciones distintas: delimitar una región no prueba haber identificado su categoría ni su conformidad. Los modelos guiados por puntos o cajas necesitan además evaluar la interacción humana o el procedimiento que genera esas indicaciones.

4.8.2. No confundir tres tipos de umbral

Un detector puede descartar cajas por puntuación baja. Puede después aplicar NMS para reducir duplicados entre cajas predichas usando otro umbral de IoU. Finalmente, la evaluación compara predicciones con referencias con su propio umbral de IoU. El mismo nombre «IoU» aparece en las dos últimas etapas, pero compara pares distintos y cumple funciones diferentes.

La práctica principal no aplica NMS. Sus cajas proceden de componentes. Añadir NMS no separaría dos piezas que ya se han fusionado en una sola componente: para eso haría falta recuperar dos candidatos o dos regiones antes de filtrarlos.

4.9. De píxeles cuadrados a medidas físicas

Con escala uniforme de s mm por píxel en un plano, un área de N píxeles representa aproximadamente Ns^2 mm^2. Si $s = 0{,}1$, cada píxel representa 0,01 mm^2. El supuesto puede fallar por perspectiva, distorsión, superficies fuera del plano o contornos inciertos. La conversión de unidades no sustituye calibración y validación de medida.

4.9.1. Una escala para cada dirección

Si la escala horizontal es s_x mm por píxel y la vertical es s_y, el área nominal es

$$A = N s_x s_y.$$

Con $N = 2400$, $s_x = s_y = 0{,}05$, resulta $A = 6$ mm^2. Una caja de 80 por 30 píxeles tiene también 2400 píxeles cuadrados de extensión, pero solo coincide con área visible de material si toda la caja está ocupada por él. Una máscara con huecos ocupa menos.

Si una escala uniforme s se sobreestima un 10 %, el área calculada se multiplica por $1{,}1^2 = 1{,}21$: un error del 21 %, aun con una máscara perfecta. Esto ilustra la importancia de la calibración; no es una estimación de incertidumbre del sistema real. Si la escala varía con la posición por perspectiva, no basta una constante global.

4.9.2. Qué superficie representa la máscara

El área de una proyección en imagen no es necesariamente el área de una superficie curva o inclinada. Tampoco incluye material oculto tras otra pieza cuando las anotaciones describen solo lo visible. Para estimar otra propiedad se necesita una relación geométrica o un sistema de medida adicional.

Un límite de área en píxeles puede servir para experimentar, pero un requisito de producto suele expresarse en unidades físicas o en una decisión de conformidad. Hay que relacionar ambos mediante un procedimiento calibrado y comprobar su error con referencias adecuadas.

4.10. Guía de trabajo con las seis actividades

1. Definir salidas.
Señala qué información conserva la caja, la máscara binaria y el mapa de instancias. Comprueba que el agujero de la arandela se mantiene como fondo.

2. Aprender RGB.
Cuenta escenas y píxeles por separado. Explica por qué un píxel no incluye forma y por qué se reservan escenas completas para validación.

3. Elegir umbral.
Registra la media de IoU de cada opción en validación con el mismo filtro de área. Compara sin limpieza allí y explica qué regiones se eliminan. Cierra las decisiones antes de prueba.

4. Obtener cajas.
Comprueba orden de coordenadas, extremos excluidos y área de intersección. Contrasta el área de una región no rectangular con el área de su caja.

5. Estudiar contacto.
Compara número de componentes y piezas visibles junto a IoU semántica. Describe qué separación de identidades falta, aunque la máscara parezca correcta.

6. Evaluar.
Suma TP, FP y FN entre escenas antes de calcular precisión y recall de cajas, como hace el notebook. Informa por separado la media de IoU semántica, prueba y diagnósticos.

Conserva una tabla que indique origen de la escena, umbral por píxel, área mínima, conectividad e IoU mínima de evaluación. Acompaña los promedios de ejemplos concretos: una región omitida, una unión indebida o una caja ampliada por un error. Si no hay errores en una escena, comenta una limitación que el ejemplo no haya puesto a prueba.

4.11. Errores frecuentes y aplicaciones a producto

Contar componentes como piezas sin verificar.
Revisa contactos, puentes y fragmentación antes de interpretar el recuento. La conectividad describe la máscara, no la identidad física.

Usar área de caja para medir material.
Comprueba huecos y concavidades. Para superficie visible utiliza una máscara de referencia adecuada y una conversión geométrica válida.

Confundir buen promedio con buen borde.
Una IoU alta puede ocultar un error pequeño decisivo para una cota o una marca fina. Añade una evaluación vinculada a la decisión cuando sea necesario.

Optimizar la máscara mirando prueba.
La limpieza y el umbral también son parte del método elegido. Si se ajustan tras observar prueba, se requiere nueva evaluación independiente.

Extrapolar del color sintético a fotografías.
Las sombras, los reflejos y los materiales pueden alterar la relación RGB entre pieza y fondo. Hay que recoger escenas representativas y anotaciones revisadas.

En una bandeja de piezas, contar puede requerir instancias aunque la clasificación pieza/fondo funcione bien. En una inspección de poros visibles, eliminar componentes pequeñas podría borrar precisamente el defecto buscado. En una estimación de área proyectada, mantener agujeros y declarar oclusiones resulta esencial. La misma operación puede ayudar en una tarea y perjudicar otra.

4.12. Ejercicios con soluciones razonadas

1. Convertir una caja

Una caja tiene extremos $(12, 8, 32, 18)$. Calcula ancho, alto, centro y área con la convención del notebook.

Solución. El ancho es 20 y el alto 10. El centro geométrico es $(22, 13)$ y el área, 200 píxeles cuadrados. No se añade uno porque los extremos superiores están excluidos.

2. Elegir la conectividad

¿Cuántas componentes tiene $\left(\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{smallmatrix}\right)$ con cuatro vecinos? ¿Y con ocho? ¿Cuántos píxeles positivos tiene en cada caso?

Solución. Hay tres componentes con cuatro vecinos y una con ocho. En ambos casos hay tres píxeles positivos. Cambia su agrupación, no la máscara ni su área.

3. Recuentos de una máscara

Una referencia contiene 80 píxeles de pieza. La predicción recupera 60 y añade 20 de fondo. Calcula IoU y Dice.

Solución. $TP = 60$, $FN = 20$ y $FP = 20$. La unión es 100 y la IoU, 0,6. La predicción tiene 80 píxeles, de modo que Dice es $120/160 = 0{,}75$, igual a $2 \cdot 0{,}6/(1 + 0{,}6)$.

4. Duplicados y objetos omitidos

Hay tres referencias. Se predicen tres cajas: dos duplicadas sobre la primera pieza y una correcta sobre la segunda. Todas las cajas indicadas coinciden perfectamente con su pieza y no con las otras. Calcula métricas con emparejamiento uno a uno.

Solución. Solo una de las duplicadas puede emparejarse con la primera referencia. Con la segunda pareja obtenemos $TP = 2$, $FP = 1$ y $FN = 1$. Precisión y recall son $2/3$. La tercera referencia permanece omitida aunque existan tres predicciones.

5. Área física y reducción

Una máscara contiene 1200 píxeles de material, con escala uniforme de 0,1 mm por píxel. ¿Cuál es su área nominal? Si se reduce cada dirección a la mitad, ¿qué escala corresponde y cuántos píxeles esperarías aproximadamente para conservar la misma área?

Solución. El área nominal es $1200 \cdot 0{,}1^2 = 12$ mm^2. Tras reducir, la escala es 0,2 mm por píxel y esperamos unos 300 píxeles: $300 \cdot 0{,}2^2 = 12$. La igualdad es geométrica ideal; la discretización del borde puede alterar el recuento real.

6. Una máscara casi perfecta que cuenta mal

Dos piezas tienen en total 200 píxeles. La predicción añade un píxel de fondo que las conecta y mantiene todos los demás correctos. ¿Qué IoU tiene y cuántas componentes puede producir con cuatro vecinos?

Solución. La IoU es $200/201 \approx 0{,}995$. Si ese píxel crea el puente indicado, aparece una componente en lugar de las dos piezas. Un buen solapamiento semántico no garantiza un recuento correcto.

Conexión con las prácticas

La actividad 1 compara salidas; 2 aprende pieza/fondo; 3 selecciona umbral y limpieza; 4 obtiene cajas y calcula IoU; 5 estudia contacto; 6 separa prueba y diagnósticos. El mapa y las cajas comparten origen, pero no resuelven necesariamente la misma pregunta. Material de la sesión en el repositorio.

Capítulo 5

Tema 4. Inspección y detección de anomalías

Objetivos y recorrido

Aprenderemos a construir una referencia de superficies correctas, convertir desviaciones locales en una puntuación y elegir una decisión de alarma. Al terminar deberías poder explicar por qué «inusual» no significa necesariamente «defectuoso», interpretar los denominadores de las tasas y describir qué supuestos necesita el método.
La práctica utiliza una referencia estadística por posición, no un autoencoder ni una red profunda. Las seis actividades separan aprendizaje de referencia, selección del umbral, evaluación y diagnóstico. Los modelos profundos aparecen como ampliación conceptual. No se necesita GPU ni descargar datos.

5.1. Normalidad estadística y conformidad

Una anomalía se aparta de una referencia de datos. Un defecto incumple una especificación. Un acabado admisible poco frecuente puede ser anómalo; una imperfección repetida en la referencia puede parecer normal. El modelo no define por sí mismo qué es aceptable para el producto.

Aprender lo normal es útil cuando hay muchas piezas correctas y pocos ejemplos de todos los posibles defectos. No garantiza detectar cualquier fallo desconocido. La representación usada decide qué variaciones se vuelven visibles para el modelo.

5.1.1. Tres formas de plantear la inspección

En una clasificación supervisada de defectos, entrenamos con ejemplos etiquetados de las categorías que queremos distinguir. En una estrategia basada en normalidad, modelamos lo admisible y buscamos desviaciones. También podemos construir una referencia de normalidad y utilizar ejemplos etiquetados posteriores para elegir cómo convertir sus puntuaciones en alarmas, como hacemos aquí.

La práctica no utiliza defectos para estimar la media y la desviación de referencia, pero sí los utiliza en validación para escoger un umbral. Por tanto, el procedimiento completo no prescinde de etiquetas de defecto en todas sus fases. Decir «aprendemos solo con correctas» describe el ajuste de la referencia, no toda la selección experimental.

Ejemplo: Una textura rara y un defecto repetido

Una pieza correcta con un acabado poco frecuente puede producir alarma porque no está bien representada. En cambio, si una marca inadmisible aparece de manera repetida entre las piezas etiquetadas como correctas, puede integrarse en la referencia. El algoritmo aprende regularidades de sus datos; la conformidad depende de una especificación y de cómo se revisaron las muestras.

5.1.2. Qué debe representar el conjunto normal

Conviene incluir variación admisible de iluminación, textura, material o posición dentro de las condiciones de uso definidas. Ampliar la variación puede reducir alarmas sobre piezas correctas, pero también hacer menos destacadas algunas anomalías. No se trata de aceptar cualquier cambio observado como normal.

La revisión de las piezas de referencia necesita un criterio independiente del algoritmo que estamos construyendo. Si usamos sus propias alarmas para decidir automáticamente qué ejemplos son correctos y los incorporamos sin revisión, podemos reforzar sus errores. Un registro de procedencia y versión permite saber con qué normalidad se comparó cada captura.

5.1.3. Los datos concretos del notebook

Se generan imágenes en gris de 64×64. La superficie tiene textura sinusoidal, ruido y variación simulada de brillo. Las rayas se dibujan como marcas oscuras alargadas; los poros, como marcas pequeñas y más débiles. Son construcciones docentes, no simulaciones completas de esos defectos físicos.

Hay 60 imágenes correctas para aprender la referencia, 80 imágenes de validación con 40 correctas y 40 defectuosas, y 120 de prueba con 60 de cada grupo. En los defectos de validación hay 20 rayas y 20 poros; en prueba, 30 de cada tipo. Las semillas identifican imágenes distintas y no se comparten entre conjuntos. Compartir el mismo generador sigue limitando la variedad de condiciones evaluadas.

5.2. Preparación y región de interés

La práctica usa superficies sintéticas alineadas. Se define una ROI interior que excluye borde y agujero. Solo se puntúan sus píxeles. No podemos reclamar detección de defectos fuera de esa región.

A cada imagen se le resta la mediana de su ROI. Esa operación reduce cambios globales aditivos de intensidad y puede compensar parcialmente la variación simulada de brillo. No elimina todos los efectos de una iluminación real ni asegura invariancia frente a escalados multiplicativos de contraste.

La mediana es el valor central de una lista ordenada. Es menos sensible a pocos valores extremos que la media. Si una anomalía ocupa una parte grande de la ROI, también puede modificar la mediana y alterar la preparación.

5.2.1. Definir la ROI también define el alcance

ROI significa *región de interés*. Aquí es una máscara fija obtenida de la geometría del dibujo: selecciona una zona interior y deja fuera bordes y agujero. No es una región aprendida por segmentación ni se desplaza automáticamente al moverse la pieza.

Si un defecto queda fuera de la ROI, no contribuye a la puntuación. Una evaluación que solo genera marcas dentro de la ROI no informa de la capacidad de inspeccionar toda la superficie. En una aplicación real hay que declarar qué zonas están cubiertas y cuáles se revisan mediante otro procedimiento.

5.2.2. Restar la mediana: qué elimina exactamente

Sea $m(x) = \text{mediana}\{x(i,j) : (i,j) \in \text{ROI}\}$. La imagen preparada es $\widetilde{x}(i,j) = x(i,j) - m(x)$. Los valores negativos son normales: indican intensidad inferior a la mediana de esa captura, no intensidades físicas negativas.

Si añadimos una constante c a todos los valores, sin saturación ni otros cambios, la mediana también aumenta en c. Por tanto,

$$\widetilde{x + c} = (x + c) - (m(x) + c) = x - m(x) = \widetilde{x}.$$

La operación elimina exactamente ese desplazamiento aditivo ideal. Si multiplicamos la imagen por $a > 0$, la preparación produce $a\widetilde{x}$, y la amplitud no desaparece. Sombras locales, reflejos y cambios de contraste tampoco se reducen a una constante común.

Ejemplo: Preparar una pequeña región

Para valores 0,50, 0,60, 0,70, la mediana es 0,60 y los preparados son −0,10, 0 y 0,10. Añadir 0,05 a los tres valores conserva ese resultado. Multiplicarlos por 2, sin recortar, produce valores preparados −0,20, 0 y 0,20. El segundo cambio no ha sido eliminado.

Una marca que modifica muchos píxeles puede desplazar la mediana y afectar a toda la imagen preparada. Incluso una marca pequeña puede cambiarla algo según los valores presentes. La mediana es resistente a algunos extremos, pero no constituye una estimación infalible de iluminación.

5.2.3. Comprobación en Python: compensación aditiva

Las pequeñas celdas de cálculo de este capítulo son independientes del notebook salvo indicación contraria.

```
import numpy as np
x = np.array([0.50, 0.60, 0.70])
preparar_vector = lambda a: a - np.median(a)
print(np.round(preparar_vector(x), 3))  # [-0.1 0. 0.1]
assert np.allclose(preparar_vector(x + 0.05), preparar_vector(x))
assert np.allclose(preparar_vector(2*x), 2*preparar_vector(x))
```

El segundo cálculo es un ejemplo algebraico sin limitar al intervalo $[0, 1]$. Si introducimos saturación, la relación puede dejar de cumplirse. No hay que atribuir a una captura real las propiedades exactas de una transformación ideal.

5.3. Referencia por posición y mapa de desviación

Para cada posición (i,j), se estima media $\mu(i,j)$ y desviación $\sigma(i,j)$ usando imágenes correctas preparadas. El mapa es

$$z(i,j) = \frac{|x(i,j) - \mu(i,j)|}{\max(\sigma(i,j), \sigma_{\text{mín}})}. \tag{5.1}$$

El valor absoluto trata desviaciones positivas y negativas como alejamiento. El suelo $\sigma_{\text{mín}} = 0{,}025$ del ejemplo impide dividir por estimaciones diminutas. Esa cifra es una decisión docente previa, no una constante física o estadística universal.

Ejemplo: Interpretar una desviación

Si una posición tiene media 0,20 y desviación de referencia 0,04, observar 0,32 da $z = |0{,}32 - 0{,}20|/0{,}04 = 3$. Significa tres desviaciones de alejamiento respecto a esa referencia. No demuestra por sí solo una probabilidad concreta de defecto: faltan un modelo probabilístico adecuado y una relación validada con las etiquetas de calidad.

Esta referencia compara posiciones. Si la pieza se desplaza, la misma coordenada puede pasar de material a agujero. La desviación será grande aunque no haya un defecto. La alineación es un supuesto del método, no una mejora opcional de presentación.

5.3.1. Una referencia por posición, no por imagen

Con $n = 60$ imágenes preparadas, la media de una posición se calcula a través de esas 60 capturas:

$$\mu(i,j) = \frac{1}{n}\sum_{r=1}^{n} \widetilde{x}_r(i,j), \qquad \sigma^2(i,j) = \frac{1}{n-1}\sum_{r=1}^{n} (\widetilde{x}_r(i,j) - \mu(i,j))^2.$$

El índice r recorre imágenes; i, j recorren posiciones. No estamos calculando una única desviación de todos los píxeles de una pieza. El resultado son dos mapas, de la misma forma espacial que la entrada.

Una posición muy variable recibe mayor denominador y una misma diferencia absoluta produce menor puntuación. Eso puede reducir alarmas sobre variación admisible y dificultar detectar defectos que se parezcan a ella. Si los datos normales contienen sistemáticamente una marca, su efecto también puede incorporarse a la media.

5.3.2. Suelo de desviación y significado de la escala

Con diferencia absoluta 0,05 y desviación estimada 0,005, dividir sin suelo daría 10. Con el mínimo 0,025 utilizado en la práctica, da 2. El suelo limita la amplificación en posiciones aparentemente estables. Cambiarlo altera el mapa y, por tanto, las puntuaciones y el umbral útil.

Las imágenes originales están en escala $[0, 1]$, de modo que 0,025 se expresa en esa escala de intensidades. Si trabajásemos con intensidades de 0 a 255, necesitaríamos transformar coherentemente tanto los datos como ese suelo: $255 \cdot 0{,}025 = 6{,}375$. Cambiar unidades solo en una parte modifica el método.

El mapa z es no negativo y no está limitado a 1. Puede superar 10 aunque la visualización se limite a ese valor. Un mapa mostrado con máximo 10 puede saturar visualmente regiones diferentes sin que sus puntuaciones sean iguales.

Una desviación estandarizada no es una probabilidad

Incluso bajo una distribución gaussiana ideal por posición, convertir z en una probabilidad de cola requeriría ese supuesto y el tratamiento de parámetros estimados. Una probabilidad de cola tampoco es $P(\text{defecto} \mid x)$. Además, aquí restamos una mediana estimada por imagen, aplicamos un suelo y seleccionamos valores extremos. No se justifica una interpretación probabilística universal a partir de una regla como «$z > 3$».

5.3.3. Por qué la alineación es esencial

El método compara siempre la misma coordenada. Si desplazamos una pieza, una posición que contenía material puede mostrar agujero o fondo. La diferencia se vuelve grande aunque la pieza sea correcta. Una ROI interior deja cierto margen, pero no garantiza cubrir movimientos arbitrarios.

Alinear significa estimar una correspondencia geométrica con una referencia. Esa transformación puede necesitar interpolación y tener error residual. La evaluación de un sistema que realinea debe incluir el procedimiento completo y los posibles fallos de alineación, no solo imágenes perfectamente registradas. En el notebook no se añade ese paso: el desplazamiento final sirve para mostrar el supuesto incumplido.

5.4. Del mapa a la puntuación de una pieza

Una media de todo el mapa puede diluir marcas pequeñas. El notebook toma el 1 % de píxeles de ROI con mayor desviación y promedia sus valores. Si hay N píxeles y $k = \lceil 0{,}01N \rceil$, la puntuación es la media de esos k valores superiores.

No usamos solo el máximo, que podría reaccionar a una mota aislada. Tampoco se garantiza detectar un poro débil: su señal puede mezclarse con la textura normal. Elegir porcentaje, suelo y preparación forma parte del diseño y debe hacerse antes de la evaluación final.

5.4.1. Máximo, media y cola superior responden distinto

Consideremos una ROI con 1000 posiciones. En una imagen, 990 valores de desviación son 1 y diez son 6. La media global vale $(990 + 60)/1000 = 1{,}05$, el máximo es 6 y la media del 1 % superior también es 6. En otra imagen, 999 valores son 1 y uno es 12. La media global es 1,011, el máximo es 12 y la media de los diez mayores es $(12 + 9)/10 = 2{,}1$.

El máximo reacciona mucho a un extremo aislado. La cola superior exige más apoyo espacial en número de píxeles, aunque no en su disposición: diez puntos separados pueden dar la misma puntuación que una región compacta con esos valores. La media global puede diluir cambios muy localizados.

5.4.2. El porcentaje es parte del método

Para N valores usamos $k = \max(1, \lceil 0{,}01N \rceil)$. Redondear hacia arriba garantiza seleccionar al menos la fracción solicitada en número de posiciones. En la ROI concreta del notebook hay 1127 píxeles y por tanto $k = 12$. La fracción realmente seleccionada es $12/1127 \approx 1{,}065\,\%$.

Seleccionar los valores mayores no equivale a tomar un percentil: el percentil describe un nivel de la distribución ordenada; aquí promediamos una cola de valores. La operación `partition` permite encontrar esa cola sin ordenar toda la lista, manteniendo el mismo conjunto de mayores valores salvo empates de igual magnitud.

5.4.3. Comprobación en Python: tres resúmenes

```
a = np.r_[np.ones(990), np.full(10, 6.)]
b = np.r_[np.ones(999), 12.]
def cola_superior(v, fraccion=0.01):
    k = max(1, int(np.ceil(fraccion * len(v))))
```

```
    return np.partition(v, len(v)-k)[-k:].mean()
for v in [a, b]:
    print(round(v.mean(), 3), v.max(), cola_superior(v))
# Primera: 1.05, 6.0, 6.0; segunda: 1.011, 12.0, 2.1
```

Ningún resumen domina en todas las tareas. Un defecto pequeño pero relevante puede verse perjudicado al promediar una cola demasiado grande. Elegir la fracción requiere datos y un criterio de validación; el 1 % no procede de una ley física.

5.5. Umbral y matriz de decisiones

Se genera alarma si $s(x) \geq t$. Definimos como clase positiva el defecto simulado. Así, TP es defecto con alarma, FN es defecto sin alarma, FP es correcta con alarma y TN es correcta sin alarma.

Referencia	Sin alarma	Con alarma
Correcta	TN	FP
Defecto	FN	TP

Para denominadores positivos, las métricas principales son

$$\text{FPR} = \frac{FP}{FP + TN}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \tag{5.2}$$

$$\text{Precisión} = \frac{TP}{TP + FP}, \qquad \text{FNR} = \frac{FN}{FN + TP} = 1 - \text{Recall}. \tag{5.3}$$

FPR mira todas las correctas; FNR, todos los defectos; precisión, todas las alarmas. Si no hay alarmas, la precisión tiene denominador cero. El código utiliza 0 como convención para ese caso: no es una estimación observada de la fracción de alarmas correctas.

5.5.1. Elegir el denominador según la pregunta

El recall responde «¿qué fracción de defectos detectamos?». La FPR responde «¿qué fracción de correctas enviamos a alarma?». La precisión responde «¿qué fracción de alarmas son defectos?». El porcentaje de falsas alarmas entre todas las alarmas es $FP/(TP + FP) = 1 -$ precisión, cuando existe ese denominador; no es la FPR.

El acierto global $(TP + TN)/(TP + TN + FP + FN)$ puede ocultar el problema si los defectos son raros. Predecir siempre correcta logra el 99 % de acierto si solo el 1 % de piezas tiene defecto, pero su recall de defectos es cero. Para una inspección necesitamos mirar las omisiones y el trabajo de revisión, no solo el acierto.

Ejemplo: Cuatro recuentos y tres preguntas

Entre 100 piezas hay 20 defectuosas. Se detectan 15 y se producen ocho alarmas sobre correctas. Entonces $TP = 15$, $FN = 5$, $FP = 8$ y $TN = 72$. El recall es $15/20 = 0{,}75$; la FPR, $8/80 = 0{,}10$; y la precisión, $15/23 \approx 0{,}652$. La fracción de alarmas falsas es $8/23 \approx 0{,}348$. Cada razón utiliza un grupo distinto.

5.5.2. Desglosar por tipo y condición

En prueba, el notebook informa detectados y omitidos para rayas y poros por separado. Una media puede ocultar que las rayas se detectan bien y los poros débiles se omiten. Cada desglose necesita su recuento: un porcentaje sobre 30 ejemplos no tiene la misma resolución que uno sobre miles de piezas independientes.

En un conjunto real también podría interesar informar resultados por material, máquina o sesión. Esas categorías deben corresponder al uso y contar con suficientes ejemplos. Buscar después el subgrupo con mejor cifra y destacarlo como resultado global puede inducir una conclusión engañosa.

5.6. Resultado: el umbral ordena un compromiso

Para puntuaciones fijas, subir t solo puede retirar alarmas, no añadirlas. TP y FP no aumentan; FN y TN no disminuyen. En una muestra finita aparecen escalones porque la decisión cambia al atravesar puntuaciones observadas. Esto no obliga a que la precisión aumente monótonamente: al retirar alarmas pueden perderse proporciones diferentes de TP y FP.

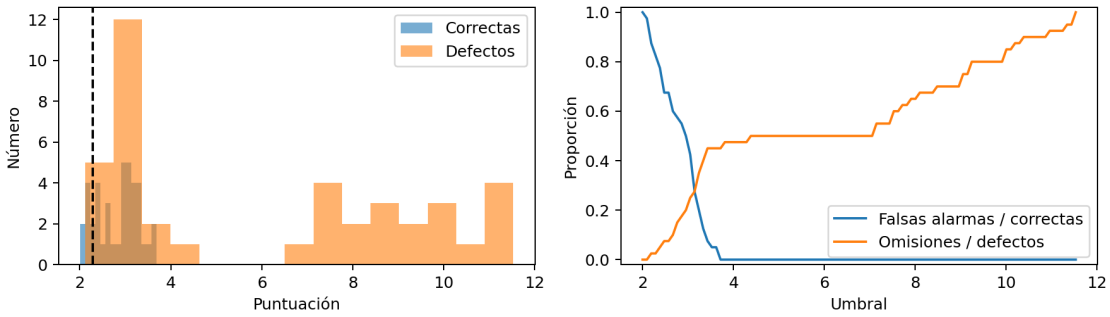


Figura 5.1: Puntuaciones en validación y tasas frente al umbral. Los poros débiles se solapan con variaciones normales, por lo que cambiar el umbral no resuelve toda la separación.

Podemos comparar penalizaciones hipotéticas como $L_1 = FP + 5FN$ y $L_2 = 5FP + FN$. El primero prioriza no omitir; el segundo penaliza más las falsas alarmas. Minimizar una de ellas en una validación equilibrada no determina una política industrial óptima. Faltan costes reales, frecuencia de defectos, restricciones operativas y evaluación pertinente.

En el ejemplo, priorizar omisiones puede generar demasiadas falsas alarmas. No debe ocultarse ese resultado. Si ningún umbral satisface las necesidades, hay que mejorar captura, representación, datos o método; el umbral no crea información discriminativa que el modelo no posee.

5.6.1. Por qué las curvas cambian a escalones

La regla de alarma usa $s \geq t$. Si ninguna puntuación está entre dos umbrales, ambos producen las mismas decisiones. Cuando el umbral sobrepasa una puntuación observada, puede retirar una o varias alarmas, especialmente si hay empates. No hay una mejora continua garantizada por moverlo una cantidad muy pequeña.

El notebook prueba 101 candidatos equiespaciados entre un valor ligeramente inferior al mínimo y otro ligeramente superior al máximo de validación. Incluye así decisiones de alarma para todos y para ninguno, pero la rejilla puede saltar intervalos que producirían otras decisiones. El mejor candidato lo es dentro de esa búsqueda concreta, no un óptimo matemático sobre cualquier regla posible.

Ejemplo: Subir el umbral puede reducir la precisión

Hay dos alarmas con puntuaciones 3 y 5. La primera corresponde a un defecto y la segunda a una pieza correcta. Con $t = 3$ hay un TP y un FP: precisión 1/2. Con $t = 4$ solo queda la falsa alarma: precisión 0. Retirar alarmas no implica retirar preferentemente las falsas.

5.6.2. Penalizaciones y frecuencia de la muestra

Para tasas $f = \text{FPR}$ y $r = \text{Recall}$, costes hipotéticos c_{FP}, c_{FN} y prevalencia π, la penalización esperada por pieza, si esas tasas se mantienen, sería

$$C = c_{FP}(1 - \pi)f + c_{FN}\pi(1 - r).$$

La fórmula cuenta cada tipo de error multiplicando su probabilidad por la penalización elegida. No incluye por sí sola coste de captura, revisión humana, demoras o consecuencias particulares de cada defecto.

Minimizar $FP + 5FN$ en una validación con mitad defectos pondera los errores según ese reparto. No equivale automáticamente a minimizar coste en una línea con prevalencia del 1 %. Se podrían estudiar ponderaciones coherentes con el contexto, pero habría que justificarlas y comprobar que las tasas se trasladan a él.

5.6.3. Qué decisión fija exactamente la práctica

Se comparan dos criterios: $FP+5FN$ y $5FP+FN$. En empate se conserva el primer candidato, que es el umbral menor. Después se fija para prueba la opción que prioriza no omitir. No se vuelve a escoger entre ambas mirando los resultados de prueba.

Un coste bajo en validación puede conseguirse activando muchas alarmas. La tabla de recuentos y tasas permite ver ese efecto. Si no hay ningún punto aceptable, conviene revisar representación y captura o recoger datos más adecuados; no ocultar el compromiso detrás de una puntuación única.

5.7. Resultado: frecuencia de defectos y precisión

Sean $\pi = P(\text{defecto})$, $r = P(\text{alarma} \mid \text{defecto})$ y $f = P(\text{alarma} \mid \text{correcta})$. Aplicando la regla del producto y sumando ambas causas de alarma:

$$P(\text{defecto} \mid \text{alarma}) = \frac{r\pi}{r\pi + f(1 - \pi)}, \tag{5.4}$$

cuando el denominador es positivo. El resultado muestra cómo una misma pareja de tasas condicionales puede dar distinta precisión si cambia la frecuencia de defectos. Si además cambian las condiciones, las tasas r y f también pueden cambiar.

Ejemplo: Defectos raros

En 10 000 piezas hipotéticas con 1 % de defectos, hay 100 defectuosas y 9900 correctas. Con recall del 90 % aparecen 90 TP. Con FPR del 5 % aparecen 495 FP. De 585 alarmas, solo 90 corresponden a defectos: la precisión es $90/585 \approx 15{,}4\,\%$. No son tasas estimadas para una fábrica; es una ilustración aritmética.

5.7.1. Del porcentaje a la carga de revisión

En el ejemplo de 10 000 piezas, la fracción total de alarmas es $585/10\,000 = 5{,}85\,\%$. Si todas requieren revisión, eso supone 585 revisiones, de las cuales 495 corresponden a piezas correctas. Un recall del 90 % no indica por sí solo si la revisión resulta asumible ni si las diez omisiones son aceptables.

Si la prevalencia bajase al 0,1 % y se mantuvieran recall del 90 % y FPR del 5 %, por cada 100 000 piezas esperaríamos 90 verdaderas alarmas y 4995 falsas. La precisión sería $90/5085 \approx 1{,}77\,\%$. Son expectativas aritméticas bajo tasas fijas, no predicciones verificadas para un proceso real.

5.7.2. Qué debe permanecer estable para trasladar tasas

La fórmula de prevalencia supone que las tasas condicionales utilizadas corresponden a las condiciones de destino. Si cambian la textura de las piezas correctas, la intensidad de los defectos o la alineación, también pueden cambiar FPR y recall. Corregir únicamente el porcentaje de defectos no resuelve ese cambio.

El conjunto de prueba del notebook tiene prevalencia artificial del 50 %. Su precisión describe ese conjunto. Sirve para estudiar el procedimiento y sus errores, pero no es una estimación directa de cuántas alarmas serían verdaderas en una fábrica. Debe acompañarse de la composición de la muestra.

5.7.3. Comprobación en Python: prevalencia y precisión

```
def precision_esperada(prevalencia, recall, fpr):
    verdaderas = prevalencia * recall
    falsas = (1 - prevalencia) * fpr
    return verdaderas / (verdaderas + falsas)
for prevalencia in [0.5, 0.01, 0.001]:
    print(prevalencia,
          round(precision_esperada(prevalencia, 0.9, 0.05), 4))
# Aproximadamente: 0.9474, 0.1538, 0.0177
```

Esta función presupone denominador positivo. Si no hubiera ninguna alarma posible, la precisión no estaría definida. El ejemplo cambia solo prevalencia para aislar su efecto; en una comparación real habría que comprobar también las tasas.

5.8. Mapas, reconstrucción y características profundas

Un mapa de anomalía indica alejamiento según una representación. No es automáticamente una máscara validada de defecto. Para evaluar localización necesitamos referencias espaciales y un protocolo, además de las etiquetas por imagen.

Un autoencoder aprende a codificar y reconstruir entradas. El error de reconstrucción puede revelar algunas anomalías, pero también responder a variaciones admisibles; una red puede reconstruir una anomalía y reducir su señal. Métodos basados en características preentrenadas comparan regiones con distribuciones o memorias de normalidad. PaDiM y PatchCore son ejemplos para ampliar el estudio, no métodos ejecutados en la práctica principal.

5.8.1. Alarma por imagen y localización son evaluaciones distintas

La práctica decide si una imagen genera alarma. Aunque el generador conserva una máscara de cada marca, no se selecciona ni evalúa un umbral espacial para convertir el mapa z en máscara de defecto. El contorno cian de la figura de errores es la referencia sintética; el mapa coloreado es una desviación.

Una imagen puede producir una alarma por una zona diferente de su defecto. Contaría como acierto por imagen según la regla del ejercicio, aunque no localizase la marca. Para afirmar localización se necesita un criterio espacial separado y referencias adecuadas, por ejemplo máscaras evaluadas con métricas pertinentes a tamaño y contorno. El umbral de alarma por imagen no es automáticamente un umbral de segmentación.

5.8.2. Autoencoders: aprender a reconstruir

Un autoencoder utiliza un codificador para representar la entrada y un decodificador para reconstruirla. Con imágenes correctas puede aprender patrones frecuentes. Una puntuación ilustrativa es la media de $|x(i,j) - \widehat{x}(i,j)|$ o de su cuadrado, donde $\widehat{x}$ es la reconstrucción. El tipo de error, la agregación y la preparación son decisiones del método.

La intuición es que una anomalía difícil de reconstruir dejaría un residuo. Pero el modelo puede reconstruir también anomalías o producir error sobre texturas admisibles. Un residuo alto no identifica automáticamente causa física. Reducir dimensionalidad no garantiza que solo se conserve lo normal.

5.8.3. Características y memorias de normalidad

Otra estrategia transforma regiones de imágenes en vectores de características. Podemos comparar un vector nuevo con una referencia estadística o con vecinos de una memoria de ejemplos normales. Una distancia ilustrativa a una memoria $\mathcal{M}$ es $d(v) = \min_{m \in \mathcal{M}} \|v - m\|$: mide proximidad al ejemplo guardado más cercano en esa representación.

Una referencia que utiliza varias componentes puede necesitar considerar que varían conjuntamente, no solo su dispersión individual. Esto introduce estimación y supuestos adicionales. PaDiM y PatchCore son familias de métodos que amplían esta idea con decisiones concretas; no deben identificarse con la fórmula escalar por píxel ejecutada aquí.

Comparar características puede aportar contexto, pero hay que comprobar qué detalles preserva la representación. Si un poro ocupa una región muy pequeña respecto a la escala de sus características, su señal puede diluirse. Cambiar de familia de modelo no elimina la necesidad de validación y separación de conjuntos.

5.9. Diagnóstico de captura e incertidumbre

Desplazar piezas correctas sin modificar la ROI genera desviaciones importantes. Antes de reentrenar automáticamente, hay que revisar si la captura cumple las condiciones de uso. Incorporar datos sin comprobar su calidad podría enseñar al modelo a aceptar cambios no deseados.

Una tasa observada con pocos casos tiene incertidumbre. Si medimos recall con 10 defectos, cada caso cambia el resultado diez puntos porcentuales. Además, varias capturas de una pieza no aportan necesariamente diez observaciones independientes. Informar tamaño y procedencia de muestra es parte de interpretar una métrica.

Una intuición sobre cero fallos

Si cada caso fuera independiente y tuviera la misma probabilidad de fallo q, observar cero fallos en n casos tendría probabilidad $(1-q)^n$. Para n suficientemente grande, el valor de q que hace esa probabilidad cercana a 0,05 es aproximadamente $3/n$. Esta «regla de tres» es una orientación estadística bajo supuestos concretos, no una garantía de seguridad ni una certificación. Si los casos son dependientes o no representativos, la interpretación deja de ser adecuada.

5.9.1. Desplazamiento, deriva y respuesta del sistema

El diagnóstico mueve cuatro píxeles a la derecha las imágenes correctas de prueba, rellena con el fondo y mantiene la ROI. Son transformaciones de las mismas capturas, no 60 piezas nuevas. Permiten observar sensibilidad al incumplimiento de alineación, pero no estiman por sí solas el rendimiento frente a cualquier movimiento.

Una **deriva** es un cambio de las condiciones o distribuciones a lo largo del uso. Puede aparecer por iluminación, máquina, material, preparación o mezcla de defectos. Antes de añadir los nuevos ejemplos a la referencia conviene comprobar la causa y si el cambio es admisible. Si actualizamos el modelo, hay que versionar referencia, puntuación y umbral y evaluar el nuevo procedimiento.

5.9.2. Cero errores observados y límites de la muestra

Bajo el modelo de casos independientes con la misma probabilidad de fallo q, el valor que satisface $(1-q)^n = 0{,}05$ es

$$q_{\text{sup}} = 1 - 0{,}05^{1/n}.$$

Se interpreta como un límite superior unilateral del 95 % para q cuando se observan cero fallos, dentro de ese modelo binomial. Con $n = 60$, vale aproximadamente 0,0487, próximo a $3/60 = 0{,}05$. No permite concluir que $q = 0$, ni que la probabilidad de un parámetro fijo sea del 95 % en una interpretación frecuentista.

Si queremos aplicar este razonamiento a FPR, los casos son piezas correctas; para omisiones, los casos son defectos. No se usa como denominador la suma de píxeles de sus imágenes. Dependencias entre capturas, selección de ejemplos fáciles o condiciones ausentes limitan la interpretación.

5.10. Guía de trabajo con las seis actividades

1. Definir normalidad.
Identifica qué variaciones admite el generador y qué regiones inspecciona la ROI. Distingue número de imágenes y número de ejemplos de cada tipo.

2. Aprender referencia.
Comprueba que solo intervienen las 60 correctas de entrenamiento. Describe los ejes de media y desviación, el suelo y la agregación del 1 % superior.

3. Elegir umbral.
Compara recuentos y penalizaciones en validación. Explica qué denominador corresponde a cada tasa. Registra el criterio y el umbral antes de abrir prueba.

4. Evaluar.
Mantén fijos preparación, referencia, puntuación y umbral. Lee la matriz completa y el desglose de rayas y poros; no cambies la regla por lo que observes aquí.

5. Examinar errores.
Contrasta imagen, referencia sintética y mapa con escala común. Si falta una categoría de error, decláralo: el ejemplo de sustitución no se convierte por ello en un error.

6. Diagnosticar.
Separa desplazamiento y ejemplo aritmético de prevalencia. El primero transforma capturas ya evaluadas; el segundo supone tasas hipotéticas ajenas al modelo entrenado.

Un informe útil conserva procedencia de datos, alcance de ROI, preparación, versión de referencia, porcentaje de cola, criterio de selección, umbral, recuentos y tasas por tipo. Las figuras ayudan a interpretar, pero no sustituyen esa información ni una evaluación cuantitativa apropiada.

5.11. Errores frecuentes y aplicación a fabricación

Llamar defecto a toda desviación.
Comprueba si la diferencia incumple la especificación o refleja una variación admisible ausente en la referencia.

Interpretar el mapa como certeza.
Un color intenso muestra una puntuación en una escala elegida. No es confirmación de defecto ni su probabilidad.

Reducir falsas alarmas sin mirar omisiones.
Subir el umbral puede retirar también defectos. Informa ambas consecuencias con sus recuentos.

Usar precisión de una muestra equilibrada como cifra de planta.
Declara prevalencia y comprueba si las tasas se mantienen en el destino. El porcentaje de correctas suele alterar mucho la carga de revisión.

Actualizar automáticamente con capturas desplazadas.
Revisa primero si la captura cumple el protocolo. Aprender ese cambio sin comprobarlo podría ocultar un problema del sistema.

En una inspección de acabado de fabricación aditiva, la orientación y textura pueden producir variación admisible mayor que la señal de una marca pequeña. Conviene definir superficie, escala y defecto observable, recoger piezas revisadas y comprobar casos difíciles. Una referencia demasiado amplia puede ocultar defectos; una demasiado estrecha puede generar revisión excesiva.

Las propiedades internas o mecánicas no se deducen automáticamente de una apariencia superficial. La alarma puede servir como apoyo a revisión según un procedimiento validado, pero el criterio de conformidad debe corresponder a la propiedad que realmente se evalúa.

5.12. Ejercicios con soluciones razonadas

1. Preparar y puntuar una posición

La mediana de una captura es 0,60. Un píxel vale 0,68. La referencia preparada tiene media 0,02 y desviación 0,01. Con suelo 0,025, calcula su desviación estandarizada absoluta.

Solución. La intensidad preparada es 0,08 y la diferencia respecto a la referencia es 0,06. El denominador efectivo es 0,025, así que $z = 2{,}4$. No se divide directamente la intensidad original por la desviación ni se interpreta 2,4 como probabilidad.

2. Agregar una cola pequeña

Los valores de una ROI son $1, 1, 1, 2, 2, 2, 3, 3, 5, 10$. Si seleccionamos el 20 % superior, ¿qué puntuación obtenemos? ¿Cuál es la media global?

Solución. Elegimos los dos mayores, 5 y 10: la puntuación es 7,5. La suma de todos es 30 y la media global, 3. Son resúmenes distintos de los mismos datos; aquí usamos 20 % para facilitar el cálculo, no el 1 % del notebook.

3. Leer las alarmas

De 200 piezas, 20 son defectuosas. Hay 16 defectos con alarma y 18 correctas con alarma. Calcula recall, FPR y precisión.

Solución. $TP = 16$, $FN = 4$, $FP = 18$ y $TN = 162$. El recall es $16/20 = 0{,}8$, la FPR $18/180 = 0{,}1$ y la precisión $16/34 \approx 0{,}471$. La proporción de alarmas falsas es 18/34, no 18/180.

4. Comparar penalizaciones

Un umbral A da $FP = 10, FN = 2$ y otro B da $FP = 2, FN = 6$. ¿Cuál gana con $FP + 5FN$? ¿Y con $5FP + FN$?

Solución. Con la primera penalización, A obtiene 20 y B 32: gana A. Con la segunda, A obtiene 52 y B 16: gana B. La preferencia depende de qué errores penalizamos y de la composición de la muestra; no demuestra que exista una política industrial óptima universal.

5. Prevalencia distinta

En 10 000 piezas con prevalencia del 2 %, suponemos recall del 80 % y FPR del 1 %. Calcula recuentos esperados de alarmas y su precisión.

Solución. Hay 200 defectos y 9800 correctas. Esperamos 160 verdaderas alarmas y 98 falsas: 258 en total. La precisión esperada es $160/258 \approx 62{,}0\,\%$. El cálculo supone que las tasas son pertinentes y se mantienen en esas condiciones; no las estima a partir de datos.

6. Cero omisiones observadas

No se omite ninguno de 30 defectos independientes de igual distribución. Bajo el modelo binomial indicado, calcula aproximadamente el límite superior unilateral del 95 % para la probabilidad de omisión. ¿Puedes afirmar que es cero?

Solución. El límite es $1 - 0{,}05^{1/30} \approx 0{,}095$, cercano a la regla $3/30 = 0{,}10$. No podemos afirmar probabilidad cero. Si las imágenes fueran capturas dependientes de unas pocas piezas, no sería adecuado tratarlas sin más como 30 ensayos independientes.

Conexión con las prácticas

La práctica aprende referencia solo con normales; elige umbral con validación; informa prueba sin reajustar; examina errores y desplazamiento. Las tasas se interpretan por tipo de defecto y con sus denominadores. Material de la sesión en el repositorio.

Capítulo 6

Tema 5. Modelos multimodales y generativos

Objetivos y recorrido

Este capítulo une dos preguntas: cómo relacionar imágenes con lenguaje y cómo explorar conceptos visuales mediante generación. Al terminar deberías poder interpretar una búsqueda por similitud, explicar por qué sus puntuaciones no verifican requisitos, distinguir el ruido directo de un generador aprendido y revisar un concepto de producto con criterios explícitos.

La ruta principal utiliza vectores reales de CLIP guardados y dos imágenes generadas previamente. No entrena CLIP ni ejecuta generación de imágenes. La ampliación opcional calcula nuevos vectores con el modelo preentrenado. Las explicaciones de difusión proporcionan una base conceptual y matemática para interpretar el experimento de ruido.

6.1. Representar imagen y texto en un espacio común

Un modelo multimodal relaciona distintos tipos de datos. En CLIP, un codificador visual transforma una imagen en un vector y un codificador textual transforma una frase en otro de la misma dimensión. Su preentrenamiento contrastivo favorece la asociación de pares de imagen y texto frente a otras combinaciones.

El modelo del curso es `openai/clip-vit-base-patch32`. Sus vectores proyectados tienen 512 componentes. Se utilizan normalizados. Los nombres de archivo y etiquetas del catálogo no se enviaron al modelo: los vectores se obtuvieron de sus píxeles y de las frases registradas.



Figura 6.1: Catálogo de ocho imágenes. Los seis dibujos son procedimentales y los soportes A y B fueron generados previamente. La colección no contiene una taza verde ni dos tazas azules juntas.

6.1.1. Qué significa multimodal

Una **modalidad** es un tipo de información, como imagen, texto o audio. Un sistema multimodal combina o relaciona varias. Esto puede significar comparar vectores de imagen y texto, responder por escrito sobre una imagen o producir una imagen condicionada por una descripción. Son funciones distintas y no se deducen unas de otras.

CLIP, tal como lo usamos aquí, devuelve representaciones y similitudes. No redacta una explicación ni genera un dibujo. Un modelo conversacional visual puede producir lenguaje, y un generador puede producir píxeles; cada sistema necesita su propio procedimiento y criterio de evaluación.

6.1.2. Qué es un embedding

Un **embedding**, o representación vectorial, traduce una entrada a una lista de números aprendida para una tarea. En vez de comparar directamente los píxeles con las letras de una frase, comparamos sus vectores. Que ambos tengan 512 componentes permite efectuar un producto escalar, pero no basta: el entrenamiento debe organizar sus relaciones de manera útil.

Las componentes no tienen por qué corresponder por separado a «azul», «metal» o «respaldo». La información puede estar distribuida entre muchas coordenadas. Dos codificadores con salidas de la misma longitud pero entrenados sin relación no producen automáticamente un espacio común interpretable.

6.1.3. Entrenamiento contrastivo: acercar pares y distinguir alternativas

Imaginemos un lote de B imágenes y B textos emparejados. Calculamos una tabla de similitudes S_{ij} entre cada imagen i y cada texto j. Los pares previstos están en la diagonal. Una pérdida ilustrativa desde imagen a texto es

$$L_i = -\log \frac{\exp(aS_{ii})}{\sum_{j=1}^{B} \exp(aS_{ij})}, \qquad a > 0.$$

La expresión tiene la misma estructura de entropía cruzada del capítulo de clasificación: favorece el texto emparejado frente a los otros del lote. También puede formularse la comparación inversa, desde cada texto hacia las imágenes. Este esquema explica la intuición contrastiva; no reproduce todos los detalles de una ejecución de preentrenamiento.

Los textos de entrenamiento pueden describir rasgos visibles con mayor o menor precisión. Aprender asociaciones no equivale a disponer de anotaciones completas de cada objeto, relación espacial o condición de fabricación. El par imagen-texto tampoco elimina automáticamente sesgos o errores presentes en los datos.

Ejemplo: Mismo espacio, dos entradas diferentes

Un dibujo de una lámpara y la frase «a drawing of a yellow desk lamp» se procesan por rutas distintas. La imagen aporta píxeles; el texto, una secuencia codificada de unidades de lenguaje. Las salidas proyectadas permiten compararlas. La semejanza se produce entre sus representaciones, no entre el identificador `lampara` del archivo y las letras de la consulta.

6.2. Similitud y recuperación

Para vectores no nulos u y v, el coseno es $s = (u \cdot v)/(\|u\|\|v\|)$. Como los vectores están normalizados, basta el producto escalar. Ordenar imágenes por esa puntuación produce un ranking de recuperación.

La puntuación depende del modelo, de la preparación de las imágenes y de la frase. Un coseno de 0,34 no significa un 34 % de probabilidad de que el objeto satisfaga todos los requisitos. Tampoco existe un umbral universal que certifique similitud funcional o fabricabilidad.

Ejemplo: Un ranking siempre tiene un primero

Si consultamos «taza verde» y el catálogo solo contiene una azul y una roja, una de ellas puede quedar primera. Esa posición solo dice que superó a las otras candidatas según la puntuación. No demuestra que exista una respuesta correcta. Diseñar un sistema con abstención requeriría un criterio específico y evaluado de ausencia de respuesta.

6.2.1. Una tabla de comparaciones, dos búsquedas posibles

Si U contiene N vectores de imagen como filas y V contiene M vectores de texto normalizados, calculamos

$$S = UV^\top, \qquad S \in \mathbb{R}^{N \times M}.$$

Una columna ordena imágenes para una consulta textual. Una fila compara descripciones para una imagen. La traspuesta $V^\top$ cambia la organización de los ejes para que la suma combine las mismas 512 componentes.

En el catálogo del curso hay ocho imágenes y trece consultas guardadas. La tabla tiene forma $(8, 13)$. Un cambio en las opciones que mostramos no requiere recalcular esos productos si sus vectores ya existen, aunque sí cambie la interpretación de la búsqueda.

6.2.2. Coseno y orden relativo

Para dos vectores unitarios $u = (1, 0)$ y $v = (0{,}8, 0{,}6)$, el coseno vale 0,8. Para $w = (0, 1)$, la similitud con u es 0. Son cálculos geométricos, no porcentajes de veracidad. En CLIP la geometría procede de una representación aprendida y se interpreta a la luz de ejemplos y de la tarea.

El primer resultado solo es el de mayor puntuación dentro del conjunto consultado. Si eliminamos esa imagen del catálogo, otra pasa a ser primera sin que haya mejorado su correspondencia con la petición. Si añadimos candidatas, pueden cambiar los puestos aunque las similitudes de las imágenes anteriores se mantengan.

6.2.3. Qué información pedir a un buscador

En una búsqueda de inspiración puede ser útil recuperar conceptos relacionados y revisarlos. En una búsqueda que exige una propiedad literal, como «sin asa», interesa comprobar si los resultados satisfacen esa condición. La relevancia depende del criterio: parecido general, categoría, color, cantidad o geometría pueden dar juicios distintos.

Para una consulta con varios resultados relevantes puede estudiarse qué fracción de los primeros k lo es. Para una consulta sin respuesta válida conviene evaluar también la decisión de abstenerse. Añadir «ninguno» como texto candidato no garantiza resolver esa segunda tarea: sigue siendo una comparación de representaciones que necesita validación.

6.2.4. Comprobación en Python: dimensiones y consulta real

Ejecuta esta celda después de cargar los recursos en el notebook principal. No consulta una API ni calcula nuevos embeddings.

```
print(vectores_imagen.shape)             # (8, 512)
print(vectores_texto.shape)              # (13, 512)
S = vectores_imagen @ vectores_texto.T
print(S.shape)                           # (8, 13)
j = consultas.index("a drawing of a blue mug")
orden = np.argsort(-S[:, j])
for i in orden[:3]:
    print(nombres[i], round(float(S[i, j]), 3))
```

Compara el ranking con la galería y justifica qué condición satisface cada imagen. No conviertas la diferencia entre el primer y segundo coseno en una probabilidad de acierto sin un procedimiento de calibración y evaluación.

6.3. Clasificar sin ajuste y el efecto de las candidatas

Podemos describir clases en texto y comparar una imagen con cada descripción. Esto suele llamarse *zero-shot* para la tarea: no entrenamos una cabeza nueva con sus etiquetas, pero sí usamos un modelo preentrenado.

CLIP convierte similitudes en logits mediante un factor de escala aprendido. Si el factor es $a > 0$, $\ell_k = as_k$. Una softmax sobre las candidatas produce pesos relativos. El factor cambia cuán concentrados quedan esos pesos, aunque no cambie qué candidata gana.

Retirar una candidata no cambia el vector de la imagen ni el coseno de las otras frases. Sí cambia el denominador de softmax. Por eso podemos obtener una distribución que suma uno entre opciones todas incorrectas. La validez semántica de las opciones debe revisarse por separado.

6.3.1. Zero-shot no significa ausencia de aprendizaje previo

La etiqueta *zero-shot* describe aquí que aplicamos las descripciones de clases sin ajustar una cabeza nueva con ejemplos etiquetados del curso. Los codificadores sí contienen información aprendida durante preentrenamiento. Elegir frases mirando resultados de un pequeño catálogo también adapta nuestras decisiones a esos ejemplos, aunque no cambien los pesos del modelo.

Si queremos comparar varias redacciones de clases de forma rigurosa, podemos desarrollarlas con ejemplos separados y evaluar después las redacciones fijadas. Probar muchas frases y mostrar únicamente las que funcionan mejor no estima la capacidad general de seguir instrucciones.

6.3.2. Un ejemplo numérico del efecto de las opciones

Supongamos similitudes ilustrativas $(0{,}30, 0{,}25, 0{,}20)$ y factor de escala $a = 10$. Los logits son $(3, 2{,}5, 2)$ y softmax produce aproximadamente $(0{,}506, 0{,}307, 0{,}186)$. Si retiramos la primera opción, las dos restantes reciben $(0{,}622, 0{,}378)$.

La segunda candidata pasa de un peso de 0,307 a 0,622, aunque su coseno sigue en 0,25 y el vector de la imagen no cambia. La distribución resume competencia entre las opciones ofrecidas. No demuestra que retirar una descripción haya hecho más verdadera otra.

6.3.3. Escala y temperatura

Escribir logits como aS con $a > 0$ equivale a escribir S/T con temperatura $T = 1/a$. Una escala mayor concentra softmax en las puntuaciones más altas; no cambia su orden. En el recurso del curso se guarda el factor de escala del modelo. El valor 10 del ejemplo anterior es didáctico y no pretende sustituir ese factor guardado.

Incluso con dos opciones incorrectas, sus pesos suman uno. El valor mayor puede acercarse mucho a uno si la escala es alta. Esto explica por qué el máximo de softmax no es una garantía de que la lista de clases cubra la imagen ni de que se cumplan todos los atributos de una frase.

6.3.4. Comprobación en Python: las opciones cambian el denominador

La siguiente celda utiliza números ilustrativos y es independiente de los recursos.

```
def pesos_relativos(cosenos, escala):
    logits = escala * np.asarray(cosenos, dtype=float)
    e = np.exp(logits - logits.max())
    return e / e.sum()
s = np.array([0.30, 0.25, 0.20])
print(np.round(pesos_relativos(s, 10), 3))       # [0.506 0.307 0.186]
print(np.round(pesos_relativos(s[1:], 10), 3))   # [0.622 0.378]
assert np.isclose(pesos_relativos(s, 10).sum(), 1)
```

La normalización numérica resta el máximo para evitar exponenciales innecesariamente grandes. Esa estabilidad de cálculo no cambia la validez semántica de las descripciones.

6.4. Idioma, negación y cantidad

La tarjeta del CLIP utilizado limita su uso al inglés, al no haberlo entrenado y evaluado deliberadamente para otras lenguas de la misma manera. Una recuperación correcta con una frase española en el pequeño catálogo no demuestra capacidad multilingüe validada.

Una representación puede captar bien «taza» y «azul» sin cumplir «sin asa» o «dos». El notebook permite observarlo. La negación y el conteo no se verifican simplemente por encontrar objetos relacionados con los sustantivos. Cuando una propiedad sea decisiva, se necesita una comprobación adecuada y ejemplos que pongan a prueba esa propiedad.

Un modelo conversacional con visión puede redactar respuestas, pero su texto puede mezclar observaciones e inferencias. Pedir justificaciones visibles ayuda a analizarlo, sin garantizar que todo lo afirmado esté sustentado por los píxeles.

6.4.1. Descomponer una instrucción en propiedades comprobables

«Dos tazas azules sin asa» combina categoría, número, color y ausencia de una parte. Que una imagen se relacione con «taza azul» no demuestra que cumpla la conjunción completa. Podemos registrar cada propiedad por separado y marcar cuál falla, en vez de valorar solo parecido global.

El banco incluye una consulta de taza verde, pero el catálogo no contiene una. Incluye «two blue mugs», pero no una escena con dos tazas azules. Son casos útiles para observar que un ranking produce respuestas aunque falte una coincidencia literal. No constituyen por sí solos una medición amplia de conteo o negación.

Ejemplo: Una respuesta aparentemente razonable pero insuficiente

Para «a blue mug without a handle», una taza azul con asa comparte categoría y color, pero incumple una condición esencial. Si queda primera, podemos decir que el ranking priorizó una imagen relacionada; no que resolvió correctamente la petición. El criterio de corrección debe fijarse antes de mirar el resultado.

6.4.2. Lo visible, lo inferido y lo no verificable

Un modelo conversacional visual puede ayudar a describir una escena o proponer hipótesis, pero conviene separar tres tipos de afirmación. «Se ven dos apoyos frontales» se puede contrastar con la imagen. «El cuerpo es rígido» requiere información adicional. «Soporta una carga concreta» necesita especificación, geometría y evaluación apropiadas.

La redacción fluida de una respuesta no es evidencia de una observación correcta. Pedir una tabla con propiedad, evidencia visible y comprobación pendiente facilita revisar sus afirmaciones. Si un requisito es geométrico o numérico, una herramienta de medida o un modelo CAD puede aportar la evidencia que una descripción visual no contiene.

6.4.3. Diseñar una prueba más informativa

Para estudiar negación podemos construir pares de imágenes donde solo cambie la presencia de una parte. Para cantidad, escenas con uno, dos o varios objetos bajo condiciones controladas. Para lenguaje, consultas equivalentes revisadas por una persona competente y un conjunto suficientemente diverso. El catálogo pequeño es una demostración que ayuda a formular esas pruebas.

Si usamos los fallos para reformular las frases, esas imágenes pasan a formar parte del desarrollo. Harían falta ejemplos reservados para evaluar la solución resultante. El mismo principio de separación que usamos al entrenar un clasificador se aplica a decisiones de redacción y de uso de modelos preentrenados.

6.5. Vectores guardados e inferencia en vivo

La ruta principal incluye vectores reales ya calculados, con identificador de modelo, revisión, versiones y huellas de los archivos. Esto permite comparar consultas del banco sin descargar los pesos. Los vectores guardados no permiten codificar una frase nueva que no esté incluida.

La ampliación opcional carga el modelo y calcula características de imágenes y textos nuevos. Usa la misma revisión y procesador para contrastar los resultados. Diferencias numéricas pequeñas pueden proceder de operaciones y entorno. La procedencia registrada ayuda a distinguir esos efectos de utilizar un modelo o una preparación diferente.

6.5.1. Qué conserva la procedencia de los recursos

El archivo de procedencia registra identificador del modelo, revisión concreta, procesador, versiones de herramientas, dimensión y huellas SHA-256 de los recursos. Una huella permite comprobar que los bytes utilizados coinciden con los registrados. No demuestra por sí sola calidad de los datos, corrección de las etiquetas ni validez del modelo para una aplicación.

La ruta principal comprueba las huellas antes de comparar vectores. Así evita mezclar una imagen modificada con una representación calculada anteriormente para otro contenido. Si

editamos un archivo, hay que recalcular de forma coherente sus recursos asociados; no basta con cambiar su nombre.

6.5.2. Qué hace y qué no hace la caché

Guardar embeddings evita repetir inferencia para entradas conocidas. Podemos reordenar candidatos, calcular productos escalares o cambiar la consulta entre las trece disponibles. No podemos obtener el vector de cualquier frase nueva a partir de su parecido textual con una frase guardada. Para eso necesitamos ejecutar el codificador correspondiente o disponer de ese vector previamente calculado.

Tampoco es posible reconstruir con garantía todos los píxeles originales a partir del embedding de CLIP. La representación se utiliza aquí para comparación, no como una codificación de archivo reversible. Una gran semejanza entre vectores no asegura igualdad de geometría o de detalle fino.

6.5.3. La ampliación opcional

El notebook 05B carga los pesos y el procesador de la revisión registrada, recalcula las imágenes de la galería y permite codificar texto nuevo. Sigue siendo inferencia con pesos preentrenados: no ajusta una cabeza, no entrena CLIP y no genera imágenes.

La ruta principal puede funcionar con los recursos locales; al abrirla desde Colab necesita obtener los archivos del repositorio si no están presentes. La ruta opcional necesita además descargar pesos y sus dependencias en un entorno apropiado. Para el seguimiento ordinario del tema bastan los vectores guardados. Material de la sesión en el repositorio.

6.6. Generación de imágenes: condicionamiento y muestreo

Generar no es recuperar un archivo ya presente en un catálogo. El sistema produce una salida condicionada por información como texto, imágenes o máscaras, usando patrones aprendidos. La instrucción orienta la salida, pero no constituye un programa geométrico que el modelo deba satisfacer exactamente.

Hay distintas familias de generadores. En una descripción introductoria de difusión, se aprende una tarea relacionada con retirar ruido a diferentes niveles. En la generación se parte de un estado ruidoso y se aplican pasos de muestreo guiados por el modelo y el condicionamiento. La formulación concreta y lo que predice la red pueden variar.

6.6.1. Recuperación, reconstrucción y generación

Recuperar selecciona un elemento existente en un catálogo. Reconstruir intenta producir una representación de una entrada que ya conocemos. Generar produce una salida bajo un modelo aprendido y un condicionamiento. Estos procedimientos pueden combinarse en un sistema, pero requieren evidencias diferentes para juzgarlos.

Un render generado puede reutilizar patrones estadísticos aprendidos sin garantizar novedad de diseño ni cumplimiento de una especificación. Tampoco hay una correspondencia automática entre cada parte de una frase y un elemento geométrico verificable de la imagen. La revisión del resultado sigue siendo una fase necesaria del proceso de diseño.

6.6.2. La intuición de retirar ruido por etapas

Durante un entrenamiento de difusión ilustrativo se toman imágenes conocidas, se perturban con ruido de magnitud controlada y se aprende una regla relacionada con esa perturbación. La red puede recibir el estado ruidoso, el nivel de ruido y una condición textual. Según la formulación, puede predecir ruido, una imagen limpia u otra cantidad equivalente bajo ciertas relaciones.

En el muestreo de imágenes nuevas no conocemos la imagen limpia objetivo. Usamos las predicciones aprendidas y una regla de actualización para recorrer estados. No es simplemente aplicar un filtro de suavizado a ruido: el modelo incorpora regularidades visuales y la condición que orienta el resultado.

Un objetivo de entrenamiento ilustrativo

Si una red g_θ predice el ruido añadido, un objetivo posible es

$$L(\theta) = \mathbb{E}\big[\|\epsilon - g_\theta(x_t, t, c)\|^2\big],$$

donde t identifica el nivel de ruido y c el condicionamiento. La esperanza resume ejemplos, niveles y realizaciones de ruido. Es una formulación didáctica de predicción de ruido; no describe el funcionamiento exacto de todos los generadores ni identifica la arquitectura de la herramienta que produjo los soportes del curso.

Aprender a predecir ruido en imágenes perturbadas y utilizar esa predicción para muestrear son etapas relacionadas, pero distintas. La actividad 5 solo construye mezclas directas. No optimiza θ, no calcula predicciones de una red y no recorre un muestreo inverso aprendido.

6.7. El proceso directo de ruido y sus supuestos

El experimento del notebook utiliza una imagen conocida x_0 y ruido ϵ:

$$x_t = \sqrt{\alpha}\, x_0 + \sqrt{1-\alpha}\, \epsilon, \qquad 0 \le \alpha \le 1. \tag{6.1}$$

Con $\alpha = 1$ queda x_0; con $\alpha = 0$, solo ϵ. Se usa una misma realización de ruido para comparar niveles, y se limita el rango únicamente para mostrar las imágenes.

Resultado: qué significan los coeficientes

Si tratamos una componente de x_0 como un valor fijo y ϵ tiene media cero y varianza uno, entonces $\mathbb{E}[x_t \mid x_0] = \sqrt{\alpha} x_0$ y $\mathrm{Var}(x_t \mid x_0) = 1 - \alpha$. Si en cambio x_0 fuera una variable de media cero y varianza uno independiente de ϵ, la varianza marginal de x_t sería $\alpha + (1 - \alpha) = 1$. Las imágenes reales del notebook no cumplen necesariamente esa última estandarización; es una explicación del esquema, no una afirmación sobre su histograma.

Conocer x_0 y el ruido añadido permite construir o deshacer esa mezcla de forma algebraica cuando procede. El problema generativo es distinto: no disponemos de la nueva imagen final que queremos producir. Necesitamos una regla aprendida y un muestreador, no solo restar una matriz de ruido que ya conocemos.

6.7.1. Cómo leer el coeficiente alpha del notebook

La fórmula utiliza raíces cuadradas. Con $\alpha = 0{,}25$, el coeficiente de imagen es 0,5 y el de ruido, $\sqrt{0{,}75} \approx 0{,}866$. No es una mezcla ordinaria con pesos 0,25 y 0,75 que suman uno. Los cuadrados

de los coeficientes suman uno, lo que permite interpretar varianzas bajo los supuestos indicados.

En formulaciones con muchos pasos, el coeficiente de una mezcla directa desde x_0 puede representar una retención acumulada de señal hasta un nivel. En el notebook `alpha` nombra directamente el coeficiente de esa fórmula simplificada. No conviene intercambiarlo sin comprobar con el parámetro de cada paso de un muestreador concreto.

La imagen se reescala aproximadamente a $[-1, 1]$ mediante $x_0 = I/127{,}5 - 1$. Esto no asegura media cero ni varianza uno en cada componente o en todo el conjunto. El ruido gaussiano generado sí tiene distribución teórica de media cero y varianza uno; una realización finita puede tener media y varianza muestrales algo diferentes.

6.7.2. Por qué no se debe recortar antes de analizar

La variable x_t puede salir de $[-1, 1]$ porque el ruido gaussiano no está limitado a ese intervalo. Para mostrarla, la práctica calcula $(x_t + 1)/2$ y limita los valores a $[0, 1]$. Esa imagen visible ya no conserva todos los valores de la mezcla. No debe utilizarse su histograma recortado para comprobar sin más las fórmulas de media y varianza del proceso anterior.

6.7.3. Una inversión algebraica y su límite

Si $\alpha > 0$ y conocemos exactamente el ruido añadido,

$$x_0 = \frac{x_t - \sqrt{1-\alpha}\,\epsilon}{\sqrt{\alpha}}.$$

Esto recupera la imagen que ya utilizamos para construir la mezcla. No demuestra que hayamos aprendido una distribución ni creado una imagen nueva. Con $\alpha = 0$ no podemos despejar x_0: la salida depende solo del ruido.

Si sustituimos el ruido real por una estimación $\widehat{\epsilon}$, el error de reconstrucción de esa fórmula es

$$\widehat{x}_0 - x_0 = \sqrt{\frac{1-\alpha}{\alpha}}(\epsilon - \widehat{\epsilon}).$$

Cuando α es pequeño, un error en la predicción de ruido puede amplificarse. La expresión ilustra una dificultad de esa inversión directa; un muestreador real usa reglas y secuencias de niveles específicas, y no se reduce necesariamente a despejar todo de una sola vez.

6.7.4. Comprobación en Python: mezclar y despejar

```
rng = np.random.default_rng(42)
x0 = np.array([-0.5, 0.0, 0.5])
epsilon = rng.normal(size=x0.shape)
alpha = 0.25
xt = np.sqrt(alpha)*x0 + np.sqrt(1-alpha)*epsilon
recuperada = (xt - np.sqrt(1-alpha)*epsilon) / np.sqrt(alpha)
assert np.allclose(recuperada, x0)
print(np.sqrt((1-alpha)/alpha))     # aproximadamente 1.732
```

Repite con la misma matriz de ruido y otro α para aislar el cambio de nivel. Si cambias simultáneamente la semilla y el nivel, la comparación mezcla dos factores.

6.8. Difusión latente, guía y reproducibilidad

En difusión latente, un codificador comprime la imagen a una representación sobre la que se realiza el proceso generativo. Un decodificador vuelve a producir píxeles. Operar en ese espacio reduce la dimensionalidad de trabajo, aunque añade el efecto de la compresión y reconstrucción.

Una semilla inicializa una fuente de aleatoriedad si el sistema la expone. No identifica por sí sola un resultado en cualquier entorno: también importan pesos, código, muestreador, parámetros y hardware. Más pasos o mayor fuerza del texto no garantizan mayor calidad o mejor cumplimiento de requisitos.

Los renders del curso se crearon con una herramienta cuyo modelo exacto y semilla no se expusieron. No afirmamos que use difusión latente ni que su generación sea reproducible a partir de una semilla inexistente. Conservamos los prompts y declaramos esos límites.

6.8.1. Espacio latente: compresión con efectos propios

Un espacio latente generativo representa una imagen mediante variables internas de un codificador y un decodificador. No es automáticamente el espacio de 512 componentes de CLIP. Ambos son representaciones aprendidas, pero cumplen tareas distintas y no se pueden intercambiar solo porque sean arreglos de números.

Trabajar con menos posiciones o componentes puede reducir el coste del proceso generativo. La reconstrucción del decodificador puede introducir diferencias y perder detalles. Si una aplicación exige cotas o texturas muy finas, hay que evaluar el resultado final en píxeles o geometría, no asumir que el proceso latente preserva todo lo necesario.

6.8.2. Guía de texto: orientación y compromiso

El condicionamiento introduce información del texto en el proceso. Algunas formulaciones combinan predicciones con y sin condición para reforzar su efecto. La intensidad de esa guía controla un compromiso y no equivale a un porcentaje de cumplimiento. Puede modificar aspecto y diversidad; aumentarla no garantiza mejorar cada requisito.

Del mismo modo, el número de pasos es una opción del muestreador. Comparar dos imágenes cambiando modelo, pasos, guía y semilla a la vez impide atribuir sus diferencias a un factor concreto. Para una demostración docente, conviene cambiar una decisión cada vez y conservar el registro de lo que el sistema expone.

6.8.3. Qué documentar para repetir una generación

Un registro útil incluye identificador y versión de pesos cuando se conocen, instrucciones, imágenes de entrada, resolución, semilla si existe, muestreador y parámetros disponibles. También conviene guardar las salidas y los criterios con que se seleccionaron. Una herramienta remota puede cambiar su implementación, por lo que repetir el texto no garantiza repetir los píxeles.

En nuestros recursos se conservan las instrucciones exactas de A y B, la herramienta utilizada y el hecho de que no se expusieron modelo ni semilla. Esa trazabilidad parcial permite describir la demostración honestamente; no permite reconstruir parámetros que no conocemos.

6.9. De la instrucción al concepto de producto

Un encargo útil describe función, contexto, aspecto y restricciones observables. Conviene distinguir requisitos visuales de los que exigen dimensiones, geometría o ensayos. Pedir «espesor suficiente» en una imagen no produce una medición de espesor.

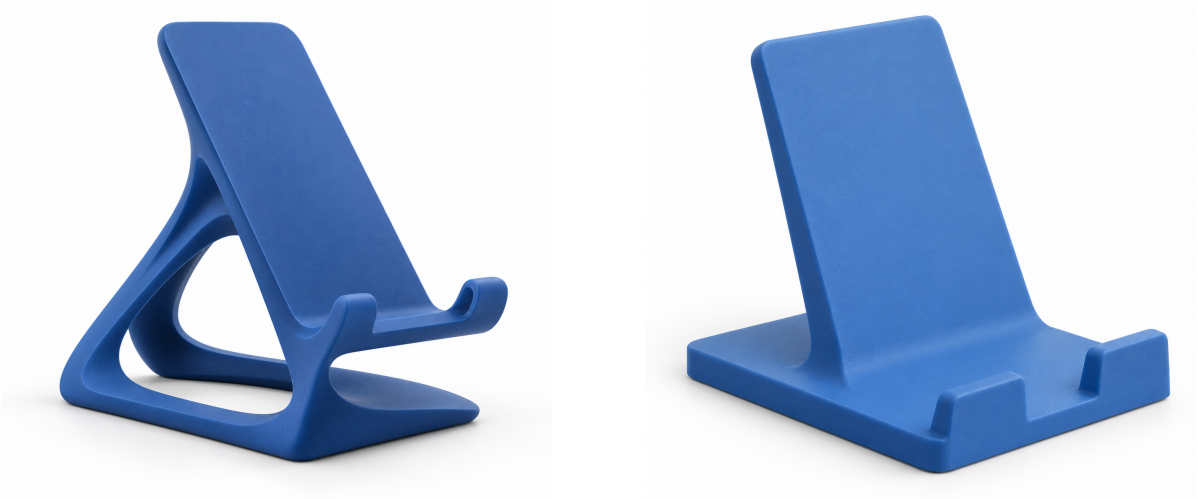


Figura 6.2: Conceptos A y B del soporte móvil. A parte de un encargo abierto; B añade restricciones visuales. Son generaciones independientes sin control de semilla, no un experimento causal que aísle el efecto del prompt.

En B se pidieron un cuerpo continuo, una base rectangular amplia, respaldo plano, dos apoyos frontales y hueco central para cable, evitando miembros muy finos y perforaciones decorativas. Podemos observar si aparecen esas características. No podemos validar solo con el render dimensiones ocultas, conectividad completa, estabilidad con una carga, tolerancias o necesidad de soportes.

Una comparación más informativa fija modelo y parámetros disponibles, cambia una familia de requisitos, obtiene varias muestras por condición y aplica la misma rúbrica sin ocultar descartes. Incluso así, la evaluación del concepto visual sigue siendo distinta de la validación de un producto fabricado.

6.9.1. Del encargo a una tabla de verificación

Antes de generar, transforma el encargo en una lista corta de requisitos. Para un soporte de móvil pueden separarse función, geometría visible, dimensiones necesarias, material y proceso. Después clasifica qué evidencia hace falta para verificar cada requisito.

Requisito	Evidencia posible en una imagen	Comprobación posterior
Dos apoyos frontales	Presencia aparente y separación visible	Geometría y cotas.
Paso de cable	Hueco visible	Dimensiones del conector y recorrido.
Un cuerpo continuo	Continuidad aparente	CAD o malla revisada.
Estabilidad con carga	No se demuestra solo con el render	Geometría, cargas y ensayo.
Fabricación prevista	Algunas zonas plantean hipótesis	Proceso, orientación y preparación.

«No verificable con esta imagen» es una conclusión útil: identifica la información que falta. No implica necesariamente que el requisito se incumpla, del mismo modo que una apariencia favorable no demuestra que se cumpla.

6.9.2. Redactar una instrucción que se pueda revisar

Una instrucción puede ordenar: objeto y uso, rasgos visibles, vista, apariencia y exclusiones. Por ejemplo: «Concepto visual de un soporte de móvil de sobremesa, vista de tres cuartos, base rectangular amplia, respaldo inclinado plano, dos apoyos frontales separados por un paso central de cable, material visual mate y fondo neutro». Cada rasgo visible ofrece algo que revisar después.

Añadir «seguro», «resistente» o «listo para imprimir» no aporta la geometría ni los ensayos necesarios para demostrarlo. Si hacen falta espesores concretos, esos valores deben trasladarse a un diseño dimensional y verificarse. La imagen puede orientar la conversación, pero no sustituye el modelo de producto.

6.9.3. Comparar varias muestras sin ocultar selección

Para estudiar una modificación de encargo, fija los parámetros disponibles y cambia una familia de requisitos. Genera varias muestras por condición cuando sea posible, revisa todas con la misma tabla y conserva también las descartadas. Informa qué variables no se controlaron. Un único par A/B sin control no permite atribuir causalmente cada diferencia al texto.

Una rúbrica visual puede puntuar presencia aparente de rasgos, claridad y coherencia de vistas. Debe mantener separados los requisitos no verificables. Promediar «atractivo» y «resistencia» en una sola nota ocultaría que la segunda propiedad no se ha medido.

6.10. Guía de trabajo con las seis actividades

1. Catálogo.
Describe qué existe y qué falta antes de buscar. Identifica dibujos procedimentales y conceptos generados. Los nombres de los archivos ayudan al alumno a leer resultados, pero no fueron entradas del codificador visual.

2. Recuperación.
Registra consulta, tres primeros resultados y cosenos. Comprueba una propiedad explícita en cada imagen y distingue relevancia general de coincidencia literal.

3. Candidatas.
Compara la taza azul con y sin su descripción adecuada. Explica qué valores permanecen y por qué cambia softmax. La suma uno no demuestra que exista una candidata verdadera.

4. Interpretación literal.
Elige dos casos de color ausente, negación, idioma o cantidad. Documenta qué se observa sin generalizar desde una única frase a todas las capacidades del modelo.

5. Ruido.
Mantén una realización de ϵ al variar el nivel. Distingue mezcla calculada, recorte para visualizar y muestreo inverso aprendido, que aquí no se ejecuta.

6. Conceptos.
Completa la tabla con evidencia y verificaciones pendientes. Reformula una familia de

requisitos y diseña una comparación más controlada. No se necesita contratar generación en vivo para entregar.

La ampliación 05B permite continuar con consultas nuevas cuando se disponga del entorno adecuado. Sus resultados deben registrarse con el mismo cuidado que los del banco. Una búsqueda acertada no demuestra conformidad del producto, y una nueva frase no equivale a entrenar de nuevo el modelo.

6.11. Errores frecuentes y aplicaciones a desarrollo de producto

Leer coseno como porcentaje.
Un valor 0,34 describe una relación geométrica en la representación, no un 34 % de cumplimiento.

Confiar en un máximo entre opciones incorrectas.
Revisa si la respuesta válida existe. Evalúa abstención con ejemplos apropiados si forma parte de la tarea.

Confundir embedding y archivo reversible.
El vector sirve aquí para comparación; no conserva necesariamente toda la geometría ni permite recuperar los píxeles.

Llamar generación a una mezcla con ruido.
Añadir ruido a una imagen conocida no ajusta un modelo ni ejecuta el proceso inverso aprendido.

Usar un render como plano.
Revisa qué dimensiones, superficies ocultas y condiciones mecánicas faltan. Traslada el concepto a geometría y evaluación de producto cuando proceda.

La recuperación multimodal puede ayudar a explorar catálogos o reunir referencias para un encargo. La generación puede ampliar alternativas visuales que luego se revisan. Para afirmar utilidad en el trabajo real conviene evaluar búsquedas representativas, registrar tiempos de revisión y comprobar propiedades decisivas, además de la apariencia de los ejemplos.

En fabricación aditiva, la revisión posterior necesita proceso, material, orientación, geometría y requisitos definidos. Dos vistas plausibles pueden ser incompatibles entre sí; una imagen única puede ocultar conexiones o espesores. Esas cuestiones se resuelven con el modelo de producto y su comprobación, no mediante una puntuación de similitud textual.

6.12. Ejercicios con soluciones razonadas

1. Dimensiones de una búsqueda

Hay 20 imágenes y siete frases, todas con vectores normalizados de 512 componentes. ¿Qué formas tienen las matrices de entrada y la tabla de similitudes?

Solución. Las imágenes forman una matriz 20×512 y los textos una matriz 7×512. El producto de la primera por la traspuesta de la segunda da 20×7. Cada celda compara una imagen con una frase.

2. Un coseno sencillo

Sean $u = (1, 0)$ y $v = (0{,}6, 0{,}8)$. Calcula sus normas y coseno. ¿El resultado significa que el 60 % de requisitos de una petición se cumple?

Solución. Ambas normas son 1 y el producto escalar, 0,6. Su coseno es por tanto 0,6. No es una fracción de requisitos verificados: para interpretar cumplimiento haría falta definir y comprobar cada condición relevante.

3. Retirar una candidata

Los logits de tres candidatas son $(3, 2{,}5, 2)$. Calcula aproximadamente el peso de la segunda con todas las opciones y después de retirar la primera.

Solución. Con las tres, el peso es aproximadamente 0,307. Con las dos restantes pasa a $e^{2{,}5}/(e^{2{,}5} + e^2) \approx 0{,}622$. Cambia el denominador y no la puntuación original de esa candidata; el aumento no prueba mayor veracidad.

4. Mezcla de señal y ruido

Para un valor fijo $x_0 = 0{,}4$ y ruido de media cero y varianza uno, usa $\alpha = 0{,}25$. ¿Cuáles son media y varianza condicionales de x_t antes de recortar?

Solución. La media es $\sqrt{0{,}25} \cdot 0{,}4 = 0{,}2$ y la varianza es $1 - 0{,}25 = 0{,}75$. La desviación es $\sqrt{0{,}75} \approx 0{,}866$. No afirmamos que la imagen visible, limitada a un rango, conserve esas propiedades.

5. Error al estimar ruido

En la inversión directa con $\alpha = 0{,}2$, una componente tiene error $\epsilon - \widehat{\epsilon} = 0{,}1$. Calcula el error correspondiente en $\widehat{x}_0$ bajo la fórmula indicada.

Solución. El factor es $\sqrt{0{,}8/0{,}2} = 2$, de modo que el error es 0,2. El cálculo explica amplificación en ese despeje; no constituye una medida del error de un generador concreto.

6. Evidencia de producto

Un render muestra un soporte con dos apoyos frontales. El encargo exige además un espesor mínimo definido y estabilidad con un móvil concreto. ¿Qué podemos concluir y qué comprobación falta?

Solución. Podemos describir la presencia aparente de dos apoyos si es visible. El espesor requiere geometría dimensional; la estabilidad, geometría, cargas y una evaluación apropiada. La imagen no demuestra esos requisitos ni permite declararlos incumplidos solo por no poder verificarlos.

Conexión con las prácticas

Las actividades 1–4 exploran catálogo, recuperación, candidatas y fallos literales; 5 ilustra el ruido directo; 6 revisa los conceptos y reformula instrucciones. La generación en vivo no es requisito de entrega. Material de la sesión en el repositorio.

Capítulo 7

Tema 6. Del problema a la evidencia

Objetivos y alcance del taller

El proyecto integra los temas anteriores en una comparación pequeña y defendible. Al terminar deberías poder conectar una decisión de producto con una salida de visión, justificar datos y particiones, registrar la selección del método y presentar una conclusión proporcional a la evidencia.
El taller dura 150 minutos y se realiza en parejas. Se reutilizan las prácticas; no se exige recoger fotografías nuevas, instalar modelos adicionales ni contratar generación. Los datos sintéticos son válidos para estudiar una hipótesis limitada si se declara su procedencia. Un resultado negativo bien explicado puede aportar más que una cifra alta sin protocolo claro.

7.1. Formular una pregunta comprobable

Un proyecto pequeño debe producir una conclusión acotada. «Usar IA en fabricación» no define una comparación. «Comprobar si reducir a la mitad la resolución mantiene la clasificación de estas familias» especifica una intervención y una respuesta observable.

La **hipótesis** expresa lo que esperamos y por qué. El **criterio de éxito** establece antes de prueba qué resultado sería suficiente. La **referencia** da contexto al resultado. Una propuesta que no mejora puede estar bien evaluada y aportar información útil para decidir qué no hacer.

7.1.1. Del encargo amplio a una decisión concreta

Un encargo inicial suele mezclar aspiraciones: reducir inspección manual, reconocer piezas o explorar alternativas. Para convertirlo en proyecto, identifica quién utilizará la salida, qué decisión tomará y qué error importa. Después elige una salida compatible: clase, caja, máscara, alarma o ranking.

«Ayudar al operario a separar tres familias en una bandeja» todavía necesita aclarar si la imagen contiene una pieza o muchas. Con una pieza completa puede bastar clasificación. Con varias quizá necesitemos localización o instancias. La tarea no se elige solo por la biblioteca que ya sabemos ejecutar.

Ejemplo: Una pregunta acotada sobre resolución

«En estos dibujos de tres familias, ¿mantiene una entrada de 16×16 un acierto de validación próximo al de 32×32, con el mismo tipo de clasificador y separación por pieza?» identifica datos, intervención y respuesta. La motivación puede ser reducir entradas y memoria. La conclusión seguirá limitada a ese generador y a las condiciones ensayadas.

7.1.2. Hipótesis, criterio y resultado son momentos distintos

La hipótesis propone una relación antes de ejecutar: «la silueta seguirá siendo suficiente al reducir resolución». El criterio establece qué consideraríamos suficiente, por ejemplo una caída máxima de cinco puntos porcentuales respecto a la referencia. Esos cinco puntos son una elección docente ilustrativa, no una tolerancia industrial universal ni una prueba formal de equivalencia.

El resultado es lo que observamos después. Puede apoyar la decisión prevista, mostrar que no se cumple o ser insuficiente para decidir. Cambiar el criterio después de ver las cifras para declarar éxito rompe la lógica de la comprobación inicial. Podemos formular una pregunta nueva, pero hay que reconocer que ya es otra investigación.

7.1.3. Un criterio de éxito necesita condiciones

Si el objetivo incluye reducir tiempo, no basta con contar parámetros. Hay que definir qué etapa se mide: preparación, entrenamiento, inferencia o procedimiento completo. Si incluye mantener calidad, conviene indicar métrica, referencia y margen permitido. Si incluye evitar omisiones, la tasa global de acierto puede ser insuficiente.

Un criterio también puede incluir una condición para no continuar: demasiadas falsas alarmas, pérdida de una clase relevante o ausencia de datos adecuados. Decidir «necesitamos otra captura» puede ser una conclusión válida cuando la evidencia muestra una limitación de la entrada.

7.1.4. La ficha inicial del proyecto

Antes de experimentar, redacta cinco frases: quién decide, qué salida necesita, qué datos utilizamos, qué cambio proponemos y cómo juzgaremos su utilidad. Los campos de `PROYECTO` en el notebook ayudan a conservar esa ficha. Los valores «Completar» son recordatorios y deben sustituirse por respuestas propias.

No se espera resolver un sistema de fabricación completo en una sesión. Es preferible comprobar bien una relación sencilla que reunir muchas técnicas sin comparación interpretable. La pregunta final debe poder responderse con las evidencias que se pueden obtener durante el taller.

7.2. Diseño de una comparación

Para interpretar un cambio conviene mantener constantes los factores que no queremos estudiar: separación de piezas, etiquetas y protocolo de evaluación. Si cambiamos resolución, número de datos y modelo a la vez, un resultado mejor no permite atribuir la mejora a uno de ellos.

Ejemplo: Comparar resolución

Usamos las mismas piezas de entrenamiento y validación para dos preparaciones: imagen original y reducida. Ajustamos cada pipeline solo con entrenamiento y comparamos en validación. Elegimos una preparación antes de prueba. Si el criterio de interés incluye tiempo o memoria, también debe medirse de forma comparable, indicando el equipo y las condiciones.

El ejemplo ejecutable del tema 6 reutiliza el generador de formas con otra realización y otras semillas. Compara mayoría con dos regresiones logísticas. Es una base de trabajo que necesita una hipótesis y un cambio propios. No es un proyecto terminado por ejecutar todas sus celdas.

7.2.1. Referencia, propuesta y factores que se mantienen

La **referencia** es el procedimiento con el que comparamos: mayoría, máscara sin limpieza, criterio de umbral previo u orden fijo de catálogo. La **propuesta** introduce un cambio justificado. Para atribuir diferencias a ese cambio, mantenemos particiones, etiquetas y protocolo tan comparables como permita la tarea.

Si comparamos resoluciones con dos pipelines, cada uno aprende su propio escalador y sus propios pesos solo con entrenamiento. No usamos los pesos de una entrada de 1024 componentes directamente sobre otra de 256. Para estudiar resolución, el procedimiento de aprendizaje se repite de manera coherente en cada representación.

7.2.2. Dos preguntas experimentales diferentes

Elegir entre varias opciones por validación pregunta «¿qué procedimiento conservamos?». Comparar después una referencia y la propuesta fijada en prueba pregunta «¿qué diferencia observamos entre estos procedimientos en datos reservados?». Consultar prueba para elegir entre todos los candidatos convierte esa comparación en otra selección.

El notebook ofrece dos valores de C, elige el primero con máximo acierto de validación si hay empate y conserva la referencia de mayoría si la mejor propuesta no la supera. Esa regla evita seleccionar automáticamente una propuesta peor. El código de partida no incluye todavía una intervención original del equipo ni una justificación de producto.

Ejemplo: Una reducción que necesita dos criterios

Una entrada pasa de 1024 a 256 valores: sus datos numéricos ocupan una cuarta parte si el tipo es el mismo. Eso no demuestra que la inferencia completa tarde exactamente cuatro veces menos. Si el proyecto afirma ahorro de memoria de entrada, el recuento basta para ese aspecto. Si afirma velocidad, debe medirla bajo condiciones comparables y declarar qué incluyó.

7.2.3. Unidad independiente y muestra de evaluación

La unidad que esperamos encontrar nueva determina cómo separar. En el ejemplo hay 180 piezas y tres capturas por pieza. Las 108 imágenes de prueba corresponden a 36 piezas. Para calcular acierto por imagen dividimos por 108; para razonar sobre variación entre piezas no tratamos esas capturas como 108 piezas nuevas.

Podemos definir una medida por pieza, por ejemplo la proporción de piezas cuyas tres capturas se clasifican correctamente. Esa medida responde a otra pregunta y debe declararse antes de utilizarla para elegir. No se obtiene sumando indistintamente imágenes y piezas en el mismo denominador.

7.2.4. Registro de intentos y comparación emparejada

Conserva una fila por configuración: qué cambió, qué se mantuvo, qué resultado de validación dio y qué decisión provocó. Si un intento no mejora, no desaparece del registro. Una exploración grande sobre pocas muestras puede adaptarse a sus peculiaridades, aunque no actualice directamente los pesos con validación.

Para comparar dos modelos en las mismas imágenes, podemos contar cuatro situaciones: ambos aciertan, solo acierta A, solo acierta B y ninguno acierta. Si sus recuentos son $n_{11}, n_{10}, n_{01}, n_{00}$

y suman n, la diferencia de aciertos es

$$\mathrm{acc}_B - \mathrm{acc}_A = \frac{n_{01} - n_{10}}{n}.$$

Los casos en que ambos hacen lo mismo se cancelan al restar. Esta tabla muestra dónde se produce la diferencia, pero no demuestra significación estadística ni elimina dependencias entre capturas.

7.2.5. Comprobación en Python: una diferencia con sus casos

La celda usa predicciones ilustrativas, no resultados del proyecto.

```
import numpy as np
y = np.array([0, 0, 1, 1, 2, 2])
pred_a = np.array([0, 1, 1, 0, 2, 0])
pred_b = np.array([0, 0, 1, 0, 2, 2])
a = pred_a == y
b = pred_b == y
solo_a = np.sum(a & ~b)
solo_b = np.sum(~a & b)
print(int(solo_a), int(solo_b))       # 0, 2
print(round(float(b.mean()-a.mean()), 3))  # 0.333
assert np.isclose(b.mean()-a.mean(), (solo_b-solo_a)/len(y))
```

Comentar las dos imágenes corregidas es más informativo que mostrar solo la resta de porcentajes. Si una mejora cambia también condiciones de captura o datos, hay que declarar esos cambios antes de atribuirle una causa única.

7.3. Cuatro rutas y evidencia mínima

En clasificación se necesita una comparación y matriz por clase. En detección o segmentación se debe explicitar la salida y el protocolo espacial. En anomalías hay que fijar qué significa alarma y cómo se cuentan errores. En búsqueda se define relevancia antes de mirar el ranking y se reconocen consultas sin respuesta correcta.

La revisión de conceptos generados puede complementar el proyecto, pero no sustituye una evaluación de visión. Del mismo modo, mostrar tres imágenes bien elegidas no estima un rendimiento: hace falta indicar cómo se seleccionaron y qué casos quedaron fuera.

7.3.1. Ruta A: clasificación

Elige una intervención sobre resolución, regularización o cantidad de piezas de entrenamiento. Mantén juntas las capturas de cada pieza. Informa recuentos por clase, comparación en validación, selección fijada y matriz de prueba. Examina errores y, si no existen, dos límites concretos de la cobertura.

Si reduces datos de entrenamiento, selecciona piezas completas y conserva validación y prueba. Una selección que retire preferentemente las piezas difíciles cambia algo más que el tamaño. Si el objetivo es estudiar cantidad de datos, explica cómo se eligió el subconjunto y qué variación puede introducir esa elección.

7.3.2. Ruta B: detección o segmentación

Decide primero qué salida te interesa. Para material visible, compara máscaras; para localizar cada objeto, evalúa cajas con parejas uno a uno; para contar, examina también contacto y fragmentación. Puedes comparar dos filtros de área o una máscara sin limpieza con otra filtrada usando validación.

Conserva conectividad, convenciones geométricas y umbral de IoU de evaluación, o declara cuál cambias y por qué. Presenta al menos las tres escenas anotadas indicadas por la ruta y los fallos relevantes. La precisión de cajas en un punto de operación no se llama mAP. Si dos piezas se fusionan, no ocultes ese error detrás de una buena IoU semántica.

7.3.3. Ruta C: anomalías

Mantén la referencia normal ajustada solo con correctas de entrenamiento. Compara un criterio de umbral o una agregación del mapa y elige con validación. Si cambias el modo de puntuar, el umbral anterior deja de tener necesariamente el mismo significado: cada alternativa necesita una selección coherente.

Informa TP, FP, FN, TN, denominadores y composición del conjunto. Desglosa rayas y poros, y distingue alarma por imagen de localización espacial. Una modificación que detecta más poros pero dispara alarmas sobre correctas exige explicar ambas consecuencias. No presentes precisión de una muestra equilibrada como estimación directa de planta.

7.3.4. Ruta D: búsqueda visual

Define qué imágenes son relevantes para cada consulta antes de ver el ranking. Una referencia sencilla puede ser devolver siempre la primera imagen del catálogo. Compara con CLIP sobre consultas reservadas después de fijar la forma de consulta. La independencia de consultas parecidas puede ser limitada si comparten la misma intención y el mismo catálogo diminuto.

Para una consulta con respuesta válida, el acierto de primera posición vale 1 si el primer resultado pertenece al conjunto relevante y 0 en otro caso. Para una consulta sin respuesta válida, un sistema que siempre devuelve algo no puede acertar una imagen relevante. Conviene informar esos casos por separado o definir explícitamente una regla de abstención evaluable.

Ejemplo: Cobertura y acierto de búsqueda

Hay diez consultas, de las cuales ocho tienen alguna respuesta relevante en el catálogo. CLIP acierta la primera posición en seis de esas ocho. El acierto sobre consultas con respuesta es $6/8 = 75\,\%$; la cobertura del catálogo es $8/10 = 80\,\%$. Si el criterio exige una imagen válida para todas las consultas y no existe abstención, el acierto global es $6/10 = 60\,\%$. Ninguna cifra sustituye a las otras: hay que declarar qué se cuenta.

La revisión de renders puede complementar una ruta con requisitos visuales y comprobaciones pendientes. No basta generar o seleccionar una imagen atractiva para demostrar rendimiento de una tarea de visión.

7.3.5. Cómo elegir una ruta que quepa en el tiempo

Escoge aquella cuya pregunta puedas explicar y cuya evaluación puedas ejecutar con los materiales existentes. Reserva tiempo para interpretar y redactar: cambiar de modelo al final suele dejar sin evidencia suficiente la nueva propuesta. Si eliges B, C o D, reutiliza su notebook

y adapta las secciones de encargo, decisiones e informe; no es obligatorio ejecutar además el ejemplo de clasificación.

7.4. Cierre de decisiones y reproducibilidad

El notebook deja `EVALUAR_PRUEBA=False` por defecto. Ese interruptor ayuda a trabajar por fases; no oculta técnicamente los datos ni impide consultarlos. La separación es una disciplina metodológica que debe quedar documentada.

Reproducir requiere registrar datos, particiones, preparación, parámetros y versiones. Una semilla ayuda, pero no sustituye esos registros. Reiniciar y ejecutar en orden permite detectar dependencias ocultas del estado del notebook. Repetir sin cambios para comprobar ejecución no es lo mismo que ajustar parámetros hasta obtener una cifra de prueba mejor.

7.4.1. Qué significa cerrar decisiones

Anota el modelo o procedimiento, preparación, parámetros, criterio de selección, partición y fecha de cierre. Después evalúa la versión fijada. Si descubres un error de código que invalida la medición, corrígelo y explica qué resultados deben descartarse. Corregir un fallo no justifica ocultar que ya se observaron datos reservados.

Si una observación de prueba inspira una mejora, puede abrir una fase de desarrollo nueva. Sus resultados sobre ese mismo conjunto sirven para explorar, pero una estimación independiente de la nueva selección necesita otros datos apropiados. La distinción depende de cómo se utilizó la información, no del nombre que tenga una variable.

7.4.2. El interruptor de la plantilla y los estados exportados

Con `EVALUAR_PRUEBA=False`, el ejemplo deja sin calcular predicciones y resumen de prueba. La exportación guarda los resultados disponibles en `resultados/tema6/pendiente/`. Tras evaluar, utiliza `resultados/tema6/evaluado/`. Los nombres ayudan a evitar mezclas, pero el estado «evaluado» no demuestra por sí solo que se haya respetado el protocolo.

La celda de exportación crea `validacion.csv`, `particiones.csv`, `registro.json` e `informe_borrador.md`; cuando hay predicciones, añade `predicciones_prueba.csv`. El informe mantiene una marca de borrador y necesita respuestas propias. Volver a exportar el mismo estado puede sobrescribir sus archivos, por lo que conviene conservar las versiones que se quieran comparar.

7.4.3. Procedencia, versiones y huellas

El registro del ejemplo incluye versiones de herramientas y una huella de los arreglos sintéticos. La huella permite comprobar identidad de esos bytes, no demostrar que los datos sean representativos ni que la separación sea correcta. Las particiones identifican qué imagen y pieza pertenecen a cada conjunto; son necesarias para reconstruir la comparación.

Con datos externos, documenta origen, condiciones de uso, etiquetas y transformaciones, sin incorporar datos personales a la entrega pública. Para reproducir importa también el estado de la preparación: una media aprendida con otra versión de entrenamiento cambia la entrada aunque el clasificador conserve su nombre.

7.4.4. Ejecutar desde un estado limpio

Los notebooks conservan variables entre ejecuciones. Puede quedar un modelo antiguo junto a una tabla reciente, o una figura de otra configuración. Reiniciar el kernel y ejecutar por orden comprueba si el documento final contiene las dependencias necesarias. También permite detectar resultados que solo aparecen por haber ejecutado celdas en un orden no documentado.

Repetir la versión fijada para verificar que funciona no convierte la comprobación en una nueva prueba independiente. Si aparecen diferencias numéricas, investiga semillas, datos, versiones y operaciones. No cambies selectivamente la configuración para conservar solo la repetición más favorable.

7.4.5. Medir tiempo de manera interpretable

Si el proyecto incluye tiempo, indica equipo, número de imágenes, etapa medida y si se incluyeron carga de archivos y preparación. Repetir una operación puede revelar variabilidad de medida. Conviene distinguir una primera ejecución con inicializaciones de las posteriores y conservar una comparación equivalente entre métodos.

Una media o mediana de varios tiempos resume esas ejecuciones bajo esas condiciones. No garantiza el mismo rendimiento en Colab, otro ordenador o un sistema desplegado. Una mejora de velocidad que empeora una clase importante requiere informar el compromiso, no solo el tiempo ganado.

7.5. Informar sin exagerar

Un informe breve debe permitir seguir la cadena: problema, datos, referencia, cambio, evaluación y conclusión. Los números tienen que poder vincularse con tablas o figuras y con las condiciones bajo las que se obtuvieron.

Observación, explicación y comprobación futura

«El poro no generó alarma» es una observación. «Quizá su contraste se mezcla con la textura normal» es una hipótesis explicativa. «Compararemos poros de distintos contrastes en nuevas piezas» es una comprobación futura. Separar los tres niveles evita presentar una causa posible como un hecho demostrado.

Si no hay errores, hay que describir lo que no se probó: otro producto, otra cámara, nuevos lotes o defectos fuera del generador. No hace falta inventar un caso fallido, ni sería correcto concluir que el sistema es perfecto.

7.5.1. Estructura de un informe de una página

El informe final es una síntesis, no una transcripción de todas las salidas. Puede organizarse en cinco párrafos: encargo e hipótesis; datos y separación; referencia y cambio; resultados con evidencias vinculadas; conclusión, límites y siguiente paso. Una tabla compacta o una figura pueden sustituir listas largas de cifras.

Cada métrica debe indicar conjunto y denominador. «Acierto 0,91» necesita aclarar si procede de validación o prueba, cuántos ejemplos y cuántas piezas lo sostienen y qué versión del procedimiento se evaluó. El notebook y los archivos complementarios conservan el detalle que no cabe en una página.

Ejemplo: Una conclusión proporcionada

«En esta muestra sintética, la preparación reducida cumple el margen de acierto fijado frente a la original y utiliza una cuarta parte de valores de entrada. Esto justifica explorar esa preparación en nuevas capturas. No hemos medido tiempo completo ni evaluado fotografías, por lo que no afirmamos un ahorro operativo ni rendimiento industrial». El ejemplo muestra cómo redactar; no informa de un resultado obtenido en la plantilla.

7.5.2. Analizar dos casos sin inventar causas

Para cada caso, identifica qué se observa en la entrada, qué produjo el sistema y qué referencia se utilizó. Después propone una explicación posible y una comprobación que permita distinguirla de otras. Una placa confundida al estar desplazada puede sugerir sensibilidad a posición, pero también puede estar parcialmente recortada: hace falta separar esas posibilidades.

Si no hay errores en prueba, comenta dos condiciones ausentes o límites del protocolo, como otro material o una captura fuera de las variaciones del generador. Un ejemplo transformado puede ilustrar una fragilidad, pero debe figurar como diagnóstico posterior y no como una nueva muestra independiente.

7.5.3. Diferencias pequeñas y lenguaje de equivalencia

Supongamos que A acierta 100 de 108 imágenes y B, 99. La diferencia es de un acierto, aproximadamente 0,93 puntos porcentuales. No basta para afirmar que «son equivalentes» ni que «A siempre es mejor». Podemos decir cuál fue la diferencia observada y si satisface el criterio práctico predefinido, indicando el tamaño y alcance de la muestra.

Una afirmación formal de equivalencia o superioridad necesitaría un diseño estadístico adecuado, un margen justificado y el tratamiento de dependencias. El taller se centra en una comparación descriptiva bien documentada. No necesita añadir una prueba estadística que no pueda interpretarse correctamente.

7.5.4. La propuesta puede perder y el proyecto estar bien resuelto

Si reducir resolución destruye un detalle necesario, el resultado descarta una opción bajo las condiciones ensayadas. Si un filtro borra poros, revela un compromiso relevante. Si CLIP recupera imágenes relacionadas pero incumple negación, el análisis identifica qué propiedad no está verificada. En todos los casos, el valor del proyecto está en la claridad de la pregunta y de la evidencia.

No se debe seleccionar solo el resultado favorable ni sustituir una conclusión por «hace falta más información» sin precisar cuál. Un siguiente paso útil identifica qué nuevos casos recoger, qué factor variar o qué medida añadir y qué duda resolvería.

7.6. Evaluación del proyecto y defensa

La rúbrica asigna 25 % a problema y datos, 25 % a método, 30 % a evaluación y errores y 20 % a claridad y reproducibilidad. Cada dimensión se puntúa de 0 a 3. Si sus niveles son p, m, e, r, la nota orientativa sobre 10 es

$$10\left(0{,}25\frac{p}{3} + 0{,}25\frac{m}{3} + 0{,}30\frac{e}{3} + 0{,}20\frac{r}{3}\right). \tag{7.1}$$

Los máximos producen 10. La ponderación dentro de la asignatura corresponde a la coordinación docente.

La defensa de cinco minutos dedica uno a problema y datos, uno a método, dos a resultados y límites, y uno a conclusión y siguiente paso. El notebook basta como soporte. La revisión entre equipos debe aportar una fortaleza sustentada, una limitación y una pregunta concreta sobre datos o evaluación.

7.6.1. Cómo leer los niveles de la rúbrica

Cada dimensión se evalúa de 0 a 3. El nivel 0 significa que no hay evidencia suficiente para valorarla. El nivel 1 es inicial; el 2, adecuado; el 3, sólido. Los pesos son los del enunciado vigente del proyecto; el manual los explica y no añade una ponderación distinta para la asignatura.

Problema y datos, 25 %**.**
Para un trabajo sólido, concreta usuario, decisión, criterio previo, unidad de separación y cobertura. No basta decir que se han usado «imágenes de piezas».

Método, 25 %**.**
Explica referencia, hipótesis y cambio. Muestra cómo se seleccionaron las decisiones en validación y qué intentos se registraron.

Evaluación y errores, 30 %**.**
Declara protocolo, denominadores y límites. Una fuga de información impide presentar la evaluación como sólida hasta corregir el procedimiento o reconocer que se necesita otra prueba.

Claridad y reproducibilidad, 20 %**.**
Vincula cifras, figuras y configuración. Entrega un notebook ejecutable, un informe claro y una comprobación futura concreta.

7.6.2. Comprobación en Python: cálculo de la nota orientativa

La celda comprueba la ponderación, no asigna automáticamente niveles al trabajo.

```
pesos = np.array([0.25, 0.25, 0.30, 0.20])
niveles = np.array([3., 2., 2., 3.])
nota = 10 * np.sum(pesos * niveles / 3)
print(round(float(nota), 2))       # 8.17
assert np.isclose(pesos.sum(), 1)
assert np.isclose(10 * np.sum(pesos * np.ones(4)), 10)
```

En el último control, el vector de unos representa nivel/3 = 1 en las cuatro dimensiones. El cálculo no sustituye el juicio docente sobre la evidencia aportada.

7.6.3. Preparar una defensa de cinco minutos

En el primer minuto, presenta usuario, decisión, datos y límite de alcance. En el segundo, muestra referencia y cambio. En los dos siguientes, presenta la tabla o figura principal y dos errores o limitaciones. En el último, responde a la pregunta del proyecto y propone el siguiente paso. No necesitas otra presentación si el notebook permite seguir ese recorrido.

Evita ocupar la defensa en instalar paquetes o ejecutar un entrenamiento largo. Las salidas ya guardadas permiten discutir resultados. Si alguien pregunta por una cifra, debe poder encontrarse

su procedencia. Una duda no resuelta puede declararse y convertirse en una comprobación futura; no hace falta improvisar una certeza.

7.6.4. Revisión entre equipos

Una revisión útil cita una decisión respaldada, una limitación que falta y una pregunta concreta sobre datos o evaluación. Por ejemplo: «La separación por pieza está comprobada; falta indicar cuántas piezas reales sostienen la tasa; ¿cómo cambiaría la evaluación si las capturas proceden del mismo día?». La crítica se dirige a la evidencia y al método, no a la persona.

El bloque común de 35 minutos admite hasta cinco equipos con cinco minutos de defensa y dos de preguntas cada uno. Si hay más equipos, el enunciado propone estaciones simultáneas: dos rondas de 15 minutos y cinco para puesta en común, con revisiones asignadas por el profesor.

7.7. Organización del taller de 150 minutos

El reparto de sesión contempla 15 minutos de encargo, 20 de datos, 30 de experimentos, diez de descanso, 15 de evaluación, 15 de informe, 35 de defensas o estaciones y diez de cierre. Los tiempos orientativos de las celdas forman parte de esos bloques y se ajustan al trabajo de cada pareja.

Al cerrar el bloque de experimentos debería existir una decisión registrada. En evaluación se ejecuta esa versión. En informe se interpretan resultados, no se abre una nueva búsqueda de parámetros. Si algo impide evaluar, se describe el estado pendiente y lo que falta, sin completar cifras inventadas.

7.8. Guía de trabajo con las seis actividades

1. Encargo.
Completa `PROYECTO` y el criterio previo. La hipótesis debe nombrar un cambio y una consecuencia observable.

2. Datos.
Describe origen, etiquetas, unidad y particiones. Revisa recuentos por clase y pertenencia de piezas; para otras rutas conserva el mismo razonamiento aunque cambien las variables.

3. Comparación.
Ejecuta referencia y propuesta con el cambio propio. Registra intentos y resultados de validación. No confundas los candidatos ya incluidos con una intervención original del equipo.

4. Prueba.
Registra la decisión cerrada y activa el bloque cuando corresponda. Conserva la configuración seleccionada y declara si surgió una incidencia que invalide alguna medición.

5. Informe.
Analiza dos casos o límites, completa el borrador y revisa que tablas y figuras correspondan a la misma ejecución. Descarga notebook y archivos cuando trabajes en Colab.

6. Defensa.
Prepara el recorrido de cinco minutos y una revisión concreta de otro equipo. Usa la rúbrica para comprobar qué evidencia sostiene cada afirmación.

7.9. Ejercicios con soluciones razonadas

1. Reformular un encargo

Un equipo propone «usar IA para mejorar la calidad de impresión». Reformula la idea como una pregunta viable con los materiales del curso.

Solución orientativa. «En las superficies sintéticas del tema 4, ¿cómo cambia el número de poros omitidos y falsas alarmas al comparar dos criterios de umbral elegidos en validación, manteniendo fija la referencia normal?». Se acota salida, datos y comparación, y no se afirma medir toda la calidad de impresión real.

2. Imágenes y piezas

Hay 120 piezas con cuatro capturas cada una. Se reserva por pieza el 20 % para prueba. ¿Cuántas piezas e imágenes contiene? ¿Qué denominador usarías para acierto por imagen?

Solución. Son 24 piezas y 96 imágenes. El acierto por imagen usa 96 como denominador. Eso no convierte las 96 capturas en piezas independientes; cualquier análisis que requiera esa independencia debe considerar su estructura por pieza.

3. Comparación emparejada

En 40 imágenes, ambos modelos aciertan 28; solo A acierta cuatro; solo B acierta seis; ninguno acierta dos. Calcula ambos aciertos y la diferencia.

Solución. A acierta $32/40 = 0{,}80$ y B $34/40 = 0{,}85$. La diferencia es $(6 - 4)/40 = 0{,}05$, cinco puntos porcentuales. Es una diferencia observada en esa muestra, no una garantía de superioridad general.

4. Criterio previo y selección

Se fija que la propuesta no pierda más de cinco puntos de acierto frente a referencia. En validación pierde tres y en prueba pierde siete. ¿Qué debe informar el equipo?

Solución. La propuesta satisfizo el margen en validación, pero no en prueba. Debe informar ambos resultados y concluir que el criterio de calidad no quedó satisfecho en esa evaluación final. No se cambia el margen a siete después para declarar éxito. Una nueva propuesta necesita desarrollo y otra evaluación independiente.

5. Cobertura de búsqueda

De 12 consultas, nueve tienen respuesta válida en el catálogo. El primer resultado es relevante en seis de esas nueve. Calcula cobertura y acierto condicionado a que exista respuesta.

Solución. La cobertura es $9/12 = 75\,\%$ y el acierto sobre consultas con respuesta es $6/9 \approx 66{,}7\,\%$. Si se exige una imagen relevante en todas sin abstención, el acierto global es $6/12 = 50\,\%$. Declarar solo 66,7 % ocultaría las tres consultas sin respuesta.

6. Aplicar la rúbrica

Un proyecto recibe niveles $(2, 3, 2, 3)$ en problema/datos, método, evaluación y claridad. Calcula la nota orientativa con los pesos del enunciado.

Solución. La nota es $10(0{,}25 \cdot 2/3 + 0{,}25 \cdot 3/3 + 0{,}30 \cdot 2/3 + 0{,}20 \cdot 3/3) \approx 8{,}17$. No es un promedio simple de porcentajes de acierto, sino una suma ponderada de niveles de evidencia.

7.10. Cierre del curso: elegir, comprobar y comunicar

El recorrido empieza por reconocer qué representan los píxeles y termina en una decisión sostenida por datos. Clasificación, segmentación, anomalías y modelos multimodales producen salidas distintas; su utilidad depende de que correspondan a la pregunta de producto y se evalúen con el protocolo adecuado.

Ante una tarea nueva, vuelve a esa cadena: propiedad observable, captura y datos, salida necesaria, referencia sencilla, comparación, evaluación reservada y conclusión. Si falta información visual o una medida física, identifícala antes de ampliar el modelo. El proyecto final debe dejar claro qué permite decidir la evidencia y qué sigue pendiente de comprobar.

Conexión con las prácticas

Las seis actividades acompañan encargo, datos, comparación, prueba, informe y defensa. La exportación guarda borradores y resultados disponibles en carpetas distintas según el estado. El informe necesita respuestas y conclusiones propias; el programa no las redacta por el equipo. Material de la sesión en el repositorio.

Apéndice A

Notación y glosario de consulta

Símbolo o término	Significado en este manual
$I(i, j, c)$	Valor de imagen en fila i, columna j y canal c.
x_i, y_i	Entrada y etiqueta del ejemplo i.
θ	Conjunto de parámetros aprendidos del modelo.
w, b	Pesos y sesgo de una transformación afín.
μ, σ	Media y desviación de una referencia; deben interpretarse con su población o muestra.
$\|u\|$, $u \cdot v$	Norma euclídea y producto escalar.
η, λ	Tamaño de paso y peso de una penalización, respectivamente.
t	Umbral de decisión; su significado depende de la puntuación que se compara.
α	Coeficiente de señal en la ilustración de ruido del tema 5.
TP, FP, FN, TN	Aciertos positivos, falsas alarmas, omisiones y aciertos negativos. Siempre se define la clase positiva.
ROI	Región de interés donde se inspecciona o calcula una puntuación.
Lote / epoch	Grupo de ejemplos de una actualización / recorrido de entrenamiento.
Parámetro	Valor aprendido, como un peso.
Hiperparámetro	Decisión de configuración o entrenamiento, como regularización o número de neuronas.
Calibración probabilística	Correspondencia entre probabilidades predichas y frecuencias observadas bajo una evaluación adecuada.
Calibración geométrica	Relación validada entre coordenadas de imagen y geometría o medidas físicas. No es lo mismo que calibración probabilística.
Dominio	Condiciones y tipo de datos de una tarea: producto, cámara, estilo, entorno, etc.
Fuga de información	Uso en desarrollo de información que debería quedar reservada para evaluación.
Representación / embedding	Vector de características con el que se realizan comparaciones o predicciones.
Inferencia	Aplicación del modelo fijado a entradas.
Abstención	Decisión de no asignar una respuesta cuando se cumple un criterio definido; requiere diseño y evaluación.
Referencia de evaluación	Etiquetas o anotaciones con las que se compara una predicción. No son infalibles: pueden contener error.

Apéndice B

Ejercicios de comprensión

Los ejercicios no requieren entrenamiento adicional. Conviene justificar la respuesta antes de consultar el apéndice siguiente.

1. Una imagen tiene forma $(120, 200, 3)$ y tipo `uint8`. ¿Cuántos valores contiene y cuántos bytes ocupan sin compresión? ¿Qué forma tiene el recorte `[10:30,40:90]`?

2. Calcula la normalización a $[0, 1]$ de un canal con valor 128. ¿Cambia el número de píxeles? ¿Qué riesgo tiene sumar brillo con enteros de 8 bits?

3. Un ancho de 80 mm ocupa 800 píxeles. ¿Cuánto ocupa aproximadamente una marca de 0,3 mm? ¿Basta para garantizar detección?

4. Para $u = (3, 4)$ y $v = (0, 5)$, calcula normas, producto escalar y coseno. ¿Qué cambia al normalizar ambos?

5. Una pieza tiene seis fotografías. ¿Es adecuada una partición que reserva tres para entrenamiento y tres para prueba si se pretende generalizar a piezas nuevas?

6. Calcula softmax para $(\log 2, 0)$. ¿Qué sucede al sumar 100 a ambas puntuaciones? ¿Y al eliminar una candidata?

7. Una predicción asigna 0,8 a la clase correcta y otra 0,2. Calcula sus pérdidas $-\log p$. ¿Cuál penaliza más y por qué?

8. Dos cajas de área 100 cada una tienen intersección 60. Calcula IoU y Dice. ¿Por qué no pueden contarse dos aciertos si dos predicciones se emparejan con la misma referencia?

9. Tres piezas en contacto forman una máscara casi perfecta y una sola componente. ¿Qué objetivo puede estar bien resuelto y cuál puede fallar?

10. En una prueba aparecen TN=80, FP=20, FN=5 y TP=15. Calcula acierto, FPR, recall y precisión. Identifica el denominador de cada una.

11. En el caso anterior, calcula $FP + 5FN$ y $5FP + FN$. ¿Cuál penaliza más ese mismo resultado? ¿Significa que exista un umbral mejor sin examinar puntuaciones?

12. Si una posición tiene media 0,4, desviación 0,02 y suelo 0,025, calcula su desviación estandarizada absoluta para un valor 0,5. ¿Es una probabilidad de defecto?

13. En 1000 piezas con 2 % de defectos, usa recall hipotético 80 % y FPR 10 % para calcular TP, FP y precisión esperados. ¿Por qué puede haber más alarmas falsas que correctas?

14. ¿Puede un peso softmax alto entre «taza roja» y «taza verde» demostrar que una taza azul es verde? Explica qué parte de la operación depende de las candidatas.

15. En la mezcla de ruido, ¿qué ocurre con $\alpha = 0$ y $\alpha = 1$? ¿Por qué no estamos ejecutando el proceso generativo inverso?

16. Un equipo prueba cinco modelos, observa prueba y selecciona el mejor según ella. ¿Qué problema aparece y cómo debería evaluar una propuesta posterior?

17. Una propuesta tiene niveles 3, 2, 2 y 3 en las cuatro dimensiones de la rúbrica. Calcula la nota orientativa sobre 10.

18. Un modelo no falla en 20 imágenes de dos piezas repetidas. ¿Qué afirmaciones sobre independencia y rendimiento futuro deben evitarse?

Apéndice C

Soluciones comentadas

1. Hay $120 \cdot 200 \cdot 3 = 72\,000$ valores y bytes. El recorte tiene 20 filas, 50 columnas y tres canales: $(20, 50, 3)$. El archivo comprimido puede ocupar otra cantidad.

2. $128/255 \approx 0{,}502$. No cambia la resolución. La suma en `uint8` puede desbordar y volver al inicio del rango; se convierte a decimal antes de operar y se decide cómo tratar la saturación.

3. La escala idealizada es 10 píxeles/mm: la marca ocuparía unos 3 píxeles. No garantiza detección; importan contraste, enfoque, ruido, forma y posición en la rejilla.

4. Ambas normas valen 5, el producto es 20 y el coseno $20/25 = 0{,}8$. Tras normalizar, el producto escalar es 0,8. Las direcciones se conservan y las longitudes pasan a 1.

5. No estima adecuadamente el objetivo planteado: las fotografías comparten pieza. Deben separarse piezas completas, y considerar otros grupos si el uso exige lotes o condiciones nuevos.

6. Las exponenciales son $(2, 1)$ y los pesos $(2/3, 1/3)$. Sumar la misma constante no los cambia. Si queda una sola candidata, su peso es 1 aunque su descripción sea incorrecta.

7. $-\log 0{,}8 \approx 0{,}223$ y $-\log 0{,}2 \approx 1{,}609$. La segunda asigna menos peso a la etiqueta verdadera y recibe mayor penalización.

8. Unión $100 + 100 - 60 = 140$; IoU $60/140 \approx 0{,}429$; Dice $120/200 = 0{,}6$. El emparejamiento uno a uno impide premiar duplicados: la segunda caja debe quedar como predicción sin pareja.

9. Puede resolverse bien pieza/fondo y fallar el recuento o la separación de instancias. Una componente no es necesariamente un objeto físico.

10. Total 120; acierto $95/120 \approx 0{,}792$. FPR $20/100 = 0{,}20$; recall $15/20 = 0{,}75$; precisión $15/35 \approx 0{,}429$. Los denominadores son todas las imágenes, las correctas, los defectos y las alarmas, respectivamente.

11. Las penalizaciones son 45 y 105. La segunda da mayor peso a los 20 FP observados. No informa por sí sola de qué umbrales alternativos existen ni de si mejorarían: necesitamos sus puntuaciones y validación.

12. El denominador es $\max(0{,}02, 0{,}025) = 0{,}025$. El resultado es $0{,}1/0{,}025 = 4$. Es alejamiento respecto a la referencia, no una probabilidad de defecto.

13. Hay 20 defectos y 980 correctas. TP esperado 16 y FP esperado 98. Precisión esperada $16/114 \approx 14{,}0\,\%$. La población de correctas es mucho mayor; una tasa moderada de falsas alarmas puede dominar el total.

14. No. Softmax solo reparte peso entre opciones dadas; retirar «azul» no hace verdadera otra descripción. Los cosenos existentes permanecen, pero cambia el denominador de la normalización.

15. Con cero queda ruido y con uno queda la imagen conocida. No hemos aprendido a inferir una imagen nueva desde un estado ruidoso: construimos una mezcla usando una imagen que ya teníamos.

16. Prueba participa en la selección y deja de ser una estimación independiente del procedimiento seleccionado. La nueva propuesta debe desarrollarse con datos de desarrollo y evaluarse en otros datos reservados y pertinentes.

17. $10(0{,}25 + 0{,}25 \cdot 2/3 + 0{,}30 \cdot 2/3 + 0{,}20) = 8{,}167$ aproximadamente. La rúbrica no identifica esa nota con acierto del modelo.

18. No son 20 piezas independientes. No debe afirmarse ausencia general de fallos, ni aplicarse sin más una estimación de incertidumbre que suponga independencia entre todas las imágenes. Deben evaluarse piezas nuevas representativas.

Apéndice D

Lecturas y trazabilidad del material

D.1. Lecturas para ampliar

Estas fuentes permiten profundizar; el manual puede leerse sin abrirlas. No se reproducen páginas de libros ni figuras de terceros.

Visión artificial general
Richard Szeliski, *Computer Vision: Algorithms and Applications*, segunda edición. Formación de imagen, procesamiento, segmentación y reconocimiento.

Aprendizaje profundo
Christopher M. Bishop y Hugh Bishop, *Deep Learning: Foundations and Concepts*. Redes, optimización y modelos de visión.

CLIP
Radford y colaboradores, *Learning Transferable Visual Models From Natural Language Supervision* (2021). Para el modelo usado: `https://huggingface.co/openai/clip-vit-base-patch32`.

Difusión latente
Rombach y colaboradores, *High-Resolution Image Synthesis with Latent Diffusion Models* (2022): `https://arxiv.org/abs/2112.10752`.

Herramientas de las prácticas
Documentación oficial de NumPy, Pillow, scikit-learn, scikit-image y Transformers/CLIP. Las APIs y los entornos deben comprobarse para la versión concreta utilizada.

D.2. Correspondencia de archivos

Los materiales del alumno se organizan en las carpetas 01 a 06 del repositorio vision-ia-producto. Cada una incluye el PDF de la sesión y su notebook o taller. Los fuentes Beamer y las guías docentes permanecen en el directorio privado, fuera de ese repositorio. Este manual también es material privado de apoyo.

Las figuras incluidas se han copiado al subdirectorio `figuras/` del manual para que el proyecto LaTeX no dependa de rutas externas. Son originales del curso: dibujos y gráficos obtenidos mediante código, y dos conceptos generados con la herramienta de imágenes. Las tablas de resultados ilustran las ejecuciones de referencia de los notebooks; no son estimaciones de funcionamiento industrial.

En el tema 5, los archivos de procedencia registran modelo, revisión y huellas de imágenes y vectores; los prompts de A y B se conservan íntegros. No se distribuyen aquí pesos de modelos. Una actualización de los notebooks debe acompañarse de una revisión de las explicaciones y figuras que dependan de ella.